

Thary: Sistema de apoyo a la decisión basado en predicción de demanda para la gestión de inventarios

Danna Andrea Rodriguez Galarcio

Trabajo de grado presentado para optar al título de ingeniero de sistemas

Asesor:

Ivan Mauricio Cabezas



Universidad de San Buenaventura
Facultad de Ingeniería (Cali)
Ingeniería Sistemas
Santiago de Cali, Colombia
2026

Citar/How to cite	Nombre . [1]
Referencia/Reference	[1] Nombre . “Thary: Sistema de apoyo a la decisión basado en predicción de demanda para la gestión de inventarios”, Trabajo de grado profesional , Ingeniería Sistemas , Universidad de San Buenaventura Santiago de Cali, Colombia , 2026 .
Estilo/Style:	
IEEE (2020)	



Biblioteca Digital (Repositorio)
www.bibliotecadigital.usb.edu.co

Bibliotecas Universidad de San Buenaventura

Biblioteca Fray Alberto Montealegre O.F.M. - Bogotá.

Biblioteca Fray Arturo Calle Restrepo O.F.M. - Medellín, Bello, Armenia, Ibagué.

Departamento de Biblioteca - Cali.

Biblioteca Central Fray Antonio de Marchena – Cartagena.

Universidad de San Buenaventura Colombia - www.usb.edu.co

Bogotá - www.usbbog.edu.co

Medellín - www.usbmed.edu.co

Cali - www.usbcali.edu.co

Cartagena - www.usbctg.edu.co

Editorial Bonaventuriana - www.editorialbonaventuriana.usb.edu.co

Revistas científicas – www.revistas.usb.edu.co

Dedicatoria

Texto de dedicatoria centrado.

Agradecimientos

Texto de agradecimientos centrado.

TABLA DE CONTENIDO

Resumen	11
Abstract	12
Glosario	13
1 Capítulo 1: Introducción	15
1.1 Contexto	15
1.2 Planteamiento del Problema	15
1.3 Justificación del Problema	16
1.3.1 Impactos	16
1.4 Propuesta de Solución	17
1.5 Pregunta de Investigación	18
1.5.1 Preguntas Derivadas	18
1.6 Objetivos	19
1.6.1 Objetivo General	19
1.6.2 Objetivos Específicos	19
1.7 Restricciones	19
1.8 Recursos y Viabilidad Técnica	20
2 Capítulo 2: Marco de Referencia	21
2.0.1 Criterios de inclusión	21
2.0.2 Criterios de exclusión	21
2.1 Fundamentos Teóricos	21
2.1.1 Gestión de inventarios	21
2.1.2 Tipos de gestión de inventario	22
2.1.3 Predicción de demanda	22
2.1.4 Series de tiempo	23
2.1.5 Modelos de predicción de demanda	23
2.1.6 Redes Neuronales Artificiales (RNA)	23
Long Short-Term Memory (LSTM)	23
Gated Recurrent Unit (GRU)	24
2.1.7 Modelos estadísticos y de Machine Learning tradicionales	25
Regresión lineal	25
2.1.8 Modelos basados en árboles de decisión	25

XGBoost	25
2.2 Antecedentes	26
2.2.1 Comparación de modelos de predicción de demanda	26
2.2.2 Implementaciones de referencia consultadas	28
2.3 Estándares de Ingeniería Aplicables	29
3 Capítulo 3: Diseño y Desarrollo	30
3.1 Enfoque Metodológico	30
3.1.1 Gestión del Proyecto (Planeación)	31
3.1.2 Ciclo de Vida del Software (Ejecución)	31
3.2 Procesos Técnicos y Diseño (ISO/IEC/IEEE 12207)	32
3.2.1 Identificación de Stakeholders	32
3.2.2 Elicitación de Requisitos	33
3.2.3 Especificación de Requisitos de Software (SRS)	35
3.2.4 Diseño de la Arquitectura	40
3.2.5 Escenarios Arquitecturales	40
3.2.6 Análisis de Alternativas y Decisiones de Diseño (ADRs)	41
3.2.7 Representación de la Arquitectura (Modelo C4)	43
3.2.8 Diseño Detallado	45
3.2.9 Etapa 1: Carga y limpieza de datos	45
3.2.10 Etapa 2: Segmentación de productos	46
3.2.11 Etapa 3: Construcción de variables	48
3.2.12 Etapa 4: Modelos de predicción	55
Definición de variables y funciones básicas	55
Modelos de demanda	57
XgBoost	57
Media Móvil	59
Regresión Lineal	59
3.2.13 Selección del campeón por producto	64
3.2.14 Pronóstico futuro	68
Ciclo recursivo	69
3.2.15 Cálculo de reposición (EOQ/ROP)	73
Lead time	74
Stock de seguridad y punto de reorden	74
Cantidad económica de pedido (EOQ)	74
3.2.16 Modelo de Datos	75

3.2.1	Diseño UX/UI	80
3.2.1	Análisis de Amenazas y Diseño Seguro	87
3.3	Construcción, Integración y Despliegue	90
3.3.1	Construcción	90
3.3.2	Integración	92
	Integración continua (CI)	94
3.3.3	Despliegue	95
3.4	Plan de Verificación y Validación	99
3.4.1	Estrategia de verificación	99
3.4.2	Estrategia de validación del modelo de predicción	99
3.4.3	Criterios de aceptación	99
3.4.4	Plan de validación con usuarios	99
4	Capítulo 4: Análisis de Resultados y Juicio de Ingeniería	100
4.1	Resultados de Verificación (Pruebas de Software)	100
4.2	Resultados de Validación (Evaluación del Usuario)	100
4.2.1	Validación del modelo de predicción	100
4.2.2	Consistencia mediante validación cruzada temporal	101
4.3	Evaluación de la Seguridad y Estándares	104
4.4	Análisis de Impactos de la Solución	104
4.5	Conclusiones	104
4.6	Recomendaciones de Operación y Mantenibilidad	104
4.7	Trabajos Futuros	104
4.8	Declaración de Uso Responsable de IA	104
	Referencias Bibliográficas	105
	ANEXOS	107
	Anexo A Documento completo SRS (historias de usuario / backlog)	108
	Anexo B Modelos de datos extensos, diccionarios de datos, mockups	120
	Anexo C Manual de usuario y/o guía de instalación y despliegue	121
	Anexo D Enlaces a repositorios de código fuente y tableros de gestión (Azure DevOps, Jira, GitHub)	122
	Anexo E Protocolo y seguimiento de la revisión sistemática de literatura (si aplica)	123

LISTA DE TABLAS

I	Comparación de técnicas de predicción aplicadas a problemas de demanda y series de tiempo	27
II	Implementaciones de referencia consultadas para el desarrollo de Thary.	28
III	Análisis de interés e influencia de los stakeholders	33
IV	Trazabilidad entre casos de uso y requisitos funcionales	38
V	Entidades principales del modelo de datos.	76
VI	Métricas del holdout final por modelo.	100
VII	Resumen de la validación cruzada temporal (promedio $\pm$ desviación, 3 folds). . . .	103
VIII	Especificación del caso de uso CU-01: Iniciar sesión	108
IX	Especificación del caso de uso CU-02: Cargar historial de consumo	109
X	Especificación del caso de uso CU-03: Procesar archivo	110
XI	Especificación del caso de uso CU-04: Consultar demanda pronosticada	111
XII	Especificación del caso de uso CU-05: Consultar estado del inventario	112
XIII	Especificación del caso de uso CU-06: Consultar recomendaciones de reposición y presupuesto	113
XIV	Especificación del caso de uso CU-07: Registrar pedido	114
XV	Especificación del caso de uso CU-08: Gestionar usuarios	115
XVI	Especificación del caso de uso CU-09: Deshacer pedido	116
XVII	Especificación del caso de uso CU-10: Consultar actividad de la empresa	117
XVIII	Especificación del caso de uso CU-11: Consultar calendario de pedidos	118
XIX	Especificación del caso de uso CU-12: Consultar perfil	119

LISTA DE FIGURAS

1	Esquema detallado de entradas, proceso y salidas del sistema.	16
2	Esquema detallado de la propuesta de solución.	18
3	Casos de uso del Administrador	36
4	Casos de uso del Personal de Inventario	37
5	Diagrama de contexto del sistema (nivel 1 del modelo C4).	43
6	Diagrama de contenedores del sistema (nivel 2 del modelo C4).	44
7	Diagrama de componentes del sistema (nivel 3 del modelo C4).	44
8	Estructura de la tabla <code>empresas</code> vista desde MySQL Workbench, conectado a la base de datos local.	79
9	Consulta sobre la tabla <code>corridas</code> en MySQL Workbench, mostrando el esquema completo de 12 tablas y el historial acumulado de corridas.	79
10	Logotipo de Thary, compuesto por el nombre del sistema y el icono representativo del búho	80
11	Pantalla de inicio de sesión de Thary	80
12	Pantalla de registro de Thary	81
13	Pantalla de dashboard de Thary	81
14	Pantalla de predicción de demanda de Thary	81
15	Pantalla de gestión de usuarios de Thary	82
17	Estado del inventario	83
20	Próximos pedidos	84
21	Calendario de pedidos	84
22	Cerrar sesión	85
23	Alertas visuales	85
24	Dashboard del empleado	86
25	Dashboard en modo claro	86
26	Dashboard en modo mobile	87
27	Servicios desplegados en Railway: aplicación web (Thary) y base de datos MySQL.	96
28	Detalles de construcción y despliegue del servicio Thary en Railway.	97
29	Variables de entorno del servicio Thary en Railway, con los valores ocultos.	98
30	Tablas del esquema relacional de Thary en el entorno de producción, vistas desde el explorador de datos de Railway.	98
31	Pantalla de inicio de sesión de Thary, accedida desde su URL pública en Railway.	99
32	Comparación de WAPE entre los tres modelos candidatos (holdout final).	101

LISTA DE CÓDIGOS

1	Función <code>cargar_y_limpiar</code> y etiquetas de las columnas obligatorias.	45
2	Función <code>calcular_comportamiento_producto</code> : variabilidad y clasificación ABC de cada producto.	47
3	Unión de las categorías ABC y de variabilidad a la tabla mensual.	49
4	Extracción del año y del mes de cada registro.	49
5	Codificación cíclica del mes mediante seno y coseno.	50
6	Cálculo de los rezagos <code>lag_1</code> , <code>lag_2</code> , <code>lag_3</code> y <code>lag_6</code> por producto.	51
7	Cálculo de las estadísticas móviles <code>media_movil_3</code> , <code>media_movil_6</code> y <code>desviacion_movil_3</code>	52
8	Incorporación de la media y la desviación históricas de cada producto.	52
9	Cálculo del inventario del mes anterior (<code>inventario_lag_1</code>).	53
10	Descarte de las filas sin historial suficiente para calcular los rezagos.	53
11	Lista de variables de entrada (<code>feature_cols</code>) y variables opcionales de precio e inventario.	54
12	Relleno final de los valores vacíos de las variables de entrada con 0.	55
13	Nivel de significancia de la prueba de Diebold-Mariano.	55
14	Constantes que definen el tamaño del periodo de prueba (<i>backtest</i>).	55
15	Función <code>tamano_test_backtest</code> , que decide el tamaño del periodo de prueba.	56
16	Parámetros del modelo XGBoost del <i>Ejemplo 3</i>	57
17	Función <code>entrenar_xgboost</code> , con los hiperparámetros del modelo.	58
18	Función <code>prediccion_baseline_movil</code> , que implementa el modelo de media móvil.	59
19	Filtro de las columnas sin variación (función <code>_seleccionar_features_regresion</code>).	60
20	Escalado de las variables con <code>MinMaxScaler</code>	60
21	Selección de variables mediante <i>Recursive Feature Elimination</i> (RFE).	61
22	Cálculo del VIF de las variables seleccionadas.	62
23	Eliminación de variables con VIF infinito o superior al límite.	62
24	Eliminación de variables según el p-valor del modelo OLS.	63
25	Función <code>entrenar_regresion_lineal</code> , que entrena el modelo con las variables seleccionadas.	64
26	Identificación del mejor y el segundo mejor modelo de cada producto (función <code>elegir_campeon_por_producto</code>).	65
27	Diferencia de error entre el mejor y el segundo mejor modelo.	65

28	Prueba de Diebold-Mariano con la corrección de Harvey, Leybourne y Newbold. . . .	66
29	Función <code>tamano_test_backtest</code> , que ajusta el periodo de prueba al historial disponible.	68
30	Definición de las tablas <code>empresas</code> , <code>usuarios</code> , <code>corridas</code> y <code>segmentacion</code> con sus claves foráneas.	77
31	Configuración de la conexión a MySQL (<code>app/services/db.py</code>).	78
32	Función <code>crear_usuario</code> , que almacena la contraseña como hash.	88
33	Función <code>verificar_credenciales</code> , que compara la contraseña ingresada con su hash.	88
34	Función <code>pedido_confirmar</code> , que registra los pedidos con la cantidad y el costo calculados por el sistema.	88
35	Registro de la actividad del usuario al confirmar un pedido (función <code>pedido_confirmar</code>).	89
36	Función <code>carpeta_uploads</code> , que aísla los archivos cargados por cada empresa. . .	89
37	Registro de los blueprints <code>main_bp</code> y <code>auth_bp</code> en la aplicación Flask.	92
38	Función <code>ejecutar_sistema</code> , que orquesta las nueve etapas del pipeline.	92
39	Función <code>carpeta_resultados</code> , que define la carpeta de resultados de cada empresa.	93
40	Decorador <code>admin_required</code> , que restringe el acceso de una ruta al perfil administrador.	94
41	Flujo de trabajo de GitHub Actions para ejecutar las pruebas automatizadas (<code>tests.yml</code>).	95
42	Contenido del archivo <code>Procfile</code> para el servidor Gunicorn.	96
43	Constantes y función <code>_armar_folds</code> de la validación cruzada temporal.	102

Resumen

resumen *Palabras clave* — **Predicción de demanda, gestión de inventarios, sistemas de apoyo a la decisión, series de tiempo**

El presente proyecto tiene como objetivo desarrollar un sistema de apoyo a la decisión orientado a la predicción de la demanda de productos en los inventarios.

Esta problemática surge a partir de la falta de información precisa sobre las cantidades necesarias de productos, lo cual dificulta la correcta planificación y puede llevar a que haya desabastecimiento, afectando negativamente la atención de los clientes.

Para abordar esta situación, en este trabajo se propone el uso de modelos de predicción basados en datos históricos de consumo. El sistema permitirá estimar las cantidades requeridas de productos cada mes, así como generar indicadores que apoyen la toma de decisiones para los encargados del manejo del inventario, tales como puntos de reposición y alertas.

Metodológicamente, se emplean diferentes técnicas y enfoques de la ingeniería de sistemas para obtener esta herramienta funcional, que contribuya a mejorar la gestión del inventario, optimizar la planificación de compras y reducir el riesgo de desabastecimiento.

Abstract

Abstract *Keywords* — **Demand forecasting, inventory management, decision support systems, time series** The objective of this project is to develop a decision support system designed to predict demand for products in inventory.

This problem arises from a lack of accurate information regarding the required quantities of products, which hinders proper planning and can lead to shortages, negatively affecting customer service.

To address this situation, this study proposes the use of forecasting models based on historical consumption data. The system will enable the estimation of the required quantities of products each month, as well as the generation of indicators to support decision-making for inventory managers, such as reorder points and alerts.

Methodologically, various techniques and approaches from systems engineering are employed to develop this functional tool, which will help improve inventory management, optimize purchasing planning, and reduce the risk of stockouts.

Glosario

Demanda: Cantidad de productos requeridos durante un período determinado, calculada a partir del comportamiento histórico de consumo o ventas.

Predicción de demanda: Proceso mediante el cual se estiman las cantidades futuras de productos utilizando modelos estadísticos o de aprendizaje automático basados en datos históricos.

Gestión de inventarios: Conjunto de actividades orientadas al control, almacenamiento, abastecimiento y seguimiento de los productos disponibles en una organización.

Inventario Conjunto de productos, materiales o recursos almacenados y disponibles para su uso, venta o distribución.

Desabastecimiento: Situación en la que un producto no se encuentra disponible en el inventario cuando es requerido.

Sistema de apoyo a la decisión: Herramienta informática diseñada para asistir a los usuarios en la toma de decisiones mediante el análisis de datos y la generación de información relevante.

Modelo de predicción: Algoritmo o método matemático utilizado para estimar valores futuros a partir de datos históricos y patrones identificados en la información.

Datos históricos: Registros de consumo, ventas, compras o movimientos de inventario ocurridos en períodos anteriores y utilizados para el análisis y la predicción.

Punto de reposición: Nivel de inventario a partir del cual se debe realizar un nuevo pedido para evitar el desabastecimiento.

Tiempo de entrega (Lead Time): Período que transcurre entre la realización de un pedido y la recepción efectiva de los productos solicitados.

Serie de tiempo: Conjunto de observaciones registradas en orden cronológico que permiten analizar patrones, tendencias y comportamientos a lo largo del tiempo.

Aprendizaje automático (Machine Learning): Rama de la inteligencia artificial que permite a los sistemas aprender patrones a partir de datos y realizar predicciones sin ser programados explícitamente para cada situación.

Dominio de inventario: Contexto o sector específico en el que se gestionan inventarios, como

retail, manufactura, logística, salud, restaurantes o comercio electrónico.

1 Capítulo 1: Introducción

1.1 Contexto

La gestión eficiente de inventarios supone un desafío constante en diversos sectores. Problemas como sufrir de debastabecimiento pueden surgir por distintos factores como retrados en la entrega por parte de los proveedores, variaciones inesperadas de la demanda, o deficiencias en los procesos de planificación. Estas situaciones, en muchos casos, se encuentran fuera del control de los administradores, lo que incrementa la complejidad para ellos a la hora de tomar decisiones relacionadas con el abastecimiento.

Actualmente, normalmente en muchas empresas se cuenta con un software para gestión de inventarios tipo ERP, con este software se administra la disponibilidad de los productos mediante el uso de diferentes tipos de movimientos de inventario:

Uno de los tipos de movimientos consiste en las entradas de inventario, las cuales responden a la adquisición de productos a través de proveedores y son registradas mediante facturas que incluyen información como fecha, proveedor y costo.

Por otro lado, están las salidas por consumo, las cuales representan la venta, distribución o consumo de los productos, registrando información relacionada con las cantidades entregadas y el destino de estas. Adicionalmente, existen salidas de inventario asociadas a sucesos como devoluciones, productos dañados, vencimientos, pérdidas o ajustes de inventario, se registra la cantidad de productos que deben de ser retirados del inventario junto con su respectivo motivo. Posteriormente, se realizan ajustes de inventario, los cuales pueden ser tanto positivos como negativos, con el fin de corregir diferencias entre las cantidades registradas en el sistema y las existencias reales, garantizando así la consistencia de la información.

En general, un buen análisis de estos movimientos resulta fundamental para comprender el comportamiento del consumo de productos y así mejorar los procesos de planificación del abastecimiento.

1.2 Planteamiento del Problema

En diferentes entornos, existe una problemática en la que la gestión del inventario de productos se ve afectado debido a la falta de información precisa sobre la demanda mensual de los mismos, lo que

dificulta la planificación adecuada del abastecimiento. En este contexto, surge la necesidad de contar con herramientas que permitan analizar los datos históricos de inventario y generar estimaciones de demanda que apoyen la toma de decisiones relacionadas con el abastecimiento.



Fig. 1. Esquema detallado de entradas, proceso y salidas del sistema.

1.3 Justificación del Problema

Este proyecto se justifica teniendo en cuenta que esta es una problemática que no se ha visto abordada a profundidad y continua siendo una dificultad para organizaciones de diferentes sectores. Además este proyecto permite aplicar conocimientos de la ingeniería de sistemas como análisis de datos, modelos de predicción y aprendizaje automático para la construcción de una herramienta que contribuya a mejorar los procesos de planificación y administración, generando beneficios tanto para las organizaciones y los usuarios finales.

1.3.1 Impactos

Mejorar la disponibilidad de los productos mediante una mejor planificación de los inventarios, apoyar la toma de decisiones del personal encargado del abastecimiento, y fortalecer el uso de herramientas tecnológicas en el sector administrativo.

1.4 Propuesta de Solución

Con este proyecto se propone el desarrollo de un software que ayude a predecir la demanda de productos en diferentes dominios y contextos de gestión de inventarios. Este sistema se realizará tras analizar datos como historial de consumo, esto para poder estimar exactamente las cantidades necesarias de cada producto en periodos mensuales.

La solución implementará modelos de predicción de demanda que procesarán la información histórica para identificar patrones y en base a ellos, generar estimaciones futuras. A partir de estas predicciones, el sistema podrá proporcionar información relevante para la toma de decisiones, tales como cantidades recomendadas de compra, puntos de reposición y posibles alertas de desabastecimiento.

Se utilizarán datos reales del año 2021 del sector salud como caso de estudio, que serán complementados con datasets con datos públicos de diferentes dominios esto para poder evaluar el comportamiento del sistema en diferentes contextos.

Adicionalmente, el sistema tendrá en cuenta variables como tiempo de entrega de proveedores (lead time) y cantidad de productos disponibles, con el propósito de lograr que el sistema sea lo más preciso posible.

Esta información será presentada en una interfaz que permitirá visualizar los resultados de manera comprensible, esto para facilitar su uso por parte del personal encargado de la gestión de inventarios.

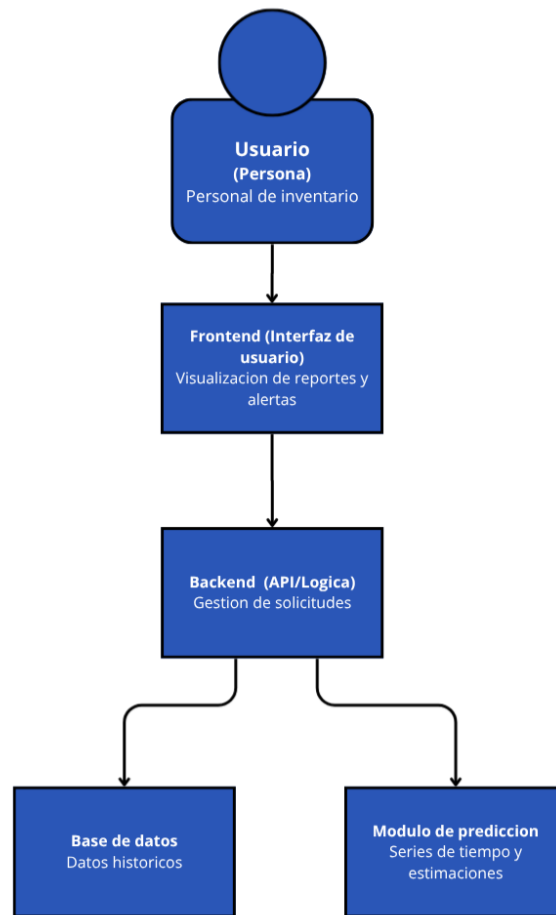


Fig. 2. Esquema detallado de la propuesta de solución.

1.5 Pregunta de Investigación

¿Cómo se puede diseñar e implementar un sistema de apoyo a la decisión que, mediante modelos de predicción de demanda, permita estimar las cantidades necesarias de productos en un sistema de gestión de inventarios?

1.5.1 Preguntas Derivadas

¿Qué variables y datos históricos son necesarios para estimar la demanda mensual de productos?

¿Qué modelos de predicción de demanda son más adecuados para estimar el consumo o venta de productos en diferentes dominios?

¿Cómo debe estructurarse el proceso de análisis y procesamiento de datos para generar

predicciones confiables?

¿Qué tipo de salidas debe generar el sistema (por ejemplo, cantidades estimadas, punto de reposición, alertas) para apoyar la toma de decisiones?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un sistema que le permita a los usuarios poder predecir cual va a ser la demanda mensual de productos mediante el analisis de datos historicos, para garantizar su correcta planificacion y abastecimiento

1.6.2 Objetivos Específicos

Identificar las fuentes de datos relevantes, incluyendo el historial de consumo y las variables del inventario necesarias para la predicción de la demanda de productos.

Seleccionar modelos de predicción de demanda adecuados para estimar el consumo o venta mensual de productos en diferentes contextos de los inventarios.

Incorporar variables como el tiempo de entrega de proveedores y el punto de reposición dentro del sistema para complementar las estimaciones de demanda.

Diseñar un módulo de visualización que presente las predicciones de manera clara y comprensible para apoyar la toma de decisiones del usuario.

Evaluar el desempeño del sistema desarrollado mediante la realización de pruebas con datos históricos, con el fin de analizar la precisión de las predicciones generadas y verificar su utilidad en el apoyo a la toma de decisiones en la gestión del inventario.

1.7 Restricciones

El proyecto tiene como principal alcance el desarrollo de un sistema de apoyo a la decisión orientado a la predicción de la demanda de medicamentos críticos en una farmacia hospitalaria, esto utilizando los datos históricos de consumo como principal fuente de información. El sistema permitirá estimar la cantidad mensual requerida de medicamentos mediante la implementación de

modelos de predicción de demanda, integrando funcionalidades como visualización de resultados, generación de reportes y apoyo en la toma de decisiones relacionadas con la planificación del abastecimiento. Asimismo, el sistema proporcionara al usuario cálculos como el punto de reposición, con el fin de contribuir a una mejor organización y control de los recursos disponibles en los entornos hospitalario.

El desarrollo del proyecto presenta algunas restricciones que deben ser consideradas. En primer lugar, el sistema se basará en un conjunto de datos históricos reales de la Clínica Nuestra señora de los remedios limitado al año 2021, lo que puede afectar la capacidad del modelo para capturar comportamientos actuales o cambios en la demanda a lo largo del tiempo. Además, no se contará con integración en tiempo real con sistemas hospitalarios, por lo que las predicciones se realizarán sobre datos estáticos.

1.8 Recursos y Viabilidad Técnica

2 Capítulo 2: Marco de Referencia

Para la construcción del estado del arte, se realizó una revisión de literatura en bases de datos académicas reconocidas, con el fin de identificar estudios relevantes relacionados con la predicción de demanda y la gestión de inventarios.

2.0.1 Criterios de inclusión

Se consideraron los siguientes criterios para la selección de los estudios:

- Trabajos disponibles en español o inglés.
- Publicaciones académicas en bases de datos como Scopus, ScienceDirect, Google Scholar y demás bases de datos que ofrece la universidad de San Buenaventura.
- Estudios enfocados en gestión de inventarios.
- Investigaciones que utilicen modelos de predicción de demanda.
- Estudios que describan el diseño de sistemas informáticos de apoyo a la decisión.

2.0.2 Criterios de exclusión

Se excluyeron los estudios que cumplieran con las siguientes características:

- Estudios con más de 10 años de antigüedad.
- Documentos incompletos o resúmenes sin acceso al texto completo.

2.1 Fundamentos Teóricos

2.1.1 Gestion de inventarios

La gestion de inventarios consiste en el proceso que se realiza sobre la mercancía para conocer la cantidad disponible, esto permitiendo analizar los niveles de existencia y la aproximación del consumo real.[1]

Los pasos basicos para realizar una correcta gestion de inventario consiste en: Compra del inventario, en el que los productos se entregan al almacen o punto de venta, Almacenamiento del inventario, en el que este se guarda hasta que se necesite y Aprovechamiento del inventario, en donde se controla la cantidad de productos que hay a la venta para finalmente enviarlo o venderlo a los clientes.[2]

2.1.2 Tipos de gestion de inventario

Gestión periódica de inventarios Este método implica realizar un recuento físico del inventario a intervalos específicos. Se toma el inventario al inicio del periodo, se suman las nuevas compras y se deduce el inventario final para calcular el costo de las mercancías vendidas (COGS).[2]

Gestión de inventario de códigos de barras Utiliza un sistema que asigna un número único a cada producto. Permite asociar datos como proveedor, dimensiones del producto, peso y cantidad en stock.[2]

Gestión de inventario RFID Transmite de forma inalámbrica la identidad de un producto mediante un número de serie único. Mejora la eficiencia y visibilidad del inventario, facilitando el seguimiento de artículos.[2]

2.1.3 Prediccion de demanda

La prediccion de la demanda es un proceso que permite mejorar la gestion de los niveles de inventario mediante la utilizacion de datos historicos. Una correcta prediccion de la demanda le permite a las organizaciones el poder tomar decisiones mas inteligentes a la hora de manejar sus provisiones, evitando la falta o exceso de productos, de esta manera, aumentando tanto la satisfaccion del cliente como la creacion de estraregias de negocio mas efectivas a corto y largo plazo [3].

En los ultimos años, los mercados han evolucionado y se han vuelto cada vez mas exigentes en cuanto la calidad y precision de los servicios, es por eso, que muchas organizaciones se ha visto inclinadas a incluir el uso de nuevas herramientas y tecnologias como Inteligencia Artificial. Machine Learning o analisis predictivo, para automatizar y perfeccionar esta tarea, ya que los metodos actuales estaticos que son la norma en muchas empresas y organizaciones han empezado a quedar obsoletas para satisfacer las necesidades del mercado actual.[3], [4]

La principal debilidad de los modelos de inventario actuales es que al ser poco robustos, no

toman en contemplación diferentes factores externos de manera inteligente [4].

2.1.4 Series de tiempo

Las series temporales son usadas para analizar datos históricos recogidos en intervalos regulares, como ventas diarias o mensuales, para identificar patrones, tendencias estacionales y ciclos. Estos patrones se identifican mediante técnicas como gráficos ACF y PACF, también estos patrones sirven para ajustar modelos estadísticos o de aprendizaje automático (como ARIMA, SARIMAX o Prophet) que permiten proyectar valores futuros [5]

Se utilizan estos modelos para hacer predicciones, los datos se agrupan en intervalos (mensualmente) para reducir variabilidad y facilitar el análisis. Tras el análisis de datos, se ajustan los modelos y se evalúan mediante métricas como el RMSE, para que las predicciones sirvan en la toma de decisiones de gestión y planificación de recursos

2.1.5 Modelos de predicción de demanda

2.1.6 Redes Neuronales Artificiales (RNA)

Las redes neuronales artificiales (RNA) son técnicas útiles para la previsión de la demanda, gracias a su capacidad para manejar datos no lineales y captar relaciones funcionales sutiles entre datos empíricos, incluso cuando las relaciones subyacentes son desconocidas o difíciles de describir.

Long Short-Term Memory (LSTM):

Las redes neuronales de memoria a largo y corto plazo o LSTM es un tipo de red neuronal que se ha demostrado eficaz en la predicción de series temporales, esta técnica está diseñada para procesar datos secuenciales y temporales que fue creada para superar las limitaciones de las redes neuronales tradicionales.[6].

Su funcionamiento se basa en una estructura de memoria que permite a la red aprender y recordar información a lo largo del tiempo, lo que es especialmente útil para capturar dependencias a largo plazo en los datos. LSTM cuenta con las tres siguientes puertas:

Puerta de olvido (forget gate): Decide qué información se descarta. Esta se calcula mediante la función sigmoide, que toma como entrada el vector de entrada actual y el estado oculto anterior, de esta manera, generando un valor entre 0 y 1 para cada componente de la celda de la memoria.

$$f_t = \delta(W_f[h_{t-1}, X_t] + b_f) \quad (1)$$

Puerta de entrada (input gate): Decide qué información se almacena en la memoria. También usa una función tanh para normalizar la información que pasa a través de la red neuronal.

$$i_t = \delta(W_i[h_{t-1}, X_t] + b_i), \quad L_t = \tanh(W_c[h_{t-1}, X_t] + b_c) \quad (2)$$

Puerta de salida (output gate): Decide qué información se utiliza para generar la salida oculta actual.

$$o_t = \delta(W_o[h_{t-1}, X_t] + b_o) \quad (3)$$

La actualización del estado celular se realiza mediante:

$$C_t = f_t * C_{t-1} + i_t * L_t \quad (4)$$

Finalmente, la salida oculta se calcula aplicando la función tanh al estado celular y multiplicándolo por la puerta de salida como:

$$h_t = o_t * \tanh(C_t) \quad (5)$$

Gated Recurrent Unit (GRU):

GRU es un modelo de predicción bastante parecido a LSTM, pero con una arquitectura mucho más simple, lo que la hace muchísimo más compacta y potencialmente menos costosa computacionalmente que una LSTM equivalente. Estas han demostrado tener éxito en diferentes aplicaciones relacionadas con datos secuenciales o temporales, principalmente si son secuencias largas. GRU reduce el número de tres redes compuestas distintas que usa LSTM en solo 2, esto mediante la combinación de la compuerta de entrada y la compuerta de olvido mediante una suma directa. Las compuertas de GRU se denominan: Compuerta de actualización y Compuerta de reinicio. [7]

El modelo GRU se presenta como:

$$h_t = g(Uh(r_t \odot h_{t-1}) + Wx_t + b_h) \quad (6)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (7)$$

donde z_t es la compuerta de actualización, r_t es la compuerta de reinicio y $\tilde{h}_t$ es la propuesta de nueva información. Las dos compuertas se presentan como:

$$z_t = \sigma(U_z h_{t-1} + W_z x_t + b_z) \quad (8)$$

$$r_t = \sigma(U_r h_{t-1} + W_r x_t + b_r) \quad (9)$$

2.1.7 Modelos estadísticos y de Machine Learning tradicionales

Regresión lineal:

Los modelos de regresion son utilizados para resolver problemas de prediccion, el objetivo de este modelo puede ser explicativo, donde se busca, por ejemplo, evaluar como ciertas variables independientes afectan a una variable dependiente, o predictivo, en el que se busca estimar el valor de la variable dependiente en funcion de las independientes [8].

Un modelo de regresion se expresa como:

$$y = (x_1, x_2, \dots, x_n)\beta + \epsilon \quad (10)$$

Existen distintas opciones para poder estimar un modelo de regresion, siendo las mas comunes: Regresion lineal y regresion logistica. La eleccion depende del tipo de variable dependiente que se tenga, si es una variable continua se utiliza regresion lineal, mientras que si se trata de una variable dicotomica se utiliza una variable dicotomica.

2.1.8 Modelos basados en árboles de decisión

XGBoost:

XGBoost se ha utilizado ampliamente en numerosos campos para lograr resultados de vanguardia (state-of-the-art) en tareas relacionadas con datos y predicción. Su funcionamiento consiste en en un sistema de aprendizaje automatizado basado en el metodo de Tree Boosting que incluye un algoritmo de aprendizaje de arboles. EL Tree Boosting es un algoritmo de aprendizaje por conjuntos o ensemble learning, capaz de transformar varios clasificadores debiles en uno fuerte con el objetivo de obtener un mejor rendimiento en le clasificacion, es decir, se combinan multiples arboles para obtener un modelo mas potente. Un modelo tree boosting se escribe de la siguiente manera[9]:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), f_k \in F \quad (11)$$

Donde:

$$F = \{f(x) = w_{q(x)}\}(q : R^m \rightarrow T, w \in R^T) \quad (12)$$

Representa el espacio de los arboles de regresion o clasifiacion

El ciclo de vida del software se rige por **iso12207**, y los fundamentos de ingeniería aplicados se basan en **sommerville2016**. La arquitectura se representa siguiendo **c4model**.

2.2 Antecedentes

2.2.1 Comparación de modelos de predicción de demanda

Debido a que existen diferentes modelos y tecnicas para la prediccion de la demanda, se realizo una revision de literatura de diferentes estudios que emplean los modelos mas utilizados para la prediccion de la demanda, esto con el fin de realizar el siguiente analisis comparativo para determinar cual es el mas adecuado para la prediccion de demanda en el presente proyecto, teniendo en cuenta ventajas, limitaciones y características como: tipo de datos utilizados, dominio de aplicación, tecnica de aplicacion, metricas de evaluacion y resultados obtenidos, dataset utilizado, entre otros.

TABLA I

Comparación de técnicas de predicción aplicadas a problemas de demanda y series de tiempo

Aspecto	Redes neuronales (ANN)	LSTM	Regresión lineal	GRU	XGBoost
Referencia	<i>Demand Forecasting Using Neural Network for Supply Chain Management</i>	<i>LSTM Model Based Demand Forecasting in Supply Chain Using Efficient Optimizer</i>	<i>Application of ABC Analysis and Linear Regression Model in Inventory Management</i>	<i>StockNet-GRU Based Stock Index Prediction</i>	<i>XGBoost-Based Demand Forecasting in Supply Chain Management Using Machine Learning Algorithm</i>
Datos utilizados	Datos históricos de ventas organizados como series de tiempo. Se emplean ventas mensuales de un filtro de combustible entre 2011 y 2013 para predecir la demanda de 2014.	Datos históricos de ventas organizados como series de tiempo. Incluyen fecha, unidades vendidas, ganancias y ubicación, seleccionando las variables relevantes para el entrenamiento.	Datos históricos de inventario y ventas. Incluyen información de productos, inventario, órdenes de compra y venta, cantidad y precio de venta, fecha, marcas, clasificación e impuestos.	Series de tiempo financieras correspondientes a precios históricos de apertura del índice bursátil CNX Nifty y de las 10 acciones con mayor correlación. Los datos se normalizan y organizan mediante ventanas deslizantes de 20 días.	Datos logísticos y de cadena de suministro relacionados con la demanda, incluyendo variables asociadas con proveedores y logística. Los datos se normalizan mediante Z-score y posteriormente se seleccionan las variables relevantes.
Dominio	Gestión de la cadena de suministro, control de inventarios, manufactura y distribución. También presenta aplicaciones en turismo, consumo eléctrico, supermercados y ventas.	Gestión de la cadena de suministro y predicción de demanda.	Gestión de inventarios, cadena de suministro y análisis de ventas.	Predicción de índices bursátiles, mercados financieros y análisis de series de tiempo.	Gestión de la cadena de suministro, predicción de demanda, logística y optimización de inventarios. El enfoque puede aplicarse a diferentes sectores que requieran pronósticos de demanda.
Técnica	Red neuronal <i>feed-forward</i> entrenada mediante <i>Backpropagation</i> . Se comparan Gradient Descent, Conjugate Gradient y Levenberg-Marquardt, siendo este último el de mejor desempeño.	Red neuronal recurrente LSTM con tres capas, regularización mediante Dropout y optimizadores Adam y SGD. Se experimenta con épocas, tasa de aprendizaje, tamaño de lote y longitud de secuencia.	Modelo de regresión lineal para identificar tendencias en cantidad y precio de venta. Utiliza Gradient Descent para minimizar el error cuadrático, después de realizar limpieza de datos y selección de variables.	Red neuronal recurrente GRU integrada en la arquitectura StockNet. Utiliza ventanas deslizantes de 20 días, optimizador Adam, activación ReLU y datos normalizados entre 0 y 1.	El proceso incluye normalización Z-score, selección de características mediante un Algoritmo Genético, predicción mediante XGBoost y optimización de hiperparámetros mediante Particle Swarm Optimization (PSO).
Métricas	Error absoluto de pronóstico (<i>Forecasting Error</i>) y porcentaje de error, comparando los valores pronosticados con los valores reales.	RMSE para medir la diferencia entre valores reales y predichos. También se compara el tiempo promedio por época para evaluar la eficiencia computacional.	Coefficiente de determinación (R^2). El estudio reporta un valor de 70,57 % para la predicción de la cantidad vendida y cercano al 80 % para el precio de venta.	RMSE, MAE y MAPE. También se considera el tiempo de ejecución y se utiliza la prueba estadística de Wilcoxon para validar diferencias entre modelos.	MAE, RMSE y coeficiente de determinación. Se compara el desempeño de XGBoost con SARIMA, Decision Tree (DT) y Random Forest (RF).
Dataset	Datos reales de consumo mensual de un filtro de combustible perteneciente a una empresa manufacturera. Contiene registros mensuales entre 2011 y 2013 para entrenamiento y validación.	<i>Sales Dataset</i> con información histórica de ventas utilizada para entrenar y evaluar el modelo.	Dataset de inventario de una tienda de comestibles (<i>grocery store</i>) con información de productos, inventario y ventas. Después del preprocesamiento se utilizan aproximadamente 60 observaciones correspondientes a dos meses.	Índice bursátil CNX Nifty y las 10 acciones más correlacionadas del National Stock Exchange (NSE) de India, obtenidos desde Yahoo Finance. Contiene 5.904 días de negociación entre abril de 1996 y enero de 2020.	Dataset de demanda de cadena de suministro con información relacionada con logística y proveedores. Se utiliza para entrenar y evaluar el modelo de predicción.
Comentario / reseña	Las ANN permiten modelar relaciones no lineales y presentan resultados favorables frente a métodos tradicionales de series de tiempo. Son útiles para predicciones de corto plazo, aunque requieren suficiente información y ajuste de parámetros.	LSTM constituye una alternativa eficaz para la predicción de demanda debido a su capacidad para capturar dependencias temporales de largo plazo. Puede contribuir a la optimización de inventarios y a la toma de decisiones en la cadena de suministro.	La regresión lineal permite identificar y proyectar tendencias generales de ventas e inventario y presenta una menor complejidad. Sin embargo, tiene limitaciones para representar relaciones complejas y eventos extraordinarios.	GRU permite capturar dependencias temporales y presenta resultados favorables en la predicción de series de tiempo. Sin embargo, el estudio analizado se desarrolla en el dominio financiero, por lo que su aplicación a inventarios requiere evaluación adicional.	XGBoost presenta capacidad para modelar relaciones complejas y puede utilizarse para apoyar la planificación de inventarios, reducir costos y optimizar operaciones en la cadena de suministro.

Fuente: Elaboración propia a partir de los estudios revisados.

2.2.2 Implementaciones de referencia consultadas

Ademas de los estudios academicos revisados en la Seccion 2.2.1, se consultaron cuatro implementaciones practicas de codigo abierto, disponibles en Kaggle y GitHub, que sirvieron como base y punto de partida para la construccion del pipeline de prediccion de Thary. La Tabla II resume cada una de estas implementaciones, el identificador que se usara para referirse a ellas a lo largo del documento, y el aporte que realizaron al diseño del sistema.

TABLA II
Implementaciones de referencia consultadas para el desarrollo de Thary.

ID	Autor	Título	Aporte a Thary
Ejemplo 1 (SKU Demand Forecasting)	Dhruvi Shah [10]	<i>Demand Forecasting & Inventory Optimization for Consumer Products</i>	Base para el flujo general de prediccion y calculo de reposicion
Ejemplo 2 (Media Móvil)	Samir Saci [11]	<i>Machine Learning for Retail Demand Forecasting: Comparative Study of Demand Forecasting Methods for a Retail Store (XGBoost Model vs. Rolling Mean)</i>	Base para el uso de la media movil como modelo candidato de comparacion
Ejemplo 3 (XGBoost)	ksmooi [12]	<i>Forecasting Inventory Demand Using XGBoost</i>	Base para el entrenamiento y ajuste del modelo XGBoost
Ejemplo 4 (BoomBikes)	Tushar Verma [13]	<i>Linear Regression: Demand Forecasting</i>	Base para la metodologia de seleccion de variables en el modelo de Regresion Lineal (RFE, eliminacion por p-valor y VIF)

A lo largo del Capítulo 3 se hace referencia a estas cuatro implementaciones mediante el identificador presentado en la Tabla II (Ejemplo 1, Ejemplo 2, Ejemplo 3 o Ejemplo 4), indicando en cada caso que partes específicas de Thary se basaron en ellas y cuáles corresponden a extensiones propias desarrolladas durante el proyecto.

2.3 Estándares de Ingeniería Aplicables

3 Capítulo 3: Diseño y Desarrollo

3.1 Enfoque Metodológico

El enfoque metodológico de este proyecto se basa en dos componentes principales. El primero se enfoca en determinar de manera empírica cual o cuales modelos de predicción de demanda resultan mas adecuados para el caso de estudio. El segundo componente se centra en la estructuración de dicho modelo dentro de una herramienta funcional, siguiendo procesos técnicos alineados con la norma ISO/IEC/IEEE 12207.

Para la evaluación de los modelos de predicción, el enfoque adoptado es empírico y por producto, ya que en lugar de seleccionar un único modelo ganador para la totalidad del catálogo de productos, se entrenan y comparan distintos modelos candidatos, y para cada producto, se selecciona de forma individual aquel que presente el menor error de pronóstico durante el periodo de prueba. Esta decisión responde a que no existe un modelo que universalmente pueda adaptarse a todos los patrones de demanda como: productos de consumo estable, esporádico o altamente variable, cada uno de los patrones puede favorecerse de distintas técnicas. Por otra parte, en cuanto a la selección de variables del modelo de Regresión Lineal, este se llevo a cabo siguiendo la misma metodología reportada en la literatura revisada en el Capítulo 2, es decir, teniendo en cuenta factores como la eliminación recursiva de características seguida de eliminación hacia atrás por significancia estadística y factor de inflación de varianza.

Debido a que los datos que fueron empleados corresponden a series de tiempo (registros de consumo mensual), se respeta el orden cronológico de la información. La separación entre los datos de entrenamiento y prueba se realiza por fecha, de manera que, se entrenan con los periodos mas antiguos y se evalúa sobre los mas recientes. Adicionalmente, se incorpora un procedimiento de validación cruzada que repite el proceso de entrenamiento y evaluación sobre distintas ventanas de tiempo sucesivas, con el fin de confirmar que el desempeño del sistema no dependa de las particularidades de un solo periodo de prueba.

Finalmente, el enfoque también incluye la evaluación mediante el uso de un caso de estudio real, ya que se usaron los datos históricos de consumo del año 2021 de la clínica Nuestra Señora de los Remedios, que constituye la validación del sistema en el dominio de la salud.

3.1.1 Gestión del Proyecto (Planeación)

La planeación del proyecto se estructuró en cuatro fases secuenciales, cada una con un conjunto de entregables y actividades que se detallan a continuación:

- Fase 1 (Fase de fundamentación): En esta fase se plantean los objetivos del proyecto y realiza la revisión de literatura y construcción del marco de referencia, se definen los criterios de inclusión y exclusión de los estudios consultados y se compara el desempeño reportado por los distintos modelos de predicción de demanda.
- Fase 2 (Fase de análisis y diseño): Se definen los requisitos del sistema, la arquitectura y el modelo de datos
- Fase 3 (Fase de construcción): Se implementa el sistema de predicción, con el módulo de cálculo de puntos de reposición y la interfaz de usuario, en esta fase se desarrollan el sistema identificando limitaciones en los datos o en los modelos, así como limitaciones propias de la arquitectura inicial del sistema, las cuales se fueron corrigiendo a medida que avanzaba el desarrollo y surgían nuevas necesidades y propuestas para satisfacerlas.
- Fase 4 (Fase de evaluación): En la fase final se le aplica al sistema un plan de verificación y validación, que incluye la evaluación de los modelos de predicción de demanda y la validación del sistema en el dominio de la salud, mediante el uso de un caso de estudio real.

3.1.2 Ciclo de Vida del Software (Ejecución)

La ejecución del proyecto se realizó siguiendo un ciclo de vida iterativo-incremental, en el cual el sistema se construyó en sucesivas versiones del mismo en el que en cada una de estas se iba ampliando o corrigiendo detalles de la anterior a partir de la evaluación de los resultados obtenidos. La elección de esta estrategia se basa en la naturaleza del proyecto, en el que al estar trabajando en el desarrollo de un sistema de predicción, es esperable que decisiones como la forma de representar los meses como variable de entrada, o el conjunto de variables históricas utilizadas por el modelo, no podrían definirse por adelantado, sino que se irían determinando y ajustando a medida que se experimentara y observara el comportamiento del sistema durante sus fases de desarrollo.

Un ejemplo de una de estas decisiones fue el cómo se llevaría a cabo la persistencia de la información en el sistema. En una primera versión se llevó a cabo mediante archivos estructurados, pero posteriormente se migró hacia una base de datos relacional al requerir un mayor control y almacenamiento de la información, más específicamente, es necesario el poder mantener un historial

de auditoria y de corridas anteriores, algo que no se habia contemplado por completo desde el inicio del proyecto.

Otro ejemplo es que durante las pruebas realizadas, se encontró un error en el que el sistema no podia reconocer la columna "precio" en español a pesar de que la interfaz la anunciaba como si hubiese sido aceptada.

De igual forma, la manera en la que iban a incorporarse las pruebas automatizadas en el proyecto tambien fue un aspecto que no se planeo desde el inicio, sino que fue parte de los detalles que se fueron incorporando al desarrollo del sistema a medida que este avanzaba.

En cuanto al diseño de la arquitectura del sistema, el diseño de la interfaz de usuario, y los mecanismos de control de acceso al sistema, se siguieron los procesos tecnicos que indica la norma ISO/IEC/IEEE 12207, que establece un conjunto de procesos a tener en cuenta como la identificacion de stakeholders, especificacion de requisitos, diseño arquitectonico, los cuales fueron definidos para el sistema.

3.2 Procesos Técnicos y Diseño (ISO/IEC/IEEE 12207)

Los procesos tecnicos que guiaron el diseño y desarrollo del sistema Thary se estructuraron siguiendo la ISO/IEC/IEEE 12207, una norma que establece un conjunto de procesos estandar para el ciclo de vida del software. Dichos procesos fueron aplicados al desarrollo del sistema de la siguiente manera:

3.2.1 Identificación de Stakeholders

A partir del planteamiento del sistema y tras un analisis de sus características y funcionalidades, se identificaron los siguientes stakeholders que estarian involucrados en el sistema, ya sea directa o indirectamente:

- Personal encargado de la gestion del inventario y el abastecimiento: Es el usuario final que interactua con el sistema, es responsable de interpretar los resultados que Thary genera y tomar decisiones de reposicion en base a ellas.
- Equipo de desarrollo: Desarrolladores y diseñadores responsables de la implementación del sistema Thary, asegurando que cumpla con los requisitos y estándares establecidos.

- **Administradores del sistema:** Es el perfil con permisos y privilegios, es responsable de cargar los datos historicos de consumo, es decir, el archivo de entrada, ejecutar el proceso de prediccion, ademas, puede crear nuevos usuarios para los empleados con menor rango.
- **Institucion usuaria del sistema:** Es la entidad interesada en utilizar Thary para poder reducir el riesgo de desabastecimiento de productos. El sistema se planeo para poder adaptarse en distintos dominios, por lo que cualquier tipo de empresa puede usarlo.
- **Proveedores de productos:** Aunque los proveedores no interactuan con Thary, indirectamente son una variable importante para el sistema, ya que factores externos como el tiempo de entrega o lead time, marcan una gran diferencia en los resultados del sistema.

TABLA III
Análisis de interés e influencia de los stakeholders

Stakeholder	Interés	Influencia
Personal de inventario y abastecimiento	Alto	Media
Administrador del sistema	Alto	Alta
Institución usuaria	Alto	Alta
Proveedores de productos	Medio	Media
Equipo de desarrollo	Alto	Alta

3.2.2 Elicitación de Requisitos

La elicitation de requisitos consiste en poder identificar las necesidades del sistema a partir de los usuarios, en el caso de Thary, el proceso se llevo a cabo mediante dos fuentes principales de informacion, en primera instancia, la revision de literatura permitio analizar que variables suelen ser relevantes para este tipo de proyectos, y que salidas le aportan mayor valor al usuario. En segundo lugar, se realizo un analisis a un caso de estudio real, los datos usados durante el desarrollo del sistema eran datos autenticos de la Clinica Nuestra señora de los Remedios de la ciudad de Cali, lo cual permitio observar como se comportaria el sistema en un escenario real como la existencia de variaciones en la demanda o datos historicos no tan limpios como historiales incompletos o datos con errores, y que ajustes serian necesarios para mejorar su rendimiento.

Es en base a todo esto, que se definieron los siguientes requisitos funcionales:

- RF-01: El sistema debe permitir que un usuario registrado como admin pueda realizar la carga de un archivo CSV con datos como el historial de consumo mensual de los productos, validando que contenga los datos minimos requeridos: fecha, id del producto y cantidad demandada.
- RF-02: El sistema debe generar una estimacion de la demanda mensual, esperada para cada producto a partir de la informacion compartida en el historial cargado
- RF-03: El sistema debe de calcular para cada producto datos como el punto de reposicion y la cantidad recomendada de compra, considerando factores externos como el tiempo de entrega del proveedor o lead time, presupuestos y el inventario actual disponible
- RF-04: El sistema debe de contar con alertas visuales claras para los productos cuyo inventario actual se encuentre en o por debajo de su punto de reposicion.
- RF-05: El sistema debe presentar los resultados, demanda pronosticada, estado del inventario, etc, de una manera clara y sencilla de entender en una interfaz intuitiva para el usuario
- RF-06: El sistema debe permitirle al personal del inventario registrar un pedido, guardando informacion importante de este como proveedor elegido, fecha del pedido y el usuario responsable de la realizacion del pedido.
- RF-07: El sistema debe restringir el acceso a la carga de los archivos y la ejecucion del proceso unicamente al perfil administrador.
- RF-08: El sistema debe permitirle al perfil de administrador el poder crear nuevos usuarios con menos privilegios para sus empleados.
- RF-09: El sistema debe permitir el registro e inicio de sesion de los usuarios.
- RF-10: El sistema debe permitir deshacer un pedido previamente confirmado.
- RF-11: El sistema debe registrar y permitirle al administrador consultar una bitácora de las acciones relevantes realizadas por los usuarios.
- RF-12: El sistema debe permitirle a los usuarios ver un calendario con las fechas en las que tienen que realizarse los pedidos de cada producto
- RF-13: El sistema debe permitirle a los usuarios acceder a su perfil.

En cuanto a los requisitos no funcionales de Thary, se identificaron los siguientes:

- RNF-01: Si el sistema es utilizado por mas de una organizacion, los datos de una no deben de ser accesibles para otra
- RNF-02: El sistema debe de poder procesar archivos de distintos dominios de inventario (salud, alimentos, ferreteria, etc) siempre y cuando contengan los datos minimos requeridos.
- RNF-03: El sistema debe poder generar una estimacion incluso si el historial de producto presenta datos faltantes.

3.2.3 Especificación de Requisitos de Software (SRS)

A partir de los requisitos especificados, se elaboro la especificacion de requisitos funcionales (SRP), la cual fue estructurada mediante casos de uso y tomando como referencia los lineamientos de la norma IEEE 29148 para la elaboracion de los mismos. El sistema contempla dos actores principales, el administrados y el personal de inventario. El administrador es el tipo de usuario que mas privilegios tiene, es responsable de cargar el historial de consumo e iniciar el procesamiento y gestionar las cuentas de los usuarios. Mientras que el personal de inventario solo usa el sistema para consultar las predicciones y, entre otras actividades, gestionar los pedidos sugeridos. La figura debajo representa los casos de uso correspondientes al Administrador:

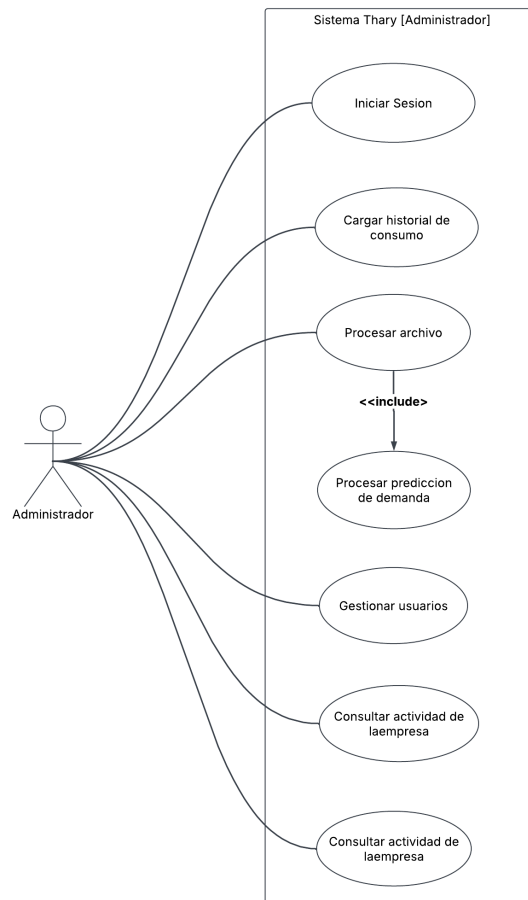


Fig. 3. Casos de uso del Administrador

Por su parte, la siguiente figura muestra los casos de uso correspondientes al Personal de Inventario:

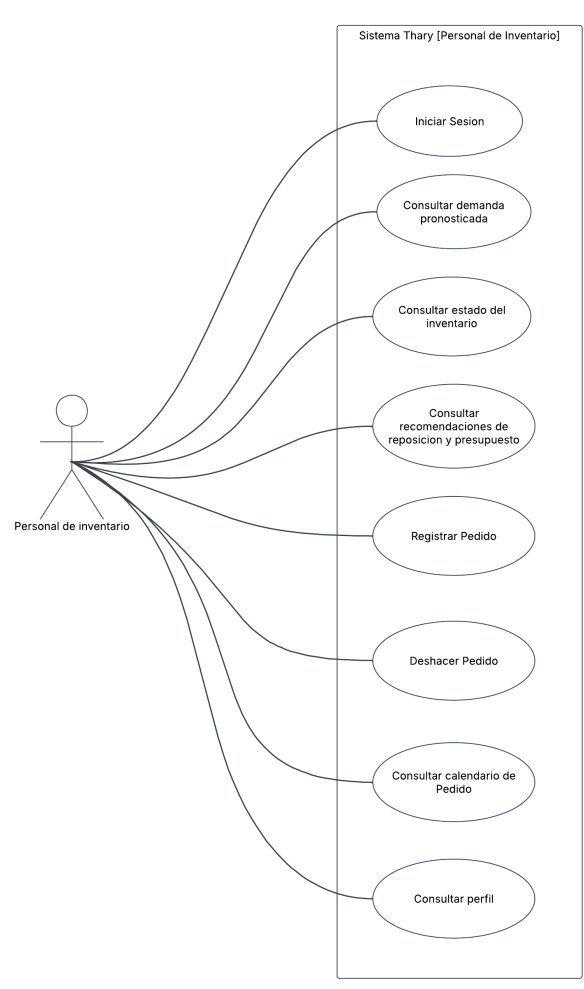


Fig. 4. Casos de uso del Personal de Inventario

Como se puede ver en los diagramas, los casos de uso identificados se relacionan con los requisitos funcionales de la siguiente manera:

TABLA IV
Trazabilidad entre casos de uso y requisitos funcionales

ID	Caso de uso	Actor principal	Requisitos relacionados
CU-01	Iniciar sesión	Usuario registrado	RF-09
CU-02	Cargar historial de consumo	Administrador	RF-01, RF-07
CU-03	Procesar archivo	Administrador	RF-02, RF-07
CU-04	Consultar demanda pronosticada	Personal de Inventario	RF-02, RF-05
CU-05	Consultar estado del inventario	Personal de Inventario	RF-04, RF-05
CU-06	Consultar recomendaciones de reposición y presupuesto	Personal de Inventario	RF-03, RF-05
CU-07	Registrar pedido	Personal de Inventario	RF-06
CU-08	Gestionar usuarios	Administrador	RF-08
CU-09	Deshacer pedido	Personal de Inventario	RF-10
CU-10	Consultar actividad de la empresa	Administrador	RF-11
CU-11	Consultar calendario de pedidos	Personal de Inventario	RF-12
CU-12	Consultar perfil	Usuario registrado	RF-13

- Caso de uso CU-01 (Iniciar sesión (Usuario registrado)): Este caso de uso corresponde al RF-09, y le permite a cualquier usuario registrado ingresar al sistema mediante sus credenciales para acceder a las funcionalidades según su rol.
- Caso de uso CU-02 (Cargar historial de consumo (administrador)): Este caso de uso corresponde al RF-01, y permite al administrador seleccionar y cargar el archivo csv con el historial de consumo.
- Caso de uso CU-03 (Iniciar procesamiento(Administrador)): Corresponde al requerimiento RF-02, que hace referencia a que una vez cargado el archivo, el administrador debe de seleccionar la opción "Procesar archivo" para que el sistema inicialice el procesamiento de los datos.
- Caso de uso CU-04 (Consultar demanda pronosticada (Personal de inventario)): Este caso de uso corresponde al RF-02 y al RF-05, y le permite al usuario consultar la demanda mensual estimada para cada producto una vez el archivo ya ha sido procesado.
- Caso de uso CU-05 (Consultar estado del inventario (Personal de inventario)): Este caso de uso corresponde a RF-04, que permite al usuario visualizar tanto el inventario disponible como los productos que presentan condiciones de alerta.

- Caso de uso CU-06 (Consultar las recomendaciones de reposición y presupuesto (Personal del inventario)): Corresponde al requisito RF-03, y le permite al usuario consultar las recomendaciones generadas por el sistema y la información asociada al presupuesto disponible
- Caso de uso CU-07 (Registrar pedido (Personal de inventario)): Este se relaciona directamente con el RF-06, y permite marcar como completado las recomendaciones de pedido generadas por Thary. Los pedidos confirmados quedan registrados con el nombre de la persona que los completa.
- Caso de uso CU-08 (Gestionar cuentas de usuario(administrador)): Le permite al administrador el poder crear nuevas cuentas al personal del inventario.
- Caso de uso CU-09 (Deshacer pedido (Personal de inventario)): Corresponde al RF-10, y le permite al personal de inventario revertir un pedido previamente confirmado, en caso de haberse equivocado o de que la confirmación ya no sea necesaria.
- Caso de uso CU-10 (Consultar actividad de la empresa (Administrador)): Este caso de uso corresponde al RF-11, y le permite al administrador consultar la bitácora completa de acciones realizadas por todos los usuarios de su empresa: carga de archivos, confirmación o reversión de pedidos, creación de usuarios, con fines de auditoría.
- Caso de uso CU-11 (Consultar calendario de pedidos (Personal de inventario)): Se relaciona con el RF-12, y le permite al usuario visualizar en un calendario las fechas estimadas en las que debe realizarse el pedido de cada producto, según las recomendaciones generadas por el sistema.
- Caso de uso CU-12 (Consultar perfil (Usuario registrado)): Este caso de uso corresponde al RF-13, y le permite a cualquier usuario, sin importar su rol, poder consultar los datos de su propia cuenta y el historial de las acciones que el mismo ha realizado en el sistema.

Todos los casos de uso se documentan individualmente en la especificación SRS presentada en el Anexo A, la cual contempla detalles como objetivo, actor principal, precondiciones y postcondiciones, flujo principales y alternativos.

En cuanto a elementos como las precondiciones, estos se establecen antes de iniciar una funcionalidad, mientras que las postcondiciones describen cuál es el estado esperado del sistema al finalizar el proceso, es de esta manera que realizar una correcta especificación de casos permite establecer una base a la hora de continuar con el proceso de diseño de arquitectura e implementación de servicios.

3.2.4 Diseño de la Arquitectura

El sistema Thary fue diseñado basandose en una organizacion pipeline para el componente de prediccion y un estilo arquitectonico en capas de las cuales se distinguen tres principales:

- Capa de Presentacion: Esta capa es la reponsable de funcionalidades como carga de archivos, visualizacion de resultados, registro de pedidos y demas funcionalidades relacionadas con toda la interaccion que el usuario final va a tener con el sistema.
- Capa de Control: Esta capa es la encargada de recibir todas las solicitudes que el usaurio le haga a la aplicacion, y devolver los resultados esperados a la capa de presentacion
- Capa de servicios de dominio: En esta capa se encuentra el componente de prediccion organizado como un pipeline secuencial compuesto por las diferentes etapas del sistema, en general, en esta capa se concentra toda la logica de Thary.

En cuanto a porque se decidió llevar la organizacion mediante un pipeline, esta desicion se tomo debido a que el proceso de prediccion de demanda, por su naturaleza, corresponde a un flujo secuencial de transformacion de los datos, por lo que el uso de esta estructuta facilita que se pueda separar responsabilidades de una manera mas sencilla y la trazabilidad de los resultados obtenidos.

Por otro lado, a la hora de elegir como se manejaría la persistencia de la informacion, se opto por usar un modelo de base de datos relacional (MySQL).Esta desicion se tomo considerando la necesidad de poder contar con un historial con las corridas anteriores del sistema. Adicional a esto, la arquitectura de Thary contempla el aislamiento de los datos para cada organizacion, de manera que todas cuentan con un espicio independiente evitando que puedan acceder a informacion de otro usuario.

3.2.5 Escenarios Arquitecturales

Para garantizar que Thary cuenta con una arquitectura que es capaz de responder a los atributos de calidad necesarios para un proyecto con sus características, se plantaron los siguientes escenarios:

- Escenario 1 (Confiabilidad ante datos escasos): Este escenario hace referencia a que cuando un usuario carga el historial de un producto con menos de cuatro meses de registros, Thary no debe de fallar ni excluir el producto, en su lugar, debe de utilizar un metodo de estimacion de respaldo, como la media movil. Esto le aporta valor al sistema cuando no exista suficiente informacion historica para usar un modelo mas complejo.

- Escenario 2 (Desempeño ante catálogos extensos): Cuando se procesa un archivo con un número alto de productos diferentes, los cálculos de variables como puntos de reposición y cantidades sugeridas deben poder completarse de manera eficiente, sin que el tiempo de procesamiento se degrade de forma desproporcionada al tamaño del catálogo.
- Escenario 3 (Confidencialidad de la información): En este escenario se tiene en cuenta que Thary debe garantizar que los datos relacionados con el inventario y el historial de consumo de la organización, no se transmitan a ningún servicio externo del que Thary se encuentra alojado.
- Escenario 4 (Aislamiento entre organizaciones): Al trabajar con tantos usuarios/administradores, lo más probable es que en algún momento, al menos dos organizaciones distintas usen el sistema de manera simultánea, Thary debe asegurarse que ningún tipo de dato quede expuesto ni pueda ser modificado por otra organización.
- Escenario 5 (Adaptabilidad del formato de datos): Este escenario se refiere a que cuando se cargue un archivo que pertenezca a un dominio diferente al caso de estudio principal, el cual fue salud, Thary tiene que ser capaz de reconocer y normalizar las columnas sin tener que modificar el código.

Estos escenarios permiten poder verificar que las decisiones arquitectónicas tomadas respondan a las necesidades que tiene el sistema, facilitando que posteriormente se pueda comprobar el funcionamiento de Thary.

3.2.6 Análisis de Alternativas y Decisiones de Diseño (ADRs)

Las decisiones de diseño que fueron adoptadas durante todo el desarrollo del sistema Thary y las alternativas consideradas.

- Decisión 1 (Persistencia basada en base de datos relacional): Debido a que el sistema va a trabajar con información relacionada con los usuarios y los historiales de consumo que compartan con el sistema, se tuvo que seleccionar la manera más adecuada de almacenar dicha información. En una primera versión de la producción del sistema, se decidió que se utilizaran archivos estructurados independientes para cada la información de cada organización. Sin embargo, una vez se quiso mantener un registro de eventos como historial de corridas o de actividades, se migró hacia un motor de base de datos relacional MySQL. La elección de esta alternativa además de ofrecer dichas posibilidades, permite una mayor integridad de los datos y mejora la capacidad de consulta a cambio de una mayor complejidad.

- Decision 2 (Selección del modelo de predicción por producto): Existen distintos productos del inventario que pueden presentar patrones de demanda diferentes entre sí, teniendo esto en cuenta, al inicio del desarrollo se propusieron dos alternativas, la primera era generalizar para todo el sistema el uso de un mismo modelo, y la segunda implementar tres modelos distintos. Se optó que la mejor opción es entrenar diferentes modelos candidatos (XGBoost, regresión lineal, y media móvil), comparar los resultados de los 3 y dependiendo de si el consumo es estable, esporádico o muy variable, seleccionar el que presente el menor error de predicción, esto basándose en el cálculo de el MAE (Error absoluto medio). La justificación del porque se eligió esta métrica es que el MAE es una medida que se expresa en las mismas unidades que la demanda, lo que facilita que se pueda interpretar más fácilmente.
- Decision 3 (Validación temporal y validación cruzada): A la hora de validar los resultados de Thary, los cuales son conseguidos a partir de series de tiempo (historial de consumo mensual), se pensó en dos propuestas, la primera se basaba en hacer una validación cruzada aleatoria, que supone hacer una partición sin considerar el orden cronológico, mientras que la segunda se basaba en una validación cruzada temporal de origen móvil, en el que se generan varias particiones de entrenamiento y pruebas consecutivas. Se decidió optar por la segunda opción, ya que la primera terminaría mezclando datos del futuro, lo cual se quiere evitar ya que la idea es representar un escenario real de uso del sistema, en el que no se tendrían estos datos del futuro. Es por esto que a pesar de que repetir el entrenamiento sobre varias particiones en lugar de una sola representa un mayor costo computacional, se eligió porque resulta más representativa al desempeño real que se espera que tenga el sistema.

3.2.7 Representación de la Arquitectura (Modelo C4)

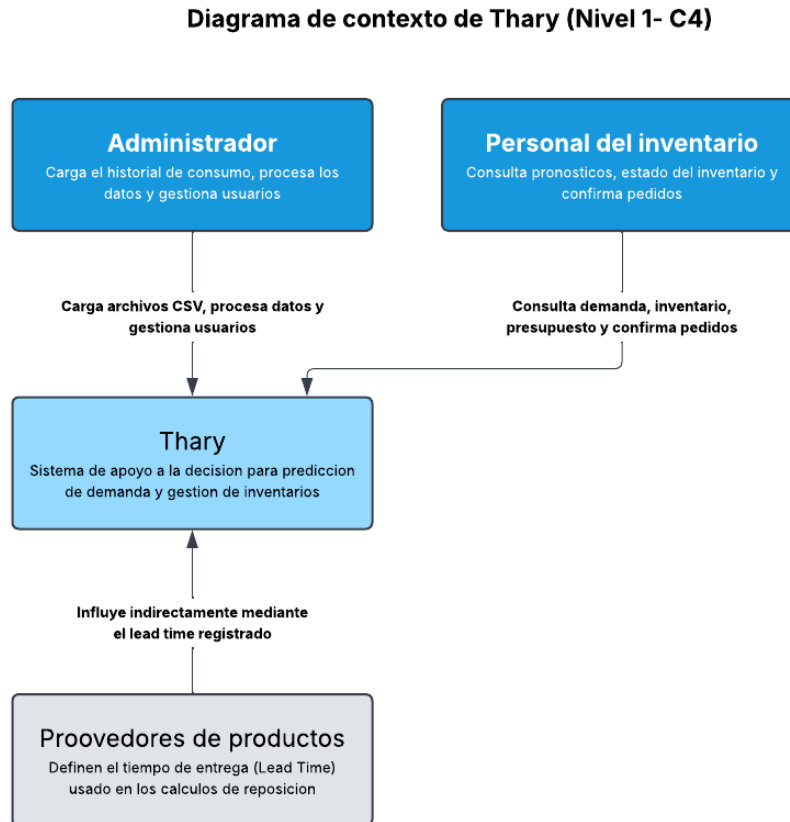


Fig. 5. Diagrama de contexto del sistema (nivel 1 del modelo C4).

Como se observa en la Fig. 5, el sistema interactúa con los actores identificados en la sección 3.2.1.

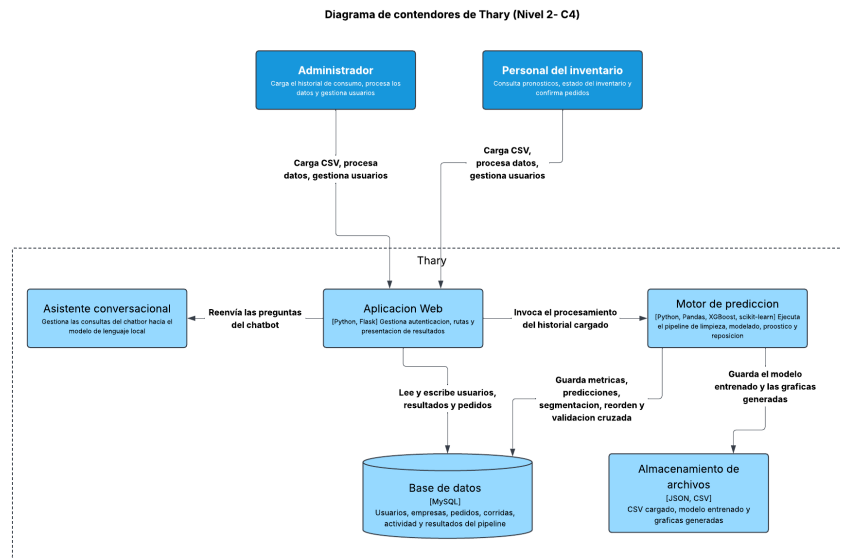


Fig. 6. Diagrama de contenedores del sistema (nivel 2 del modelo C4).

En la figura 6 se puede observar como se organizaron los contenedores del sistema Thary, factores como la aplicación web, el motor de predicción y la base de datos son contenedores independientes que se comunican entre sí para el correcto funcionamiento del sistema.

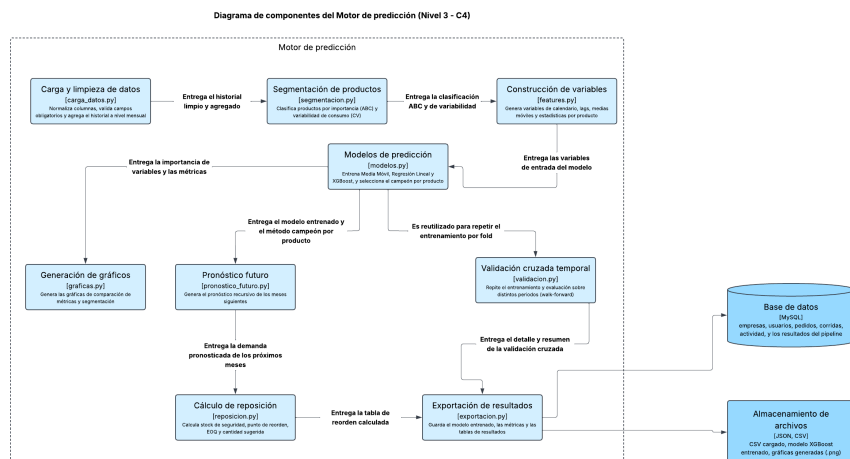


Fig. 7. Diagrama de componentes del sistema (nivel 3 del modelo C4).

Finalmente, el diagrama de componentes detalla a profundidad el funcionamiento interno del motor de predicciones.

3.2.8 Diseño Detallado

A continuación, se describe el diseño detallado del sistema Thary, el cual consiste en las siguientes etapas del pipeline:

3.2.9 Etapa 1: Carga y limpieza de datos

La primera etapa del pipeline (`app/services/prediccion/carga_datos.py`) recibe el archivo CSV cargado por el administrador y lo normaliza hacia un esquema interno unico:

Código 1. Función `cargar_y_limpiar` y etiquetas de las columnas obligatorias.

```
ETIQUETAS_COLUMNAS_OBLIGATORIAS = {
    "fecha": "Fecha (o Date, Sale_Date)",
    "producto_id": "Producto (o Product, Product_ID)",
    "demanda": "Demanda (o Demand, Sales, Quantity_Sold)",
}

def cargar_y_limpiar(ruta_archivo):
    try:
        df = pd.read_csv(ruta_archivo)
    except pd.errors.EmptyDataError:
        raise ValueError("El archivo está vacío. Sube un CSV con al menos una fila.")
    except pd.errors.ParserError:
        raise ValueError(
            "No se pudo leer el archivo como CSV. Verifica que sea un archivo  

            separado por comas (por ejemplo, no un Excel renombrado a CSV)."
        )
    except UnicodeDecodeError:
        raise ValueError(
            "El archivo tiene una codificación de caracteres que el sistema no  

            reconoce. Guárdalo como CSV con codificación UTF-8 e intenta de nuevo."
        )

    if df.empty:
        raise ValueError("El archivo no tiene filas de datos, solo encabezados.")
```

```

df = df.rename(columns={
    "Sale_Date": "fecha", "Date": "fecha", "date": "fecha", "Fecha": "fecha",
    "Product_ID": "producto_id", "Product": "producto_id", "Producto": "producto_id",
    "Quantity_Sold": "demanda", "Sales": "demanda", "Demand": "demanda",
    # ... variantes adicionales para precio, inventario, categoria, proveedor
})

columnas_faltantes = [c for c in ETIQUETAS_COLUMNAS_OBLIGATORIAS if c not in df.columns]
if columnas_faltantes:
    faltan = ", ".join(columnas_faltantes)
    raise ValueError(
        f"Al archivo le falta al menos una columna obligatoria: {faltan}. "
        f"Agrégalas al CSV (con ese nombre exacto u otro de los aceptados)."
    )

```

En esta sección del archivo (`app/services/prediccion/carga_datos.py`), se observa la etapa de carga y limpieza de los datos. Para garantizar que el sistema pueda adaptarse y funcionar para todo tipo de dominio de inventarios, se normalizan los nombres de las columnas de los archivos CSV con el fin de mantener un esquema más heterogéneo, de manera que el sistema pueda aceptar archivos de distintos dominios sin tener la necesidad de modificar el resto del pipeline. Para facilitarle al usuario el proceso de cargar un archivo, se capturan los errores mas comunes para el sistema a la hora de leer un CSV y se muestran a manera de mensaje de error en un lenguaje sencillo y facil de entender.

Otra practica que fue aplicada es el manejo de errores con mensajes dirigidos al usuario final, de esta manera, se evita que se propaguen los errores internos del sistema y el usuario sabe que es lo que esta ocurriendo y que es lo que se debe hacer para resolverlo.

3.2.10 Etapa 2: Segmentación de productos

La segunda etapa del pipeline (`app/services/prediccion/segmentacion.py`) se basa en poder clasificar cada producto segun dos criterios principales:

- **Clasificación ABC:** Esta clasificación permite poder identificar cual es la importancia relativa de cada producto dentro de la demanda total. Se calcula ordenando los productos de mayor a menor en base a su demanda total y se guarda su porcentaje.

Los productos que en conjunto representan hasta el 80% de la demanda, se clasifican como categoria A, es decir, los mas importantes. Los que se encuentran entre el 80% y el 95% se clasifican como categoria B. El resto de productos se clasifican como categoria C.

Esta clasificación permite que en el dashboard, el usuario pueda entender que tan importante y que tan errático es el consumo de cada producto

- Variabilidad: La variabilidad del consumo de los productos permite poder identificar cuáles son los productos que presentan un comportamiento más estable y cuáles son los que presentan un comportamiento más irregular, se calcula en base al coeficiente de variación (la desviación estándar dividida entre la media de la demanda), clasificando a cada producto como de variabilidad Baja (menor a 0.5), Media (entre 0.5 y 1) o Alta (mayor o igual a 1).

Código 2. Función `calcular_comportamiento_producto`: variabilidad y clasificación ABC de cada producto.

```
def calcular_comportamiento_producto(df, df_mensual):
    producto_comportamiento = (
        df_mensual.groupby("producto_id")["demanda"]
        .agg(demanda_total="sum", media="mean", desviacion="std")
        .reset_index()
    )
    producto_comportamiento["cv"] = (
        producto_comportamiento["desviacion"] / producto_comportamiento["media"]
    ).replace([np.inf, -np.inf], np.nan)

def clasificar_variabilidad(cv):
    if pd.isna(cv):
        return "Sin datos"
    if cv < 0.5:
        return "Baja"
    elif cv < 1:
        return "Media"
    else:
        return "Alta"

producto_comportamiento["variabilidad"] = producto_comportamiento["cv"].apply(
    clasificar_variabilidad
)

producto_comportamiento = producto_comportamiento.sort_values("demanda_total")
total_demanda = producto_comportamiento["demanda_total"].sum()
if total_demanda > 0:
    producto_comportamiento["demanda_acumulada_pct"] = producto_comportamiento["demanda_total"] / total_demanda
```

```
else:
    producto_comportamiento["demanda_acumulada_pct"] = 0

def clasificar_abc(pct):
    if pct <= 0.80:
        return "A"
    elif pct <= 0.95:
        return "B"
    else:
        return "C"

producto_comportamiento["categoria_abc"] = producto_comportamiento

# ...
return producto_comportamiento
```

Es en esta etapa en la que se calculan estadísticas históricas como (`media_producto` y `desviacion_producto`).

3.2.11 Etapa 3: Construcción de variables

En esta etapa, (`app/services/prediccion/features.py`), se adapta el historial de demanda a un formato que contenga las variables necesarias que los modelos usan como entrada (*features*):

- Componentes de calendario
- Rezagos de la demanda
- Estadísticas móviles
- Estadísticas históricas por producto

La implementación de esta etapa empieza con un merge inicial, en este merge se trae las categorías `abc` y `variabilidad` a la tabla. Aunque no se usan como variables de entrada (`feature_cols`), se incluyen porque posteriormente se necesitarán para mostrarlas en el dashboard y tabla de predicciones:

Código 3. Unión de las categorías ABC y de variabilidad a la tabla mensual.

```
def construir_features(df_mensual, producto_comportamiento):  
    # ...  
    df_mensual = df_mensual.merge(  
        producto_comportamiento[["producto_id", "categoria_abc", "variabi-  
on="producto_id", how="left"  
    )
```

Posteriormente, se extraen tanto el año como el mes de cada ejemplo, esto adaptado del repositorio *Demand Forecasting & Inventory Optimization for Consumer Products* [10], al que en este trabajo se le referirá como *Ejemplo 1 (SKU Demand Forecasting)*. Con la diferencia de que en Thary, estos datos se incluyeron con el fin de en la implementación que se tomó como guía, el mes "month", queda como un feature final para el modelo, mientras que en Thary, este dato posteriormente sea transformado en `mes_sin` y `mes_cos`, proceso que se explicará más adelante:

Código 4. Extracción del año y del mes de cada registro.

```
df_mensual["año"] = df_mensual["fecha"].dt.year  
df_mensual["mes"] = df_mensual["fecha"].dt.month
```

Un rezago (lag), se usa en Machine Learning y hace referencia al valor anterior a una variable, en este caso, representa el valor de la demanda del mes anterior, el cual es usado como dato de entrada para predecir el mes actual. Los lags junto con las medias y desviaciones móviles se encargan de capturar el comportamiento reciente de la demanda de cada producto. En Thary, para cada producto se tiene:

- lag_1: demanda del mes anterior
- lag_2: demanda de hace dos meses
- lag_3: demanda de hace tres meses

El modelo los usa como features porque la demanda pasada ayuda a poder determinar cuánto se va a vender el próximo mes. Por eso es que un producto necesita mínimo 3 meses de historial antes de los primeros 3 meses del dataset para poder calcular lag1, lag2 y lag3. El problema era que el sistema solo conoce los datos que vienen dentro del mismo archivo CSV que el usuario sube, por lo que no tiene forma de saber qué pasó antes del primer mes que aparece en el archivo.

Si un dataset empieza en enero de 2021, el lag1 de enero necesitaria la demanda de diciembre de 2020, dato que no esta en el historial dado por el usuario, solo seria a partir de abril 2021 que el sistema contaria con los 3 lags comparados correctamente. Es debido a que no hay informacion suficiente para calcular los lags correctamente, que el sistema descarta los primeros meses a la hora de entrenar el modelo mediante (`dropna()`). Como esos primeros 3 meses se descartan, el valor de mes de esos meses nunca aparece como algo que el modelo tuvo que predecir durante el entrenamiento, por lo que estaria el riesgo de que, si mas adelante se le pide al modelo predecir justo ese mismo mes del año siguiente (por ejemplo, enero 2022), el modelo no tenga ninguna referencia previa de que numero de mes es ese, y termine adivinando mal, generando predicciones muy bajas o poco confiables para ese mes en particular

Para evitar este problema, se decidio que durante esta etapa de features, se representaria el mes utilizando seno y coseno en vez de usar directamente el numero del mes (1-12). Esto permite que el modelo interprete el mes como un punto continuo cercano a los meses vecinos que si fueron entrenados, por ejemplo, diciembre y enero quedan numericamente uno cerca del otro, lo cual tiene mucho mas sentido que hacer que el sistema los vea como el numero 12 y 1, valores mucho mas lejanos de lo que deberian ser para dos meses vecinos.

Una vez que el modelo entienda la cercania real de los meses, es capaz de dejar de interpretarlos como categorias independientes, y puede generar una prediccion razonable de un mes que nunca halla visto antes (enero, por ejemlo), en base la informacion de un mes cercano (diciembre).

A la hora de armar la funcion para construir los features se tomaron en cuenta diferentes implementaciones previas de distintos estudios con diferentes propuestas para un sistema de prediccion de demanda, se compararon y se seleccionaron aquellas que se apataban mejor a las necesidades del sistema y le aportaban valos al mismo, junto con la implementacion de nuevas ideas y mejoras que se fueron propuestas e implemantadas en Thary. Una de estas propuestas nuevas fue la codificacion ciclica del mes, que como ya se explico antes, consiste en representar los meses mediante dos valores calculados con seno y coseno, como se puede ver a continuacion:

Código 5. Codificación cíclica del mes mediante seno y coseno.

```
# ...
df_mensual["mes_sin"] = np.sin(2 * np.pi * df_mensual["mes"] / 12)
df_mensual["mes_cos"] = np.cos(2 * np.pi * df_mensual["mes"] / 12)
```

Por otra parte, la metodologia de implementar el uso de lags, tambien se tomo del *Ejemplo 1 (SKU Demand Forecasting)* [10] y se adapto a las necesidades del sistema Thary, agregando la codificacion cíclica de los meses y la inclusion de features adicionales como el inventario y el precio.

Código 6. Cálculo de los rezagos lag_1, lag_2, lag_3 y lag_6 por producto.

```
# ...  
# (Ejemplo 1 - SKU Demand Forecasting)  
df_mensual["lag_1"] = df_mensual.groupby("producto_id")["demanda"]  
df_mensual["lag_2"] = df_mensual.groupby("producto_id")["demanda"]  
df_mensual["lag_3"] = df_mensual.groupby("producto_id")["demanda"]  
df_mensual["lag_6"] = df_mensual.groupby("producto_id")["demanda"]
```

Otra estrategia que se tomo del *Ejemplo 1 (SKU Demand Forecasting)* [10], fue el uso de las estadísticas móviles. Estas aportan al sistema ya que consisten en un resumen de varios meses juntos. Mientras que los rezagos solo representan cuanto se vendio exactamente en un mes, las estadísticas móviles son un resumen de varios meses juntos.

- `media_movil_3` y `media_movil_6`: Estas variables representan cual es el nivel reciente de la demanda de los productos calculando el promedio de los ultimos 3 y 6 meses. Esto ayuda que el modelo pueda distinguir un producto que vende de manera estable de uno irregular, ya que de ser ventas regulares, la media movil de 3 meses sera muy similar a la de 6 meses, mientras que si es inestable, ambas medias seran muy diferentes.
- `desviacion_movil_3`: Representa que tan erratica fue la demanda reciente, es decir, de los ultimos 3 meses. Los productos que cuentan con demanda estable tendran una desviacion baja mientras que los productos con ventas inestable tendran una desviacion alta.

En general, el modelo puede ver el comportamiento general reciente de los productos mas alla de solo los lags.

En cuanto a la implementacion, se incluyo en la funcion de (`construir_features`), en el que cada línea calcula por producto (`groupby("producto_id")`) una estadística sobre una ventana de meses anteriores (`.shift(1)`) para no incluir el mes actual, evitando fuga de datos.

Código 7. Cálculo de las estadísticas móviles `media_movil_3`, `media_movil_6` y `desviacion_movil_3`.

```
# ...
# (Ejemplo 1 - SKU Demand Forecasting)

df_mensual["media_movil_3"] = df_mensual.groupby("producto_id")["d
    lambda x: x.shift(1).rolling(3).mean()
)
df_mensual["media_movil_6"] = df_mensual.groupby("producto_id")["d
    lambda x: x.shift(1).rolling(6).mean()
)
df_mensual["desviacion_movil_3"] = df_mensual.groupby("producto_id
    lambda x: x.shift(1).rolling(3).std()
)
```

Por otra parte, para la construcción de la etapa de Features también se tomó en cuenta el repositorio en Kaggle *Forecasting Inventory Demand Using XGBoost* [12] o *Ejemplo 3 (XGBoost)* [12]. De este repositorio, se tomó el uso de el promedio del producto, pero esta vez, calculados en base a todo el historial completo y no solo en base a los últimos 3 o 6 meses.

Como extensión propuesta por el proyecto, junto al promedio también se incluyó la desviación del producto, esto para poder seguir una misma estructura en todo el sistema, ya que si se tiene la (`media_movil_3`) y la (`desviacion_movil_3`), es decir, el nivel y la variabilidad de los últimos 3 meses, es conveniente tener la media y desviación de todo el historial del producto.

Estos valores ya fueron calculados en la etapa de segmentación, y simplemente son agregados en los features para que el modelo también los use como variables de entrada.

Código 8. Incorporación de la media y la desviación históricas de cada producto.

```
# ...
# (Forecasting Inventory Demand Using XGBoost)
stats_cols = ["demanda_total", "media", "desviacion"]
df_mensual = df_mensual.merge(
    producto_comportamiento[["producto_id"] + stats_cols].rename(
        columns={"media": "media_producto", "desviacion": "desviac
    ),
    on="producto_id", how="left"
)
```

Después, se calcula cuanto stock había el mes anterior. La condición (`if "inventario" in df_mensual.columns:`) determina si el csv que fue subido por el usuario contiene la columna "inventario", esta es una columna opcional que simplemente representa cuantas unidades de ese producto (stock disponible) había físicamente en el inventario en ese mes, Thary funciona sin ella, pero si el usuario la incluye, el modelo la usará como una variable de entrada más para poder brindarle un análisis mucho más detallado.

(`inventario_lag_1`) es el inventario del mes anterior, con (`.groupby("producto_id")`) se asegura que este se calcule por separado para cada producto, esto con la intención de evitar que el inventario de un producto se vaya a mezclar con otro, esto usando (`shift(1)`) para poder obtener el inventario del mes anterior. Si no se hace esto, el modelo podría aprender que el inventario de un producto es igual al del mes actual, lo cual no es correcto y generaría predicciones erróneas.

Código 9. Cálculo del inventario del mes anterior (`inventario_lag_1`).

```
# ...
# (Forecasting Inventory Demand Using XGBoost))
if "inventario" in df_mensual.columns:
    df_mensual["inventario_lag_1"] = df_mensual.groupby("producto_
```

En esta parte, se implementó otra idea del *Ejemplo 1 (SKU Demand Forecasting)* [10] para poder descartar las filas sin historial suficiente para tener los lags. Para esto, se define una lista con nueve nombres de las columnas obligatorias para poder entrenar, con (`dropna(subset=feature_history)`) se revisan las 4 columnas y elimina cualquier fila que tenga un espacio vacío, posteriormente el resultado se guarda en (`df_model`), que es la tabla final que se usará para entrenar el modelo.

Código 10. Descarte de las filas sin historial suficiente para calcular los rezagos.

```
# ...
# (Inventory Demand Forecast (XGBoost Reg))
feature_history = ["lag_1", "lag_2", "lag_3", "media_movil_3"]
df_model = df_mensual.dropna(subset=feature_history).copy()

if len(df_model) == 0:
    raise ValueError(
        "Ningún producto tiene al menos 3 meses seguidos de historial
        para calcular tendencias. Sube un archivo con más meses de datos."
    )
```

Otro de los conceptos que fueron propuestos en Thary es el tratamiento que se decidió darle a la variable con los precios de venta de los productos. Aunque el CSV pueda incluir esa variable, el modelo solo usa su valor actual (el precio de ese mes), sin calcular cómo cambió respecto al mes anterior. Esto debido a que por naturaleza, el precio de un producto puede influir en la demanda de un producto, ya que cuando algo baja de precio, la gente tiende a comprar más, mientras que cuando este sube, las personas compran menos. El problema es que si esta regla se repite muy seguido, se corre el riesgo de que el modelo aprenda que el cambio de precio predice muy bien la demanda, entonces, aunque el precio es una variable que si se tiene en cuenta, se excluye por completo su variabilidad de un mes a otro.

Código 11. Lista de variables de entrada (`feature_cols`) y variables opcionales de precio e inventario.

```
# ...
feature_cols = [
    "año", "mes_sin", "mes_cos",
    "lag_1", "lag_2", "lag_3", "lag_6",
    "media_movil_3", "media_movil_6", "desviacion_movil_3",
    "media_producto", "desviacion_producto"
]
if "precio" in df_model.columns:
    feature_cols.append("precio")
if "inventario" in df_model.columns:
    feature_cols.append("inventario")
    if "inventario_lag_1" in df_model.columns:
        feature_cols.append("inventario_lag_1")
```

Finalmente, como extensión propia de Thary, se hace un relleno final. Para esto, se recorre cada una de las columnas que van a usar los modelos, si hay alguna fila vacía en la tabla, se reemplazan los vacíos de la columna con 0 mediante (`df_model[col] = df_model[col].fillna(0)`). Esto es importante porque si el modelo recibe un producto que solo aparece a partir de cierto mes, por ejemplo, abril 2021 en un dataset que empieza en enero de ese mismo año, el mes de abril pudo pasar el filtro de `dropna()` porque ya tiene (`lag_1`, `lag_2`, `lag_3` y `media_movil_3` completos), pero en este caso, el `lag_6` necesitaría la demanda de octubre de 2020, mes que no existe en el archivo porque el producto todavía no tiene 6 meses de historial.

La fila pasaría el filtro porque `lag_6` no está en `feature_history`, pero inevitablemente terminaría en `feature_cols`, y ahí es donde se ejecuta este bloque final, si `lag_6` es vacío en esa fila se rellena con 0.

Código 12. Relleno final de los valores vacíos de las variables de entrada con 0.

```
# ...
for col in feature_cols:
    if df_model[col].isna().any():
        df_model[col] = df_model[col].fillna(0)
```

3.2.12 Etapa 4: Modelos de predicción

Esta etapa se reparte en los siguientes archivos:

- app/services/prediccion/modelos.py
- app/services/prediccion/regresion_lineal.py

Definición de variables y funciones básicas: Al inicio del archivo `modelos.py`, se define el nivel de significancia `NIVEL_SIGNIFICANCIA_DM` como 5%, esta variable se va a usar posteriormente a la hora de evaluar si un resultado es estadísticamente significativo:

Código 13. Nivel de significancia de la prueba de Diebold-Mariano.

```
NIVEL_SIGNIFICANCIA_DM = 0.05
```

Como parte de la propuesta de Thary, también se implementaron las siguientes cuatro variables:

Código 14. Constantes que definen el tamaño del periodo de prueba (*backtest*).

```
MESES_ARCHIVO_UMBRAL_TEST_GRANDE = 18
TEST_SIZE_MESES_DEFAULT = 3
TEST_SIZE_MESES_GRANDE = 6

MESES_PERDIDOS_POR_REZAGOS = 3
```

Por defecto, el periodo de prueba son 3 meses, pero cuando se cuenta con un periodo de prueba mas grande, de 18 meses o mas, este pasa de 3 meses a 6 meses.

Esta decisión se tomó después de identificar un problema con el que contaba el sistema. Como punto de partida, el sistema elige el mejor modelo para cada producto comparando el MAE de los 3 modelos sobre 3 meses del historial, el problema era que con solo 3 meses, el resultado podría no ser lo suficientemente confiable, esto debido a que con tan pocos datos, un solo mes que sea atípico puede terminar haciendo que un modelo "gane" sin que eso refleje una diferencia real, sobre todo porque el MAE es una medición que cuenta con mucha varianza. Entre los riesgos identificados

se encuentra: poca capacidad de distinguir modelos parecidos, inestabilidad del resultado y sesgo estacional.

Antes de llegar a la solución definitiva se planteó una primera propuesta, en la que si el archivo contaba con menos de 18 meses, se haría la comparación del MAE de los 3 modelos sobre 3 meses, pero si contaba con 18 meses o más, la comparación se haría sobre 6 meses. Pero en realidad, esta solución no resolvía el problema "de raíz", solo mitigaba el problema de que hubiera pocos meses de prueba, además de que la decisión de que 18 meses fuese el punto de corte no tenía respaldo alguno.

Por eso fue que se corrigió y re-interpretó el enfoque de la solución, fundamentándola en que el problema a solucionar no era el número de meses, si no el cálculo del error, es decir, que tan confiable es la comparación de error entre los modelos.

Fue aquí donde se propuso la prueba Diebold-Mariano, la cual consiste en una prueba estadística que permite medir si la diferencia de error que hay entre dos modelos es significativa o es solo ruido. La prueba Diebold-Mariano requiere de un nivel de significancia, que se definió como 5%.

Por otro lado, `MESES_PERDIDOS_POR_REZAGOS` es el número fijo de meses que se pierden al calcular los rezagos (visto en la etapa anterior), es necesario para distribuir los meses utilizables del total de meses que subió el usuario en el archivo.

Una vez definidas las variables, la primera función implementada en `modelos.py` es `tamano_test_backtest`. Esta función principalmente se encarga de implementar la solución ya explicada, recibe como parámetro `n_meses_usables`, la cantidad de meses que quedaron disponibles después de que se hayan perdido los primeros por el cálculo de los lags, y devuelve cuántos meses de esos son los que se van a usar como período de prueba (backtest).

Código 15. Función `tamano_test_backtest`, que decide el tamaño del período de prueba.

```
def tamano_test_backtest(n_meses_usables):  
    n_meses_archivo_original = n_meses_usables + MESES_PERDIDOS_POR_RE  
    if n_meses_archivo_original >= MESES_ARCHIVO_UMBRAL_TEST_GRANDE:  
        return TEST_SIZE_MESES_GRANDE  
    return TEST_SIZE_MESES_DEFAULT
```

Como el umbral de decisión (`MESES_ARCHIVO_UMBRAL_TEST_GRANDE = 18`) toma en cuenta el archivo completo, y la función recibe la cantidad de meses disponibles ya con los 3 meses de lag descontados, con `n_meses_archivo_original = n_meses_usables + MESES_PERDIDOS_POR_REZAGOS` se le suma esos 3 meses de vuelta, para saber cuántos meses tenía el archivo que el usuario subió originalmente.

Luego se compara si el total de meses completo cuenta con 18 meses, de hacerlo devuelve el tamaño de test grande con 6 meses y de no hacerlo devuelve el tamaño de test por defecto de 3 meses.

Esta función se llama desde `sistema_prediccion.py`, para decidir el tamaño del split que se usa para elegir el campeón y generar los resultados, y desde `validacion.py`, para que los *folds* de la validación cruzada temporal usen ese mismo tamaño de test y conservar un criterio estandarizado en ambas partes.

Es parte de la solución que se propone en Thary, esta función no ninguna en alguna de las implementaciones que fueron revisadas ya que ninguna adapta el tamaño del período de prueba según la cantidad de historia disponible.

Modelos de demanda: Las funciones que permiten entrenar y evaluar los tres modelos de demanda que se van a utilizar en el sistema: Media Móvil, Regresión Lineal y XGBoost. Estos fueron elegidos luego de realizar una revisión de literatura en la que se compararon 5 modelos para la predicción de la demanda, tal como se vio en el capítulo 2.2 "Antecedentes", de los cuales se escogieron: XgBoost y regresión lineal al ser los que presentan mejor desempeño en este tipo de problemas, y adicionalmente se incluyó media móvil como un modelo más sencillo que puede ser útil para productos con demanda estable o que no cuenten con suficiente historial implementar para un modelo más complejo.

XgBoost: La implementación de XGBoost se hizo combinando los hiperparámetros de *Ejemplo 1 (SKU Demand Forecasting)* [10] con la transformación del objetivo propuesta en *Ejemplo 3 (XGBoost)* [12].

Ejemplo 3 (XGBoost) [12] cuenta con una configuración muchísimo más simple de la que necesitaría Thary, ya que en el caso del ejemplo, se estaba trabajando con aproximadamente 60 millones de filas, por lo que se priorizó la velocidad sobre el ajuste fino de hiperparámetros, por lo que no cuenta con factores como algún control de sobreajuste:

Código 16. Parámetros del modelo XGBoost del *Ejemplo 3*.

```
# Define the parameters for the XGBoost model
params = {
    'max_depth': 8,                # Maximum depth of a tree
    'eta': 0.5,                   # Learning rate
    'objective': 'reg:squarederror' # Objective function for regression
}
```

Por otra parte, el *Ejemplo 1 (SKU Demand Forecasting)* [10] sí cuenta con una configuración más completa que se podría reutilizar, en este se arma un `XGBRegressor` más cuidadoso en el que se incluyen variables como `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, pero principalmente se escogió seguir la lógica de este ejemplo porque de los cuatro ejemplos revisados, es el único que consiste en un sistema completo con una

premisa bastante similar a la de Thary, ya que ofrece pronóstico de demanda, traducción de ese pronóstico en recomendaciones de reposición, punto de reorden, EOQ, y todo eso bajo restricciones de lead time y presupuesto.

Es por eso, que se adoptó su enfoque general completo:

- Variables
- modelos
- hiperparámetros
- comparación de modelos
- lógica de reposición

Los otros ejemplos se revisaron posteriormente cuando se necesitó mejorar aspectos o agregar nuevas funcionalidades o en general, agregarle más valor y robustez a Thary.

Los hiperparámetros utilizados son los mismos que se encuentran en el *Ejemplo 1 (SKU Demand Forecasting)* [10]. Como ya se explicó anteriormente, al ser el ejemplo más cercano a una implementación completa de un sistema con las características de Thary, se optó por mantener su lógica de entrenamiento y ajuste de hiperparámetros, con la diferencia de que `n_estimators` se aumentó a 500, `objective` se cambió a `reg:squarederror`, mientras que en el ejemplo original no se especificaba. Esto se cambió ya que se buscaba implementar el uso de mínimos cuadrados para poder estimar la bondad de los modelos.

Parámetros como `verbosity` fueron omitidos en Thary, pero se incluyó `n_jobs`, el cual permite utilizar todos los núcleos de procesamiento disponibles en la máquina donde se ejecuta el entrenamiento, en lugar de un único núcleo. Esto contribuye a reducir el tiempo de entrenamiento del modelo cuando se procesan catálogos con un número elevado de productos.

Código 17. Función `entrenar_xgboost`, con los hiperparámetros del modelo.

```
def entrenar_xgboost(X_train, y_train_log, X_test):  
    model = xgb.XGBRegressor(  
        objective="reg:squarederror", n_estimators=500,  
        learning_rate=0.05,  
        max_depth=4,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        reg_alpha=0.1,  
        reg_lambda=1.0,
```

```

        random_state=42,
        n_jobs=-1
    )
    model.fit(X_train, y_train_log)
    pred_xgb = model.predict(X_test)
    pred_xgb = np.maximum(np.expml(pred_xgb), 0)
    return model, pred_xgb

```

En cuanto a la transformación logarítmica, se implementó siguiendo la lógica del *Ejemplo 3 (XGBoost)* [12], en el que el modelo no entrena directamente sobre `y_train`, que sería la demanda real, si no sobre `y_train_log`, que es la transformación logarítmica de `y_train`.

Esto porque en la vida real, la demanda suele tener una distribución en la que puede haber muchos productos con demanda baja y pocos con picos muy altos. Si se entrenara directamente sobre esos valores el modelo podría enfocarse demasiado en los pocos casos extremos, esto llevaría a que se prediciera peor la mayoría de los casos normales. El logaritmo lo que hace es parejar esa diferencia: hace que los valores muy grandes dejen de pesar tanto en comparación con los valores más pequeños, así el modelo le presta atención equilibrada a todos los productos y no solo a los que tienen demanda extrema. Se usa `log1p` porque la demanda puede valer 0, y el logaritmo de 0 no existe.

Media Móvil: Media móvil hace referencia al método más simple de los tres. La técnica con la que se implementó también se basa fuertemente en el *Ejemplo 1 (SKU Demand Forecasting)* [10], y consiste en simplemente reutilizar la columna de `media_movil_3` que ya se tienen desde la etapa anterior de features. No se tiene que hacer ningún entrenamiento.

Código 18. Función `prediccion_baseline_movil`, que implementa el modelo de media móvil.

```

def prediccion_baseline_movil(test_df):
    return np.maximum(test_df["media_movil_3"].values, 0)

```

La función recibe como parámetro el `test_df` y posteriormente, para todas las filas, devuelve el valor `media_movil_3` que ya se había calculado anteriormente.

Regresión Lineal: Para implementar regresión lineal, se adaptó la lógica del *Ejemplo 4 (BoomBikes)* [13], al ser el modelo que más pasos requiere, se implementó de manera individual en el archivo `regresion_lineal.py`.

Código 19. Filtro de las columnas sin variación (función `_seleccionar_features_regresion`).

```
def _seleccionar_features_regresion(X_train, y_train, max_p_valor=
    candidatas = [c for c in X_train.columns if X_train[c].std() >
    if len(candidatas) < 2 or len(X_train) <= len(candidatas) + 1:
        return candidatas
```

Inicialmente, se filtran y excluyen las columnas que no varían demasiado, descartando las que tienen una desviación estándar de 0, ya que una desviación estándar con este valor indica que en esas columnas el valor es siempre el mismo número.

con `std() > 0`, esto se hace porque cuando se entrena regresión lineal, no siempre conviene usar todas las variables disponibles porque no todas tienen algo significativo que aportar. Es incluso perjudicial para el modelo, ya que una columna sin variación no le sirve de nada al modelo (no hay nada que "aprender" de un número que nunca cambia).

Posteriormente, se convierten todos los valores de cada columna a un rango entre 0 y 1, para mantener las proporciones relativas con `MinMaxScaler`. Se ajusta `fit_transform` usando solo los datos con los que se hace el entrenamiento, no se puede incluir a los de prueba porque eso haría que el sistema vea los datos que se supone debe predecir.

Código 20. Escalado de las variables con `MinMaxScaler`.

```
try:
    scaler = MinMaxScaler()
    X_train_esc = pd.DataFrame(
        scaler.fit_transform(X_train[candidatas]),
        columns=candidatas, index=X_train.index
    )
```

En esta parte se incluye Recursive Feature Elimination (RFE). Este consiste en un método que entrena varias veces un modelo de regresión lineal, y en cada una se elimina la variable que aporte menos al modelo hasta que al final, se quede con un número controlado de variables, en este caso, tienen que ser 2 variables como mínimo para posteriormente poder realizar el análisis, pero no pueden ser más de 8 variables porque no se pueden dejar demasiadas `n_rfe = max(2, min(8, len(candidatas) - 1))`.

Finalmente, seleccionadas es la lista de columnas que sobrevivieron esta primera etapa, aunque es posible que aun existan variables redundantes:

Código 21. Selección de variables mediante *Recursive Feature Elimination* (RFE).

```

n_rfe = max(2, min(8, len(candidatas) - 1))
rfe = RFE(LinearRegression(), n_features_to_select=n_rfe)
rfe.fit(X_train_esc, y_train)
seleccionadas = [c for c, usar in zip(candidatas, rfe.support_) if

```

La segunda etapa para seleccionar variables consiste en calcular el VIF o Factor de Inflación de Varianza para cada variable. Este se encarga de medir que tan redundante es una variable en comparacion con las demas, el criterio para decidir esto es estimar si una variable se puede predecir con exactitud a partir de las otras, de ser este el caso, su VIF sera muy alto y eso es un problema porque el modelo no podria distinguir cuál de las dos variables redundantes es la que esta causando el efecto.

En la implementacion de Thary, entre las variables que se considerian redundantes se encuentran:

- `inventario` y `inventario_lag_1`: Escencialmente tienden a ser casi la misma variable, en muchos casos, es bastante probable que el inventario de un mes sea bastante parecido al del mes anterior.
- Los lags (`lag_1`, `lag_2`, `lag_3`, `lag_6`): Tambien es comun que la demanda de un producto tienda a ser similar a la de meses anteriores, si un producto vendio bien un mes, es muy probable que tambien venda mes el siguiente mes, lo que a la larga hace que los lags sean variables muy parecidas entre si.
- (`media_movil_3` y `media_movil_6`) con los lags: Que todos los Lags sean similares llevaria a que, inevitablemente, la media movil de 3 meses y la de 6 meses tambien lo sean debido a que se calculan a partir de promediar los lags.
- (`media_producto`) y las medias móviles: (`media_producto`) consiste en el promedio historico general del producto, si los productos cuentan con un comportamiento estable, es probable que sea similar a (`media_movil_3` y `media_movil_6`), por lo que tambien se considera una variable redundante.

Teniendo esto en cuenta, las variables que no se considerarian redundantes son: `año`, `mes_cos`, `mes_sin`, `desviacion_movil_3` y `desviacion_producto`.

Código 22. Cálculo del VIF de las variables seleccionadas.

```

while len(seleccionadas) > 1:
    X_sm = sm.add_constant(X_train_esc[seleccionadas])
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", category=SingularMatrixWarn
        warnings.simplefilter("ignore", category=UserWarning)
        vif = pd.DataFrame({
            "feature": seleccionadas,
            "VIF": [variance_inflation_factor(X_sm.values, i + 1) f
        })

```

Posteriormente, se quitan las variables con un VIF infinito que nisiquiera se puede calcular, ya que de ser este el caso, significaría que la variable es tan redundante con otra que rompe las matemáticas del modelo. Esto ocurriría porque a la hora de calcular los coeficientes de la regresión lineal, el algoritmo necesita calcular cuanto aporta cada variable a la predicción, y para eso, necesita poder distinguir el efecto individual de cada una. El problema es que si dos variables son casi idénticas entre sí, es imposible distinguirlas.

Es por eso, que en esta etapa esas variables son eliminadas. En las siguientes líneas de código, se revisa cual variable tiene el VIF más alto si ya no hay variables con VIF infinito, si el valor del vif más alto supera el límite que se definió en `max_vif`, esa variable se elimina y finalmente se vuelve a repetir el proceso desde el principio hasta que se halla calculado el VIF de las variables restantes y todas estas cuenten con un VIF menor al límite establecido.

Código 23. Eliminación de variables con VIF infinito o superior al límite.

```

vif_problema = vif[~np.isfinite(vif["VIF"])]
if not vif_problema.empty:
    seleccionadas.remove(vif_problema.iloc[0]["feature"])
    continue

peor_vif_filas = vif.loc[vif["VIF"].idxmax()]
if peor_vif_filas["VIF"] > max_vif:
    seleccionadas.remove(peor_vif_filas["feature"])
    continue

```

Una vez completado el proceso de eliminación de las variables redundantes, se prosigue con la implementación del entrenamiento del modelo de regresión lineal, en el que se revisa el "p-valor" de cada variable.

El P-valor consiste en una medida estadística que indica que tan probable es que la variable que se esta revisando este relacionada con la demanda solo por casualidad y no porque de verdad dicha variable este relacionada de verdad con la demanda. En este caso, si se encuentra que la variable que cuenta con el peor p-valor supera el limite de `max_p_valor`, que es estandarizado como 0.05 al representar el 95% de confianza en estadística, se elimina la variable y se repite el proceso. El bucle se detiene cuando ya no haya ninguna variable con el VIF alto o con el P-valor alto, al cumplir con ambos criterios, solo se contara con variables limpias y estadísticamente significativas para el proceso.

De haber algun fallo durante esta etapa, simplemente se devuelve la lista completa de variables con el fin de garantizar de que el pipeline no se rompa y el sistema pueda continuar funcionando.

Código 24. Eliminación de variables según el p-valor del modelo OLS.

```
modelo_ols = sm.OLS(y_train, X_sm).fit()
p_valores = modelo_ols.pvalues.drop("const")
peor_p_nombre = p_valores.idxmax()
peor_p_valor = p_valores[peor_p_nombre]

if peor_p_valor > max_p_valor:
    seleccionadas.remove(peor_p_nombre)
else:
    break

return seleccionadas
except Exception:
    return candidatas
```

Por otro lado, en ese mismo archivo, se implemento la funcion `entrenar_regresion_lineal`, que se encarga de entrenar el modelo de regresion lineal y devolver las predicciones, tambien es a la que se llama desde el resto del sistema.

La funcion llama a `_seleccionar_features_regresion` para obtener la lista de variables que se van a usar en el entrenamiento del modelo, (si por algun motivo, dicha funcion le devuelve una lista que se encuentre vacia, se usan todas las variables originales.)

El modelo es entrenado con las variables limpias asegurandose que las predicciones no sean negativas al no ser algo que tenga sentido en este contexto.

Finalmente, devuelve el modelo entrenado, las predicciones y la lista de variables que se usaron para entrenar el modelo.

Código 25. Función `entrenar_regresion_lineal`, que entrena el modelo con las variables seleccionadas.

```
def entrenar_regresion_lineal(X_train, y_train, X_test, feature_cols):
    features_lr = _seleccionar_features_regresion(X_train, y_train)
    if not features_lr:
        features_lr = list(feature_cols)

    modelo_lr = LinearRegression()
    modelo_lr.fit(X_train[features_lr], y_train)
    pred_lr = modelo_lr.predict(X_test[features_lr])
    pred_lr = np.maximum(pred_lr, 0)

    return modelo_lr, pred_lr, features_lr
```

3.2.13 Selección del campeón por producto

El sistema de predicción Thary no usa el mismo método para todos los productos del catálogo. Para cada uno de los productos se selecciona cual de los tres modelos candidatos usar para su pronóstico. Esta decisión se tomó de manera independiente para el sistema Thary, ya que al revisar las implementaciones de los 4 sistemas de predicción en las que se basó el sistema, se evidenció que estos simplemente usaban el mejor modelo para todo el catálogo por igual.

En una primera versión del sistema, la selección del mejor modelo por producto se hacía en base a cual de los 3 obtuvo un menor MAE. El problema con esta estrategia es que el backtest generalmente solo cuenta con entre 3 y 6 meses de datos si es que se cuenta con un archivo más grande, son pocos meses, por lo que el hecho de que un modelo tenga el MAE más bajo no es prueba de que sea mejor, podría darse por diferentes factores o por casualidad.

Es por eso que para seleccionar el mejor modelo, lo primero que se hace es calcular el MAE de los 3 modelos, se identifican los dos modelos que obtengan el menor MAE para cada producto. Esto usando `np.argsort` sobre la tabla de errores:

Código 26. Identificación del mejor y el segundo mejor modelo de cada producto (función `elegir_campeon_por_producto`).

```
def elegir_campeon_por_producto(resultados_prediccion):
    errores_por_metodo = pd.DataFrame({
        metodo: (resultados_prediccion["demanda"] - resultados_prediccion[metodo])
        for metodo, columna in COLUMNA_PREDICCION_POR_METODO.items()
    })
    productos = resultados_prediccion["producto_id"]
    errores_por_producto = errores_por_metodo.groupby(productos).mean()

    metodos = errores_por_producto.columns.to_numpy()
    orden = np.argsort(errores_por_producto.to_numpy(), axis=1)
    mejor_metodo = pd.Series(metodos[orden[:, 0]], index=errores_por_producto.index)
    segundo_metodo = pd.Series(metodos[orden[:, 1]], index=errores_por_producto.index)
```

Se calcula la diferencia de error entre el mejor y el segundo mejor modelo para cada uno de los productos mes a mes, y se resume esa diferencia con su media, desviación estándar y cantidad de observaciones:

Código 27. Diferencia de error entre el mejor y el segundo mejor modelo.

```
mejor_por_fila = productos.map(mejor_metodo)
segundo_por_fila = productos.map(segundo_metodo)
error_mejor = np.select(
    [mejor_por_fila == metodo for metodo in errores_por_metodo.columns],
    [errores_por_metodo[metodo] for metodo in errores_por_metodo.columns],
)
error_segundo = np.select(
    [segundo_por_fila == metodo for metodo in errores_por_metodo.columns],
    [errores_por_metodo[metodo] for metodo in errores_por_metodo.columns],
)
diferencia = pd.Series(error_mejor - error_segundo, index=resultados_prediccion.index)
stats_diferencia = diferencia.groupby(productos).agg(d_media="mean", d_std="std", n_obs="count")
```

Se implementó la prueba de Diebold-Mariano [14]. El problema que resuelve es que cuando se comparan dos modelos para un producto, se quiere saber si el modelo ganador realmente es mejor o si por algún motivo tuvo suerte en esos pocos 3 meses, ya que un solo mes con un comportamiento atípico puede hacer que un modelo gane sin que eso signifique que sea mejor.

La prueba Diebold-Mariano permite determinar estadísticamente si la diferencia de error entre el mejor y el segundo mejor modelo es estadísticamente significativa, o si es solo ruido. La prueba consiste en observar mes a mes cuanto le gana un modelo al otro. Si esa diferencia es consistente (un modelo le gana al otro casi todos los meses, por una cantidad parecida), es una señal fuerte de que la ventaja es real. Si la diferencia varía mucho de un mes a otro (a veces gana uno, a veces el otro, por cantidades muy distintas), es señal de que lo que se vio como "ganador" fue más bien casualidad.

Debido a que la prueba clásica está pensada para muestras grandes, se adaptó para muestras pequeñas siguiendo la corrección de Harvey, Leybourne y Newbold, ampliamente discutida y recomendada por Diebold [15], la cual consistió en ajustar ($dm_{ajustado} = dm * \sqrt{(n-1)/n}$) y comparar el resultado contra una distribución t de Student con $n-1$ grados de libertad, en vez de la normal estándar de la prueba original. Cuantos más grados de libertad tiene una prueba estadística, más confiable es su capacidad de detectar diferencias reales, especialmente cuando se cuenta con pocos datos.

La implementación de la prueba consiste, inicialmente, en medir que tan confiable es el promedio de las diferencias, dado que se calculó con pocos datos. Cuanto más dispersas son las diferencias (d_std) o menos meses tiene (n), más "ruidoso" es ese promedio.

Para comparar que tan grande es la ventaja promedio frente a que tan confiable es: si la ventaja promedio es grande pero el error estándar (la inconsistencia) también es grande, dm resulta pequeño, lo cual es señal de que la ventaja no es confiable.

Código 28. Prueba de Diebold-Mariano con la corrección de Harvey, Leybourne y Newbold.

```
error_estandar = stats_diferencia["d_std"] / np.sqrt(stats_dif
dm = stats_diferencia["d_media"] / error_estandar
correccion_hln = np.sqrt((stats_diferencia["n"] - 1) / stats_d
dm_ajustado = dm * correccion_hln
grados_libertad = (stats_diferencia["n"] - 1).clip(lower=1)
p_valor = pd.Series(
    2 * (1 - stats.t.cdf(dm_ajustado.abs(), df=grados_libertad
    index=stats_diferencia.index,
)

suficientes_datos = stats_diferencia["n"] >= 2
sin_varianza = stats_diferencia["d_std"].fillna(0) == 0
significativo = suficientes_datos & (p_valor < NIVEL_SIGNIFICANCIA_D

campeon = mejor_metodo.copy()
```

```
campeon[~significativo] = "Media móvil"  
return campeon
```

Se descarta el resultado del MAE y se usa el modelo de Media Movil como respaldo si:

- La diferencia no resulta estadísticamente significativa, es decir, que el p-valor de la prueba Diebold-Mariano es mayor al nivel de significancia definido (5%).
- Los dos métodos se equivocaron exactamente igual (varianza cero, sin evidencia de que uno sea mejor)
- No hay al menos 2 meses para medir varianza

En estos casos se selecciona la Media Movil en vez del modelo que gano por una diferencia que no es significativa y que podria tratarse solo de ruido.

Al aplicar este mecanismo al sistema, se evidencio un cambio drastico en el resultado real, probado sobre el catalogo completo de la Clinica Nuestra Señora de los Remedios (856 productos). Antes de implementar la validacion con Diebold-Mariano, seleccionando unicamente por el MAE mas bajo, el metodo campeon se distribuia asi:

- XGBoost ganaba en 430 productos (50.2%)
- Media Movil en 227 (26.5%)
- Regresion Lineal en 199 (23.2%)

Al incorporar la prueba de Diebold-Mariano, esa distribucion cambio radicalmente:

- XGBoost paso a ganar en solo 15 productos (1.8%)
- Regresion Lineal en 12 (1.4%)
- Media Movil paso a ser el metodo campeon en 829 productos (96.8%)

De los 430 productos en los que originalmente ganaba XGBoost, 415 pasaron a usar Media Movil y solo 15 se mantuvieron con XGBoost; de los 199 en los que ganaba Regresion Lineal, 187 pasaron a Media Movil y solo 12 se mantuvieron. Los 227 productos que ya usaban Media Movil se mantuvieron igual, ya que este es el metodo de respaldo cuando la diferencia no resulta significativa. Esto evidencia que, sin la validacion estadistica de Diebold-Mariano, el sistema estaba seleccionando modelos "ganadores" cuya ventaja, en la gran mayoria de los casos, no era estadísticamente distinguible del ruido.

En cuanto al tamaño del periodo de prueba, cuando se cuenta con un test fijo de 3 meses, la prueba de Diebold-Mariano siempre tendría solo 2 grados de libertad, sin importar cuanta historia tenga el archivo, nunca ganando más poder estadístico para detectar diferencias reales. Por eso, el tamaño del periodo de prueba se ajusta según el historial disponible:

Código 29. Función `tamano_test_backtest`, que ajusta el periodo de prueba al historial disponible.

```
def tamano_test_backtest(n_meses_usables):
    n_meses_archivo_original = n_meses_usables + MESES_PERDIDOS_POR_RE
    if n_meses_archivo_original >= MESES_ARCHIVO_UMBRAL_TEST_GRANDE:
        return TEST_SIZE_MESES_GRANDE
    return TEST_SIZE_MESES_DEFAULT
```

En caso de que se cuente con un archivo csv grande (es decir, un archivo con 18 meses o más), se usa un test de 6 meses en vez de 3, dándole más grados de libertad y más poder estadístico a la prueba de Diebold-Mariano.

El corte de 18 meses es práctica definida para este proyecto, se escogió por ser un valor redondo (año y medio). No es un valor tomado de una fuente externa, a diferencia de las demás prácticas aplicadas como Diebold-Mariano o nivel de significancia.

3.2.14 Pronóstico futuro

El cálculo del pronóstico futuro es la funcionalidad principal del sistema, es decir, predecir la demanda de los productos en los meses posteriores al último mes del archivo CSV cargado por el usuario.

Después de que el usuario sube el archivo CSV con el historial de consumo, el sistema le permite configurar tanto el presupuesto disponible como el número de meses que desea pronosticar. Se implementó en `app/services/prediccion/pronostico_futuro.py`:

```
def generar_pronostico_futuro(
    df_mensual, producto_comportamiento, model, feature_cols, modelo_l
    horizonte_meses=HORIZONTE_MESES_DEFAULT,
):
    proximo_mes = df_mensual["fecha"].max() + pd.DateOffset(months=1)
    meses_pronosticados = [
        (proximo_mes + pd.DateOffset(months=h)).strftime("%Y-%m")
        for h in range(horizonte_meses)
    ]
    meses_pronosticados_legibles = [
```

```

f"{NOMBRES_MES_ES[(proximo_mes + pd.DateOffset(months=h)).month]
f"{(proximo_mes + pd.DateOffset(months=h)).year}"
for h in range(horizonte_meses)
]

```

En donde, se toma el último mes con datos del archivo, se le suma un mes, y desde ahí se generan los `horizonte_meses` meses consecutivos que se van a pronosticar.

Ciclo recursivo: Para poder calcular el pronóstico futuro, se necesita del mes anterior para predecir el mes futuro, pero cuando se quiere predecir varios meses hacia adelante, el mes anterior a ese todavía no existe. Para resolverlo, Thary utiliza una estrategia recursiva [16], es decir, para predecir un mes se usa la predicción del mes anterior como si fuera un mes real. Es recursiva porque el resultado de un mes se vuelve la entrada para predecir el mes siguiente. Es una característica propia de Thary respecto a los ejemplos de referencia *Ejemplo 1 (SKU Demand Forecasting)* [10] y *Ejemplo 3 (XGBoost)* [12], ya que estos solo pueden pronosticar el mes siguiente al último dato. La desventaja es que, al usar los meses pronosticados como entrada, es probable que el error vaya acumulándose y creciendo a medida que se pronostican meses más lejanos.

```

for h in range(horizonte_meses):
    mes_objetivo = proximo_mes + pd.DateOffset(months=h)

    filas_features = []
    metadatos = []

    for producto in productos_validos:
        estado = estado_por_producto[producto]
        demandas = estado["demandas"]
        ultima_filas = estado["ultima_filas"]

        lag_1 = demandas[-1]
        lag_2 = demandas[-2]
        lag_3 = demandas[-3]
        lag_6 = demandas[-6] if len(demandas) >= 6 else 0

        media_movil_3 = np.mean(demandas[-3:])
        media_movil_6 = np.mean(demandas[-6:]) if len(demandas) >= 6 else 0
        desviacion_movil_3 = np.std(demandas[-3:])

```

```

fila_features = {
    "año": mes_objetivo.year,
    "mes_sin": np.sin(2 * np.pi * mes_objetivo.month / 12),
    "mes_cos": np.cos(2 * np.pi * mes_objetivo.month / 12),
    "lag_1": lag_1,
    "lag_2": lag_2,
    "lag_3": lag_3,
    "lag_6": lag_6,
    "media_movil_3": media_movil_3,
    "media_movil_6": media_movil_6,
    "desviacion_movil_3": desviacion_movil_3,
    "media_producto": ultima_fila.get("media_producto", 0),
    "desviacion_producto": ultima_fila.get("desviacion_producto", 0)
}

```

En esta parte se preparan los datos de entrada que el modelo necesita para cada uno de los meses que se quiere pronosticar. La estructura cuenta con dos ciclos: el primero recorre los meses a pronosticar, por lo que, de querer predecir los próximos 3 meses con un archivo de 856 productos, el ciclo se ejecutará 3 veces, y el segundo, en cada una de las ejecuciones del primero, recorre los productos que se quieren pronosticar (los que tienen al menos 3 meses de historial), en este caso 856.

- Los lags `lag_1 = demandas[-1]`, `lag_2 = demandas[-2]`, `lag_3 = demandas[-3]` y `lag_6 = demandas[-6]` contienen las unidades vendidas hace 1, 2, 3 y 6 meses, respectivamente (si hay menos de 6 meses, `lag_6` es 0).
- `media_movil_3 = np.mean(demandas[-3:])`: Consiste en el promedio de los últimos 3 meses.
- `media_movil_6 = np.mean(demandas[-6:])`: Promedio de los últimos 6 meses (si hay menos de 6, el de los meses disponibles).
- `desviacion_movil_3 = np.std(demandas[-3:])`: Consiste en la desviación estándar de los últimos 3 meses.
- `año`, `mes_sin`, `mes_cos`: Estas variables se calculan del mes que se está pronosticando.
- `media_producto` y `desviacion_producto`: Consisten en el promedio y la desviación estándar del producto. Se toman de la última fila del producto y no cambian entre vueltas; si esa columna no existe, se usa 0.

Estos datos se guardan en el diccionario `fila_features`, que contiene las variables de entrada con las que el modelo fue entrenado.

En el segundo ciclo, cada producto aporta una fila con sus variables de entrada. Al terminar de recorrerlos todos, se tienen las filas de todos los productos acumuladas en `filas_features`, listas para que el modelo las use. En esta parte todavía no se predice nada.

El proceso es recursivo porque la lista de demanda de cada producto no es fija, se va actualizando al terminar cada mes, ya que la predicción obtenida se agrega al final de la lista, de manera que en el próximo mes se utilicen esos datos para la predicción. Así, en el primer mes pronosticado el `lag_1` es la demanda real del último mes del archivo, mientras que en el segundo mes pronosticado es la predicción del primero.

Se arma una tabla con las variables de todos los productos y se llama a `predict()` una sola vez por modelo, para predecir los productos de todo el lote. De esta manera, se pueden predecir todos los productos de un mes en una sola llamada al modelo, lo que mejora el rendimiento del sistema, ya que se obtienen los mismos resultados que si se llamara al modelo producto por producto y es mucho más rápido. Para cada uno de los productos se obtienen las tres predicciones de los 3 modelos.

```
X_futuro_lote = pd.DataFrame(filas_features)[feature_cols]
preds_xgb_lote = np.maximum(np.expml(model.predict(X_futuro_lote))
preds_lr_lote = np.maximum(modelo_lr.predict(X_futuro_lote[feature

for (producto, media_movil_3), pred_xgb_futuro, pred_lr_futuro in
    estado = estado_por_producto[producto]
    segmento = estado["segmento"]

    pred_xgb_futuro = float(pred_xgb_futuro)
    pred_lr_futuro = float(pred_lr_futuro)
    pred_baseline_futuro = float(max(media_movil_3, 0))
```

Una vez que se tienen las 3 predicciones de los modelos, se tiene que seleccionar cuál de los 3 se va a usar para cada producto, guardar dicha selección y tenerla disponible para el siguiente mes.

La selección del modelo ya se tiene lista desde la etapa anterior de "selección de modelo". Siempre se usa el resultado del mejor modelo, a no ser que ese resultado sea incoherente, para esos casos, se implementó la salvaguarda, en la que se usa la media móvil en su lugar.

Las condiciones para considerar un resultado incoherente son:

- El modelo tiene que ser XGBoost o Regresión Lineal
- Media móvil tiene que ser mayor de 0, es decir, el producto tiene que haber vendido algo en los últimos 3 meses

- La predicción del modelo ganador es menor que la mitad de la media móvil de los últimos 3 meses, es decir, el modelo ganador predice que se va a vender menos de la mitad de lo que se ha vendido en los últimos 3 meses.

Se tiene el diccionario `predicciones_por_metodo` que cuenta con las 3 predicciones, de aquí es donde se obtiene el método elegido y posteriormente, con `pronostico_futuro.append({...})` se guarda una fila con:

- Identificación del producto: `producto_id`, `nombre_producto` y `categoria`
- Mes pronosticado: `mes_pronosticado`
- Las 3 predicciones de los 3 modelos: `prediccion_xgboost`, `prediccion_regresion_lineal` y `prediccion_baseline`
- Predicción final: La que se le mostrara al usuario final (`prediccion_final`)
- Método usado: `metodo_usado`, este puede cambiar si al final se recurrió a usar la salvaguarda.

Por último, se agrega la predicción hecha al final de la lista de demandas del producto, por lo que en el mes siguiente, `demandas[-1]` (el `lag_1`) ya no representara la demanda real, sino la predicción recién hecha.

```
metodo_campeon = segmento["metodo_campeon"]
if (
    metodo_campeon != "Media móvil"
    and media_movil_3 > 0
    and predicciones_por_metodo[metodo_campeon] < media_movil_3 *
):
    metodo_usado = "Media móvil"
else:
    metodo_usado = metodo_campeon

pred_final_futuro = predicciones_por_metodo[metodo_usado]

pronostico_futuro.append({
    "producto_id": producto,
    "nombre_producto": segmento.get("nombre_producto"),
    "categoria": segmento.get("categoria"),
```



```

    "mes_pronosticado": mes_objetivo.strftime("%Y-%m"),
    "horizonte": h + 1,
    "prediccion_xgboost": round(pred_xgb_futuro, 2),
    "prediccion_regresion_lineal": round(pred_lr_futuro, 2),
    "prediccion_baseline": round(pred_baseline_futuro, 2),
    "prediccion_final": round(pred_final_futuro, 2),
    "metodo_usado": metodo_usado,
})

estado["demandas"].append(pred_final_futuro)

```

Para mostrarle los resultados al usuario, se arma una tabla en la que se resume la predicción de la demanda de los productos, mostrando su id y nombre, la predicción final de cada mes pronosticado (una columna por mes, en unidades enteras) y el total. Esta tabla se puede descargar en formato CSV para su posterior análisis.

3.2.15 Cálculo de reposición (EOQ/ROP)

El calculo de la reposicion responde a la necesidad de determinar cuanto inventario de seguridad es conveniente pedir para cada producto en un nuevo pedido, junto a otra informacion relevante como en que nivel de inventario es mas conveniente hacer el pedido, que cantidad se debe pedir y en que fecha debe hacerse el pedido.

Para esta parte, se utilizaron las formulas de gestion de inventraio clasicas para stock de seguridad, punto de reorden y cantidad económica de pedido (EOQ), los cuales tambien se usaron en los ejemplos de referencia *Ejemplo 1 (SKU Demand Forecasting)* [10] y *Ejemplo 2 (Inventory Management with LSTM)* **ejemplo2_lstm_inventory**. En este aspecto, la propuesta de valor de Thary es que estas formulas se hacen en base al pronostico futuro que fue generado en la etapa anterior, y no en base a los datos historicos.

A partir de las predicciones finales de los meses pronosticados se calculan el promedio mensual $\bar{d}$ y su desviación estándar σ_d .

```

def calcular_tabla_reorden(
    df_model, df_mensual, producto_comportamiento, pronostico_futuro_df,
    presupuesto_capital_trabajo=None
):
    #...
    stats_pronostico = pronostico_futuro_df.groupby("producto_id")["pr
        demanda_promedio="mean", desviacion_demanda=lambda x: np.std(x
    )

```

Lead time: El lead time de cada producto representa el tiempo en el que los proveedores entregarían el producto después de hacer un pedido, en el sistema se expresa en meses para que sea consistente con la unidad de tiempo de las demandas. Para asegurar esto, si el archivo original contiene el lead time en semanas, el sistema convierte los semanas de lead time a meses, dividiéndolas entre 4.345. De no tener el lead time, se asume un valor por defecto de 0.5 meses.

```
lead_time_meses = (
    fila_params["lead_time_semanas"] / 4.345
    if "lead_time_semanas" in df_mensual.columns and not pd.isna(fila_p
    else LEAD_TIME_DEFAULT_MESES
)
```

Stock de seguridad y punto de reorden: El stock de seguridad es esa cantidad de productos extra que se guarda en el inventario para cubrir la variabilidad de la demanda mientras llega un pedido.

$$SS = Z \cdot \sigma_d \cdot \sqrt{L}$$

```
stock_seguridad = SERVICE_LEVEL_Z * desviacion_demanda * np.sqrt(lead_time_meses)
```

El SERVICE_LEVEL_Z corresponde a un nivel de servicio de 95% ($Z=1.65$), que es el nivel de servicio recomendado por la literatura de gestión de inventarios **chopra2020supply**. Este porcentaje hace referencia a que se espera que el stock de seguridad sea suficiente para cubrir la demanda durante el lead time en 95% de los casos.

El punto de reorden es el nivel de inventario en el que, cuando se llega a este, se debe de hacer un nuevo pedido.

$$ROP = \bar{d} \cdot L + SS$$

```
punto_reorden = demanda_promedio * lead_time_meses + stock_seguridad
```

En el que:

- $\bar{d}$ es la demanda promedio.
- L es el lead time.
- SS es el stock de seguridad.

Cantidad económica de pedido (EOQ): E

3.2.16 Modelo de Datos

Conceptualmente, el modelo de datos del sistema Thary utiliza un modelo relacional, en el que la información se organiza en 12 tablas relacionadas entre sí, definidas mediante claves foráneas: empresas, usuarios, pedidos, actividad (el historial de movimientos), y corridas. Esta última es la tabla principal que representa cada vez que un usuario (administrador) sube un CSV, junto con 7 tablas que vienen de cada una de las corridas: segmentacion, metricas_modelos, predicciones_backtest, pronostico_futuro, reorder, validacion_cruzada_folds, validacion_cruzada_resumen. Cada archivo que se sube crea una fila nueva en corridas, aunque no se borra el historial, la app solo muestra la corrida más reciente.

Lógicamente y físicamente, el modelo hace uso de MySQL (motor de base de datos relacional), el sistema se conecta a este usando SQLAlchemy. En la tabla empresas, se almacena el id y el nombre de cada organización que es registrada en el sistema, por otra parte, en la tabla usuarios se guarda la información del usuario, como su nombre, correo electrónico, contraseña (cifrada), el rol, ya sea admin o empleado, y el id de la empresa a la que pertenece. La tabla pedidos también se almacena de forma independiente separada de los resultados del procesamiento, y es única para cada empresa, así se garantiza que cuando se cargue un nuevo historial, no se elimine el registro de los pedidos ya confirmados. Por otro lado, las tablas de segmentación, recomendación de reposición y comportamiento del producto son generadas después de que el sistema procese el archivo CSV que fue cargado por el administrador. Estos resultados se guardan y quedan asociadas a la corrida que las origina mediante `corrida_id`.

No toda la información se queda en la base de datos, ya que los archivos CSV que el usuario sube, las gráficas generadas y el modelo XGBoost entrenado se siguen almacenando como archivos en disco, los cuales están organizados en carpetas por empresa. Solo se traslada a MySQL la información que pueda estructurarse en filas y columnas.

TABLA V
Entidades principales del modelo de datos.

Entidad	Descripción	Cardinalidad
Empresa	Organización que utiliza el sistema, identificada de forma única.	1 : N
Usuario	Persona registrada en el sistema, con un rol, asociada a una empresa mediante clave foránea.	1 : N
Pedido	Proveedor, cantidad, costo y usuario responsable de un pedido confirmado, asociado a una empresa.	1 : N
Corrida	Registro de un procesamiento de archivo, con sus parámetros y métricas globales.	1 : N
Segmentación	Clasificación ABC, variabilidad y método campeón por producto, para una corrida dada.	N : 1
Métricas de modelos	MAE, RMSE, MAPE y WAPE de cada modelo candidato, para una corrida dada.	N : 1
Predicciones de backtest	Demanda real y predicha por producto-mes, para el periodo de prueba de una corrida.	N : 1
Pronóstico futuro	Demanda estimada para los próximos meses, por producto, para una corrida dada.	N : 1
Reorden (reposición)	Punto de reorden, EOQ y cantidad sugerida por producto, para una corrida dada.	N : 1
Validación cruzada (folds)	Métricas de error por modelo, calculadas en cada fold de la validación cruzada temporal.	N : 1
Validación cruzada (resumen)	Promedio y desviación de las métricas de cada modelo, a través de los folds.	N : 1
Actividad	Bitácora de acciones realizadas por los usuarios, asociada a una empresa.	1 : N

La Tabla V resume las entidades del modelo con el detalle de cada campo y su cardinalidad.

La clave foránea (FOREIGN KEY) de usuarios hacia empresas rechaza cualquier intento de crear un usuario sin una empresa asociada, garantizando el aislamiento de la información entre organizaciones. La relación entre corridas y segmentación es de dependencia en la que no tiene ningún sentido tener una fila en segmentación si no sabemos a qué corrida pertenece, ON DELETE CASCADE implica que, si se elimina una corrida, todas sus filas de segmentación (junto con las métricas, predicciones, reposición y validación cruzada) se eliminan junto con ella:

Código 30. Definición de las tablas empresas, usuarios, corridas y segmentacion con sus claves foráneas.

```
CREATE TABLE empresas (  
    id VARCHAR(32) PRIMARY KEY  
    #....  
)  
  
CREATE TABLE usuarios (  
    id INT PRIMARY KEY,  
    empresa_id VARCHAR(32) NOT NULL,  
    FOREIGN KEY (empresa_id) REFERENCES empresas(id)  
)  
  
CREATE TABLE corridas (  
    id INT PRIMARY KEY,  
    empresa_id VARCHAR(32) NOT NULL,  
    FOREIGN KEY (empresa_id) REFERENCES empresas(id)  
)  
  
CREATE TABLE segmentacion (  
    id BIGINT PRIMARY KEY,  
    corrida_id INT NOT NULL,  
    FOREIGN KEY (corrida_id) REFERENCES corridas(id) ON DELETE CASCADE  
)
```

En el siguiente fragmento, extraído de `app/services/db.py`, se observa como Thary establece la conexión hacia el motor de MySQL:

Código 31. Configuración de la conexión a MySQL (app/services/db.py).

```
_engine = None

def _config():
    return {
        "host": os.environ.get("DB_HOST", "127.0.0.1"),
        "port": os.environ.get("DB_PORT", "3307"),
        "user": os.environ.get("DB_USER", "thary_app"),
        "password": os.environ.get("DB_PASSWORD", ""),
        "database": os.environ.get("DB_NAME", "sistema_inventario"),
    }

def get_engine():
    global _engine
    if _engine is None:
        cfg = _config()
        url = (
            f"mysql+pymysql://{cfg['user']}:{cfg['password']}@"
            f"{cfg['host']}:{cfg['port']}/{cfg['database']}?charset=u
        )
        _engine = create_engine(url, pool_pre_ping=True)
    return _engine
```

Como se puede ver, la configuración de(`_config()`) no se obtiene de valores fijos, si no de las variables de entorno. Mientras que el motor (`engine`) solo se crea una vez durante toda la ejecución de la aplicación en vez de abrir una conexión nueva con cada solicitud, esto ayuda a que se evite el costo de tener que establecer una conexión nueva a la base de datos por cada petición HTTP que sea recibida por Thary.

Una vez realizada esta conexión, se utilizó MySQL Workbench para verificar que la base de datos y las tablas fueron creadas correctamente:

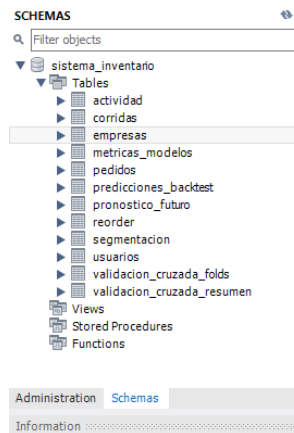


Fig. 8. Estructura de la tabla empresas vista desde MySQL Workbench, conectado a la base de datos local.

La tabla empresas existe en la base de datos con las columnas descritas anteriormente: un identificador de tipo `varchar (32)` como clave primaria, el nombre de la empresa, y la fecha de creacion del registro.

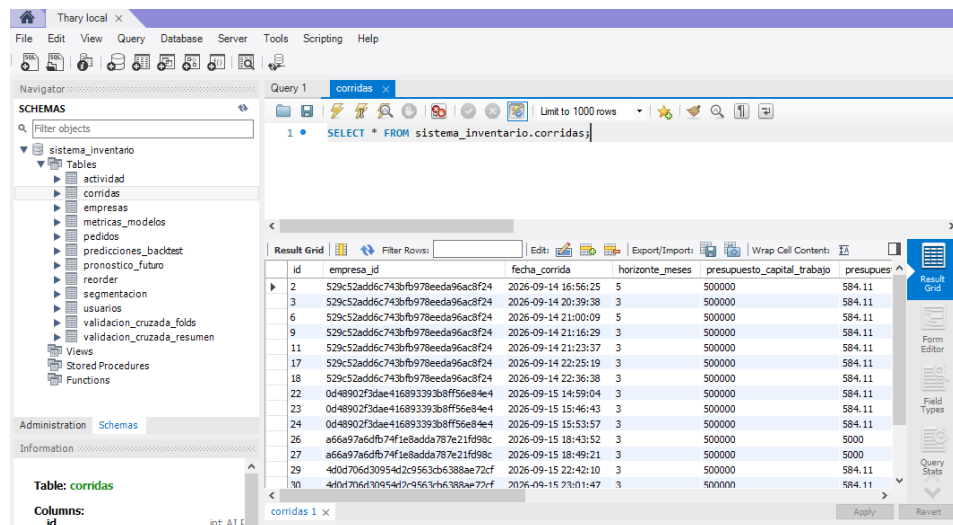


Fig. 9. Consulta sobre la tabla corridas en MySQL Workbench, mostrando el esquema completo de 12 tablas y el historial acumulado de corridas.

La Fig. 9 muestra la confirmacion de que cada procesamiento de archivo genera una fila nueva en corridas sin eliminar las anteriores. Existen multiples filas asociadas a una misma empresa,

correspondientes a distintas corridas realizadas en distintas fechas y horas.

3.2.17 Diseño UX/UI

El nombre del sistema "Thary" fue elegido en la etapa de diseño de identidad visual, es un nombre corto escogido por foneticamente sonar similar a las ultimas cuatro letras de la palabra "Inventory", (inventario en ingles). En cuanto a icono representativo, se eligio la figura de un buho, para transmitir la idea de orden y control, ya que los buhos son animales reconocidos por su capacidad de percepcion y vigilancia, relacionado a como Thary se encarga de detectar señales de riesgo que podrian afectar el inventario de una empresa.



Fig. 10. Logotipo de Thary, compuesto por el nombre del sistema y el icono representativo del búho

Para realizar el diseño de interaccion de Thary, se decidio separar las pantallas en dos: Una dirigida exclusivamente a los administradores y otra dirigida a los empleados del inventario. Al ingresar al sistema, un usuario con el rol de administrador sigue el siguiente flujo:

El usuario inicia sesion o se registra de no tener una cuenta.

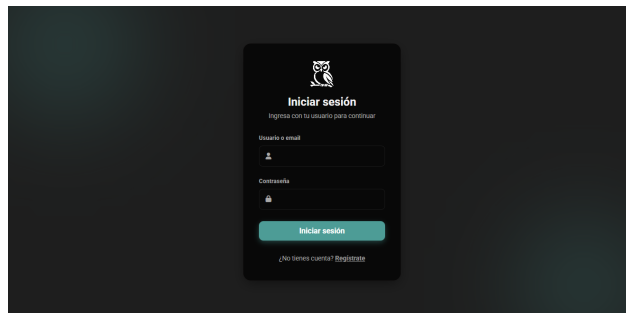


Fig. 11. Pantalla de inicio de sesion de Thary

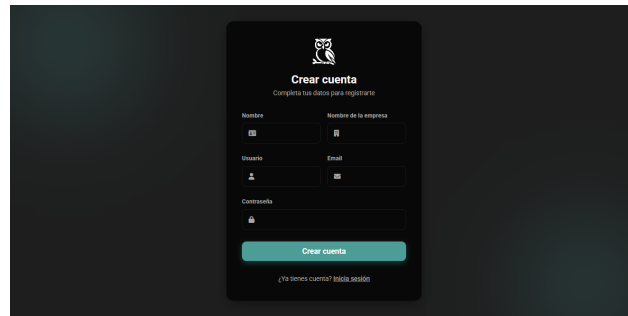


Fig. 12. Pantalla de registro de Thary

El administrador inicia en la pantalla de inicio del sistema, desde la cual puede acceder a la funcionalidad de carga de archivo (*index*), al haber cargado el archivo correctamente y que este sea válido, se le redirige automáticamente al dashboard, donde al seleccionar "Pronostico de demanda", se muestran los resultados.

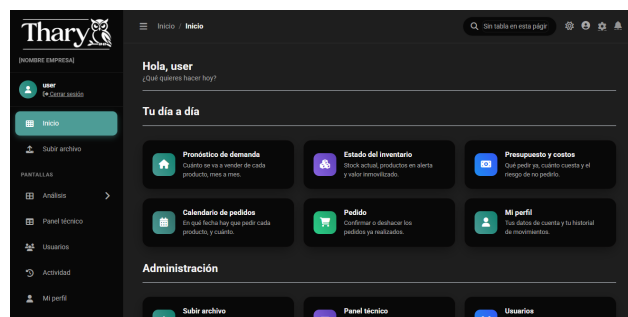


Fig. 13. Pantalla de dashboard de Thary

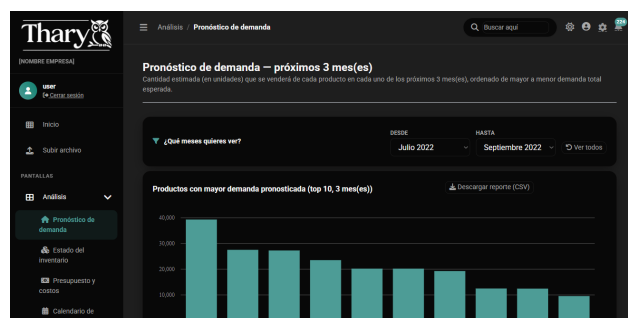


Fig. 14. Pantalla de predicción de demanda de Thary

En el sidebar, además de encontrar demás opciones como estado del inventario, presupuesto y pedidos, el administrador puede navegar hacia la gestión de usuarios, donde puede crear cuentas nuevas para su personal de inventario.

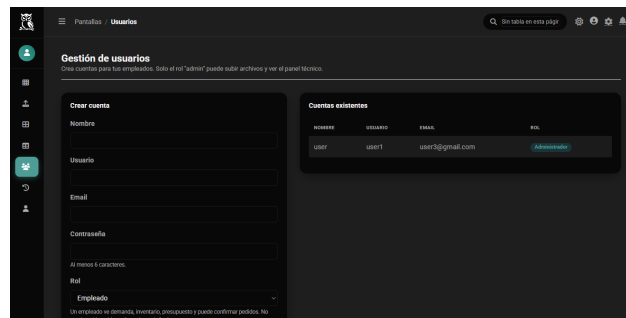
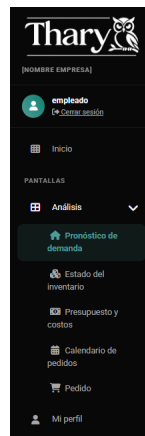
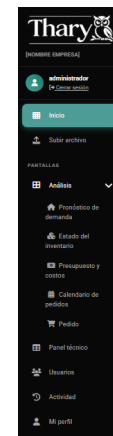


Fig. 15. Pantalla de gestión de usuarios de Thary

El flujo de los empleados es bastante similar, con la diferencia de que el perfil de un empleado cuenta con menos privilegios que el perfil de un administrador, por lo que al iniciar sesión, el empleado no puede subir ningún archivo, por lo que solo puede acceder al dashboard de la demanda pronosticada en base al csv que ya anteriormente fue cargado al sistema por el administrador que creó su cuenta, ya desde ahí, puede navegar a las mismas funcionalidades del análisis de los datos que el administrador: estado del inventario, presupuesto y pedidos.



(a) Sidebar de empleado



(b) Sidebar de administrador

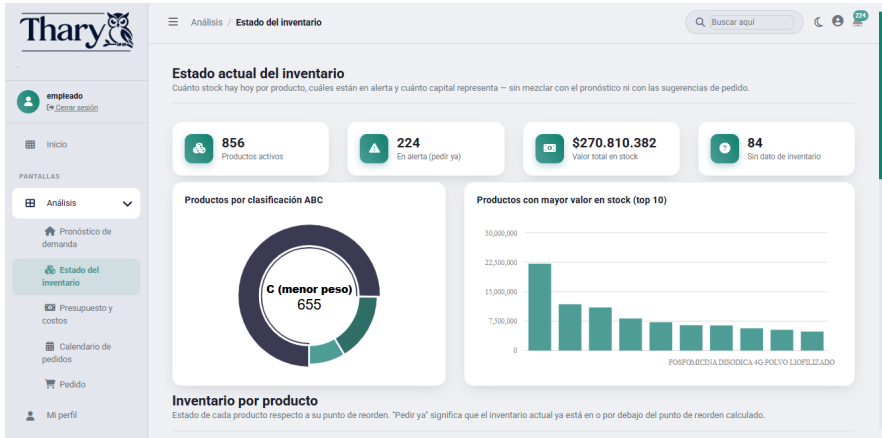


Fig. 17. Estado del inventario



(a) Presupuesto

(b) Alertas de reposición

La gestion de pedidos se puede ver en tres vistas: los pedidos sugeridos pendientes de confirmar, el historial de pedidos ya confirmados, y los proximos pedidos estimados segun el calendario de reposicion.

empleado

Inicio

Partallas

Análisis

Predicción de demanda

Estado del inventario

Presupuesto y costos

Calendario de pedidos

Pedido

Mi perfil

Analisis / Pedido

Confirmar pedidos

Elige con qué proveedor pedir cada producto en alerta y confirma cuáles ya se compraron – queda registrado quién lo hizo y cuándo.

Pendientes de confirmar

224 producto(s) en alerta todavía sin pedido confirmado.

Mostrar 10 productos

PRODUCTO	CATEGORÍA	CANTIDAD A PEDIR	COSTO ESTIM.
700800 HALOPERIDOL 5MG TABLETA	MEDICAMENTOS	11.0	\$583
705488 PROPRANOLOL TABLETA 40MG	MEDICAMENTOS	11.0	\$583
706670 ACICLOVIR TABLETA 800MG	MEDICAMENTOS	4.0	\$580
715545 LORAZEPAM TABLETA 2MG	CONTROLADOS	17.0	\$578
D006701	MEDICAMENTOS	9.0	\$578

Mostrando 1 a 10 de 223 productos

Confirmar pedidos seleccionados

Pedidos ya confirmados

Historial de pedidos marcados como realizados, con proveedor, usuario y fecha.

Mostrar 10 productos

PRODUCTO	CANTIDAD	COSTO	PROVEEDOR ELEGIDO	CONFIRMADO POR	FECHA Y HORA
700800 HALOPERIDOL 5MG TABLETA	11.0	\$583	SOLINSA	empleado	2025-09-18 14:21:49

Mostrando 1 a 1 de 1 productos

(a) Lista de Pedidos

(b) Pedidos confirmados

Próximos pedidos (siguientes 90 días)

Fecha estimada en la que cada producto va a llegar a su punto de reorden, calculada a partir del inventario actual y el ritmo de consumo pronosticado – todavía no hay que pedirlos, pero conviene tenerlos en el radar.

Mostrar 10 productos

PRODUCTO	CATEGORÍA	FECHA ESTIMADA DE PEDIDO	DÍAS RESTANTES	INVENTARIO ACTUAL	PUNTO DE REORDEN
0002101 OMEPRAZOL CÁPSULA 20MG	MEDICAMENTOS	2022-06-02	1	481.0	413.93
0004721 OXACILINA POLVO LIOFILIZADO 1G	MEDICAMENTOS	2022-07-18	47	21.0	6.0
0006422 OXIMETAZOLINA SOLUCIÓN NASAL 0.05%	MEDICAMENTOS	2022-07-10	39	2.0	0.54
103652 PALIVIZUMAB POLVO LIOFILIZADO 50 MG	MEDICAMENTOS	2022-06-24	23	2.0	0.52
118494 DEXAMETASONA SOLUCIÓN OFTÁLMICA 5ML	MEDICAMENTOS	2022-07-29	59	3.0	0.54
120804 CARBOXIMETILCELULOSA 0.5 % SOL. OFTÁLMICA	MEDICAMENTOS	2022-08-10	70	1.0	0.27
131059					

Fig. 20. Próximos pedidos

Calendario de pedidos

Cuando hay que pedir cada producto, y cuánto, en un solo vistazo.

● Hoy hay 224 producto(s) que ya necesitan pedido – están marcados en el calendario.

Junio 2022

DOM	LUN	MAR	MIE	JUE	VIE	SÁB
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

Detalle del 01/06/2022

LACTULOSA SOLUCIÓN ORAL 66.7%/15ML	0 u.
FRESUBIN HP ENERGY SOLUCIÓN ORAL 1,5KCAL/1ML (1500KCAL/1000ML) ALIMENTO LÍQUIDO	0 u.
(FRESUBIN HP ENERGY SOLUCIÓN ORAL 1,5KCAL/1ML (750KCAL/500ML)) ALIMENTO LÍQUIDO	0 u.
HIDROXICLOROQUINA SULFATO TABLETA 200MG	0 u.
CISATRACURIO 10MG/5ML SOL. INYECTABLE	21.25 u.
INSULINA DEGLUDEC SOL. INYECTABLE 100U/ML x 3 ML	0 u.

Fig. 21. Calendario de pedidos

Una vez finalizadas sus tareas, los usuarios pueden cerrar sesión en su cuenta.

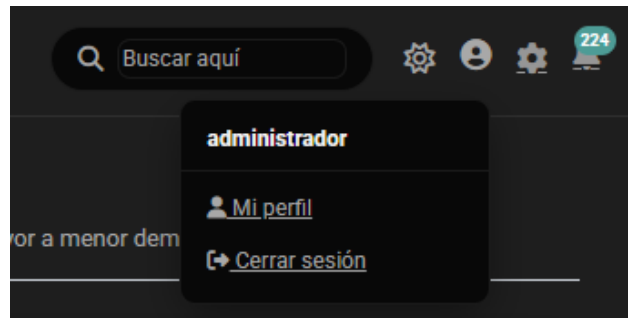


Fig. 22. Cerrar sesión

Los criterios de usabilidad que fueron tomados en cuenta fueron:

- Uso de alertas visuales codificadas por color para indicar fácilmente que productos se encuentran en o por debajo de su punto de reposición y requieren ser reabastecidos inmediatamente.

Inventario por producto
Estado de cada producto respecto a su punto de reorden. "Pide ya" significa que el inventario actual ya está en o por debajo del punto de reorden calculado.

Mostrar 10 productos Buscar:

PRODUCTO	CATEGORÍA	ABO	PROVEEDOR	INVENTARIO ACTUAL	PUNTO DE REORDEN	VALOR EN STOCK	ESTADO
0002101 OMEPRAZOL CÁPSULA 20MG	MEDICAMENTOS	A	—	481.0	413.93	\$17,797	OK
0004721 OXACOLINA POLVO LIOFILIZADO 10	MEDICAMENTOS	C	—	21.0	6.0	\$22,617	OK
0006422 OXIMETAZOLINA SOLUCIÓN NASAL 0.05%	MEDICAMENTOS	C	—	2.0	0.54	\$4,130	OK
100917 LACTULOSA SOLUCIÓN ORAL 66.7%/15ML	MEDICAMENTOS	B	—	43.0	46.15	\$45,537	Pide ya

Fig. 23. Alertas visuales

- Cuando un usuario con el rol de empleado inicie sesión, pero su respectivo administrador no subió ningún archivo csv, en vez de ver la pantalla del dashboard, verá una pantalla con el mensaje de que una vez el admin cargue el archivo, podrá ver los análisis.

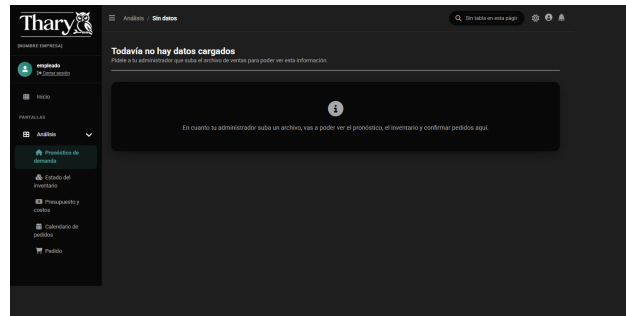


Fig. 24. Dashboard del empleado

- El sistema cuenta con un modo oscuro y modo claro para que el usuario pueda escoger según sus preferencias, mejorando la usabilidad ofreciendo ventajas como comodidad visual, menor fatiga en uso prolongado, contraste, etc.

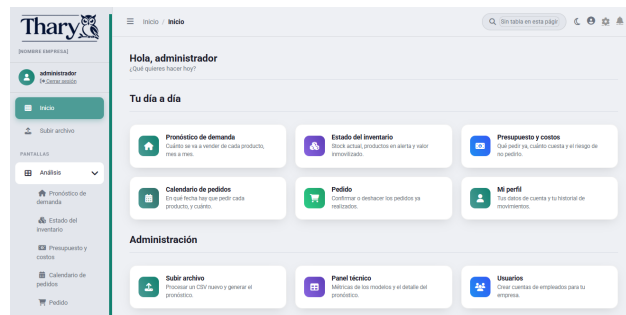


Fig. 25. Dashboard en modo claro

- Thary cuenta con una versión mobile, la cual le permite al usuario poder acceder a todas las funcionalidades de la versión de escritorio desde su teléfono móvil.

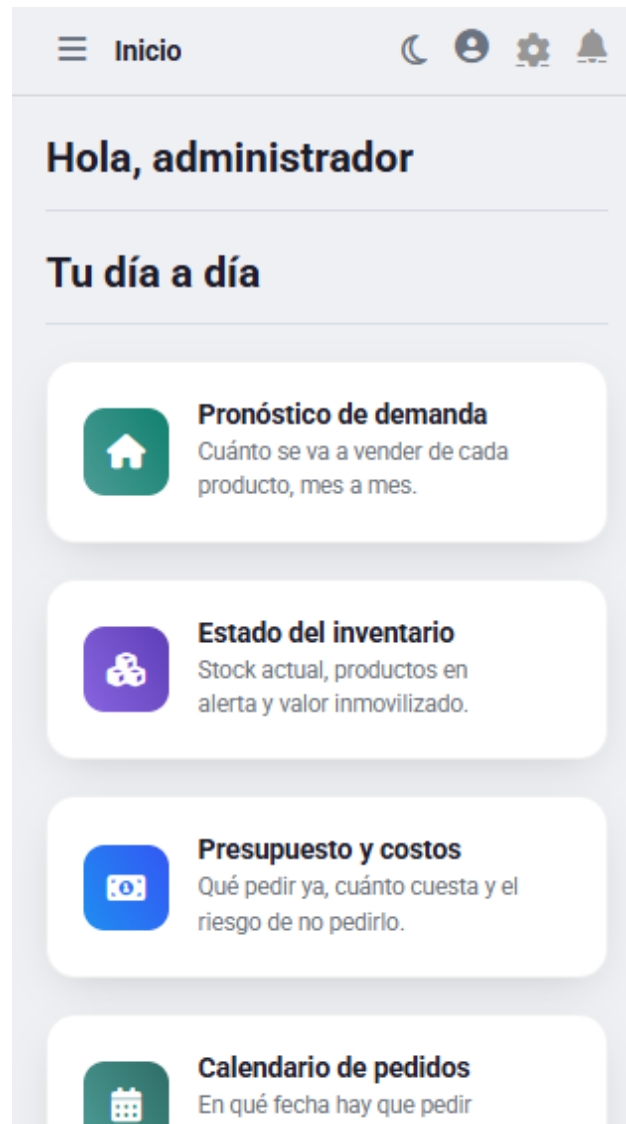


Fig. 26. Dashboard en modo mobile

En cuanto a criterios de accesibilidad, se tomaron en cuenta los criterios de usabilidad, pero no se tomaron en cuenta mecanismos adicionales para hacer mas accesible el sistema como algun tipo de lector de pantalla para personas con discapacidad visual, o implementar un sistema de navegacion completa por teclado para alguna persona que no pueda usar un mouse. Estas falencias se reconocen como una limitacion del diseño actual, y se espera mejorarlas en el futuro.

3.2.18 Análisis de Amenazas y Diseño Seguro

Debido a su naturaleza, Thary es un sistema que maneja informacion que podria considerarse potencialmente sensible, como los datos de los usuarios, de consumo, entre otros. Teniendo esto en cuenta, se realizo un analisis en el que se identificaron las amenazas siguiendo el modelado de

amenazas STRIDE:

- **Suplantación de identidad:** Mitigada mediante autenticación basada en usuario y contraseña, almacenando esta última mediante un algoritmo de hash (*scrypt*, el metodo por defecto de `werkzeug.security`) y nunca en texto plano.

Código 32. Función `crear_usuario`, que almacena la contraseña como hash.

```
def crear_usuario(nombre, username, email, password, rol="empl
#...
password_hash = generate_password_hash(password)
```

Código 33. Función `verificar_credenciales`, que compara la contraseña ingresada con su hash.

```
def verificar_credenciales(username_o_email, password):
#...
if not check_password_hash(usuario["password_hash"], password):
    return None
return usuario
```

- **Manipulación de datos:** Para evitar la manipulacion de los datos, se garantiza que la cantidad y el costo de un pedido confirmado se tome siempre del análisis generado por el sistema, y no de valores que el usuario pueda modificar en el formulario de confirmación.

Código 34. Función `pedido_confirmar`, que registra los pedidos con la cantidad y el costo calculados por el sistema.

```
def pedido_confirmar():
#...
for producto_id in seleccionados:
    fila = productos_por_id.get(producto_id)
    if fila is None:
        continue
    proveedor = request.form.get(f"proveedor_{producto_id}")
    registrar_pedido(
        empresa_id=session["empresa_id"],
        producto_id=producto_id,
        proveedor=proveedor,
        cantidad=fila.get("cantidad_sugerida_pedido"),
        costo=fila.get("costo_estimado_pedido"),
```



```

        usuario=usuario,
    )

```

- **Repudio:** El sistema Thary cuenta con una tabla en la base de datos llamada "Actividad", en la cual se lleva un registro de todas las acciones relevantes que los usuarios realizan en el sistema, guardando informacion como el nombre del usuario, fecha y hora del evento, y la accion que fue realizada.

Código 35. Registro de la actividad del usuario al confirmar un pedido (función `pedido_confirmar`).

```

def pedido_confirmar():
    if confirmados_ahora:
        lista = ", ".join(seleccionados[:5]) + ("..." if len(sel
        registrar_actividad(
            session["empresa_id"], session.get("username"), us
            "pedido_confirmado", f"Confirmó {confirmados_ahora} p
        )

```

- **Divulgación de información:** Se evita que exista divulgacion de informacion mediante el aislamiento de los datos por empresa, de forma que ninguna organización pueda acceder a los datos de otra.

Código 36. Función `carpeta_uploads`, que aísla los archivos cargados por cada empresa.

```

def carpeta_uploads(empresa_id):
    ruta = os.path.join(current_app.config["UPLOAD_FOLDER_BASE"], e
    os.makedirs(ruta, exist_ok=True)
    return ruta

def carpeta_resultados(empresa_id):
    ruta = os.path.join(current_app.config["RESULTADOS_FOLDER_BASE"]
    os.makedirs(ruta, exist_ok=True)
    return ruta

```

- **Denegacion de servicio:** Thary implementa varios controles para mitigar este riesgo. Se establecio un limite de tamaño para los archivos cargados (10 MB, mediante la configuracion `MAX_CONTENT_LENGTH` de Flask). Adicionalmente, el servidor WSGI de produccion (Gunicorn) se configuro con multiples procesos de trabajo (*workers*) y un tiempo maximo de espera ampliado a 120 segundos, de forma que una solicitud de procesamiento pesada (por ejemplo, el

entrenamiento de los modelos sobre un catalogo grande) ocupe unicamente uno de los procesos disponibles, sin bloquear el acceso al sistema para el resto de los usuarios. Finalmente, se incorporo un limite de tasa (*rate limiting*) mediante la libreria `Flask-Limiter` sobre las rutas de inicio de sesion y registro, restringiendo a 10 solicitudes por minuto por direccion IP, mitigando intentos de fuerza bruta y saturacion por solicitudes repetidas. Cabe señalar que, al usar Gunicorn con multiples *workers* y un almacenamiento de limite de tasa en memoria del propio proceso, este limite se aplica de forma independiente por cada worker y no de forma global para todo el servicio.

- **Elevación de privilegios:** Para evitar este riesgo, se implemento un control de acceso basado en roles mediante el decorador `admin_required` (definido en `app/services/auth.py`), aplicado sobre las rutas de carga y procesamiento de archivos, el panel tecnico de validacion, la gestion de usuarios y la consulta de la actividad de toda la empresa, restringiendo estas funciones exclusivamente al perfil administrador.

3.3 Construcción, Integración y Despliegue

Thary es un sistema que se desarrollo mediante el uso de las siguientes tecnologias: Python, Flask como framework, librerias como scikit-learn, Pandas, y XGBoost.

A la hora de ejecutar el sistema en Flask, se tiene que ejecutar el comando `run.py` en un entorno virtual (*venv*), todas las dependencias necesarias para la correcta ejecucion del proyecto se encuentran resumidas en el archivo `requirements.txt` para su facil instalacion.

Por su parte, el control de versiones del sistema se llevo a cabo mediante una seria de commits en GitHub en el que se documentaron los cambios mas significativos del sistema durante todo el tiempo de desarrollo.

3.3.1 Construcción

La principal practica de codificacion que se siguio durante el desarrollo e implementacion del sistema Thary es la separacion de responsabilidades, practica la cual se vio reflejada en la organizacion de archivos y carpetas del proyecto:

```
sistema_prediccion_inventario/  
  app/  
    routes/  
      auth.py  
      main.py  
    services/  
      auth.py  
      db.py  
      pedidos.py          # Registro de pedidos confirmados
```

```

actividad.py          # Acciones de usuario
prediccion/          # pipeline
    carga_datos.py    # 1. Carga y limpieza
    segmentacion.py   # 2. Segmentacion ABC/variabilidad
    features.py       # 3. Construcción de variables
    modelos.py        # 4. Entrenamiento y selección
    regresion_lineal.py# 4.1. Modelo de Regresión Lineal
    validacion.py     # 5. Validación cruzada temporal
    pronostico_futuro.py# 6. Pronóstico recursivo
    reposicion.py     # 7. Puntos de reorden y EOQ
    graficas.py       # 8. Generación de gráficos
    exportacion.py    # 9. Exportación de resultados
templates/
tests/                # pytest
    test_carga_datos.py
    test_modelos.py
    test_reposicion.py

```

Como se puede observar, la organización se divide en tres capas: `routes/` con el control del sistema, `services/` con la lógica de negocio y, dentro de esta última, `prediccion/`. Es en esa última capa, en la cual cada una de las 9 etapas del pipeline se encuentran alojadas en nueve módulos independientes: carga de datos, segmentación, construcción de variables, modelos, validación, pronóstico, reposición, gráficas y exportación. La principal ventaja de esta organización, es que permite que se pueda modificar una etapa específica del flujo sin afectar directamente a las demás partes, además de facilitar el mantenimiento y un entendimiento más fácil del código.

Como complemento a esta organización, se incorporaron pruebas automatizadas escritas con *pytest*, las cuales fueron agrupadas en tres archivos dentro de la carpeta `tests/`:

- `test_carga_datos.py`, en este archivo se verifican los mensajes de error ante archivos inválidos y el reconocimiento de columnas heterogéneas
- `test_modelos.py`, en el que se verifica la lógica de selección del modelo campeón por producto
- y por último, `test_reposicion.py`, en donde se verifican los cálculos de stock de seguridad, punto de reorden y EOQ

(Los resultados obtenidos al ejecutar esta suite de pruebas se presentan en el Capítulo 4.)

3.3.2 Integración

Las rutas de la aplicación se organizan en el archivo `app/__init__.py`, mediante el mecanismo de Blueprints de Flask, el cual permite que se pueda agrupar un conjunto de vistas en un módulo independiente y registrarlo en la aplicación principal. Thary cuenta con dos blueprints: `main_bp` (dashboard, inventario, presupuesto, pedidos y panel técnico) y `auth_bp` (inicio de sesión, registro y gestión de usuarios), que se registran de la siguiente manera:

Código 37. Registro de los blueprints `main_bp` y `auth_bp` en la aplicación Flask.

```
app.register_blueprint(main_bp)
app.register_blueprint(auth_bp)
```

Esta organización permite que cada blueprint se desarrolle y se pruebe de forma independiente, y que agregar un nuevo conjunto de rutas al sistema no implique tener que modificar el código de los blueprints que ya existen.

Por otra parte, para integrar correctamente el modelo de predicción con la aplicación web, en el archivo `services/sistema_prediccion.py` se incluyó una función de entrada, `ejecutar_sistema()`, la cual se encarga de orquestar secuencialmente las nueve etapas del pipeline y devolver un resultado. Esta decisión de integración se tomó tomando en cuenta la separación de capas ya establecida para todo el proyecto. En este caso, la capa de servicios de dominio se mantiene independiente de la de presentación.

Código 38. Función `ejecutar_sistema`, que orquesta las nueve etapas del pipeline.

```
def ejecutar_sistema(
    ruta_archivo, carpeta_resultados=None, presupuesto_capital_trabajo
):
    # 1. Carga, limpieza y agregación mensual
    df = cargar_y_limpiar(ruta_archivo)
    df_mensual = agregar_mensual(df)

    # 2. Segmentación de productos
    producto_comportamiento = calcular_comportamiento_producto(df, df_mensual)

    # 3. Feature engineering
    df_mensual, df_model, feature_cols = construir_features(df_mensual, df_model)

    # 4. Modelos
    modelo_lr, pred_lr, features_lr = entrenar_regresion_lineal(X_train, y_train)
```

```

model, pred_xgb = entrenar_xgboost(X_train, y_train_log, X_test)
pred_baseline_3 = prediccion_baseline_movil(test_df)

# 5. Validacion cruzada temporal
detalle_validacion_cruzada, resumen_validacion_cruzada = validacio

# 6. Pronostico recursivo de los proximos N meses
pronostico_futuro_df, ... = generar_pronostico_futuro(...)

# 7. Puntos de reorden y EOQ
reorder_df, ... = calcular_tabla_reorden(...)

# 8. Graficas y exportacion de artefactos
...

return {...}

```

Otro punto de integracion relevante es que Thary tiene dos maneras de almacenar la informacion: la base de datos relacional (MySQL), para datos que sean estructurados como numeros o textos cortos y el sistema de archivos para archivos que por su naturaleza no se podria poner en una tabla, en el caso de Thary, estos archivos serian el archivo del modelo XGBoost ya entrenado, y las imagenes de las graficas. Cada vez que se procesa un CSV, en MySQL se agrega una fila nueva, nunca se borra nada, pero los archivos, es decir, las graficas, modelo entrenado, etc, van siempre a la misma carpeta de la empresa a la que pertenezcan, y cada vez que se procesa un nuevo archivo, se sobrescriben los anteriores. Asi que, si se hace una serie de corridas, en MySQL habra varias filas completas, pero en la carpeta, solo existiran los archivos de la ultima corrida.

app/routes/main.py:

Código 39. Función carpeta_resultados, que define la carpeta de resultados de cada empresa.

```

def carpeta_resultados(empresa_id):
    """Carpeta donde ejecutar_sistema escribe el pronostico, los
    graficos y demas archivos de resultado de una empresa puntual."""
    ruta = os.path.join(current_app.config["RESULTADOS_FOLDER_BASE"], e
    os.makedirs(ruta, exist_ok=True)
    return ruta

```

La ruta de la carpeta depende del id de la empresa (empresa_id), y no del id de la corrida

(`corrida_id`), por lo que, la carpeta siempre es la misma para una misma empresa, sin importar cuantas veces se haya procesado un archivo.

Por otra parte, tal como se explico en capitulos anteriores, se definio un control de acceso basado en roles, esto con el objetivo de poder diferenciar a que funcionalidades debe tener acceso un usuario dependiendo de su rol. El administrador (`admin`) debe ser el unico usuario que cuente con todos los privilegios de acceso a la informacion sensible de la organizacion, mientras que para otros usuarios con menor rango, (`empleado`), convendria restringir este acceso completo a solo aquellas funcionalidades menos invasivas que sean necesarias para el desempeño de su trabajo, como todas aquellas que describen el analisis de los datos y la lista de pedidos:

`app/services/auth.py`:

Código 40. Decorador `admin_required`, que restringe el acceso de una ruta al perfil administrador.

```
def admin_required(vista):
    @wraps(vista)
    def envoltura(*args, **kwargs):
        if "username" not in session:
            flash("Inicia sesión para continuar.")
            return redirect(url_for("auth.login"))
        if session.get("rol") != "admin":
            flash("Esa sección es solo para el administrador.")
            return redirect(url_for("main.dashboard"))
        return vista(*args, **kwargs)
    return envoltura
```

Integración continua (CI): El proyecto utilizo GitHub Actions para la automatizacion de pruebas unitarias (`.github/workflows/tests.yml`). Este se ejecuta automaticamente cada vez que se haga un *push* o *pull request* en el repositorio. El archivo cuenta con una configuracion que permite instalar tanto Python como todas las dependencias necesarias, y ejecuta las pruebas unitarias con `pytest`:

Código 41. Flujo de trabajo de GitHub Actions para ejecutar las pruebas automatizadas (tests.yml).

```
name: Tests

on:
  push:
  pull_request:

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"

      - run: pip install -r requirements.txt

      - run: pytest tests/ -q
```

El flujo solo cubre la parte de CI (Integración Continua), en cuanto a CD (Despliegue Continuo), se decidió hacerlo de manera manual, principalmente porque al ser un proyecto desarrollado por una sola persona, es decir, sin múltiples entornos ni un equipo que dependa de constantes despliegues, el beneficio de un pipeline de CD completo es menor frente a la complejidad adicional que implicaría configurarlo.

3.3.3 Despliegue

El despliegue del sistema Thary (además del entorno de desarrollo local), se encuentra en la nube mediante la plataforma Railway.

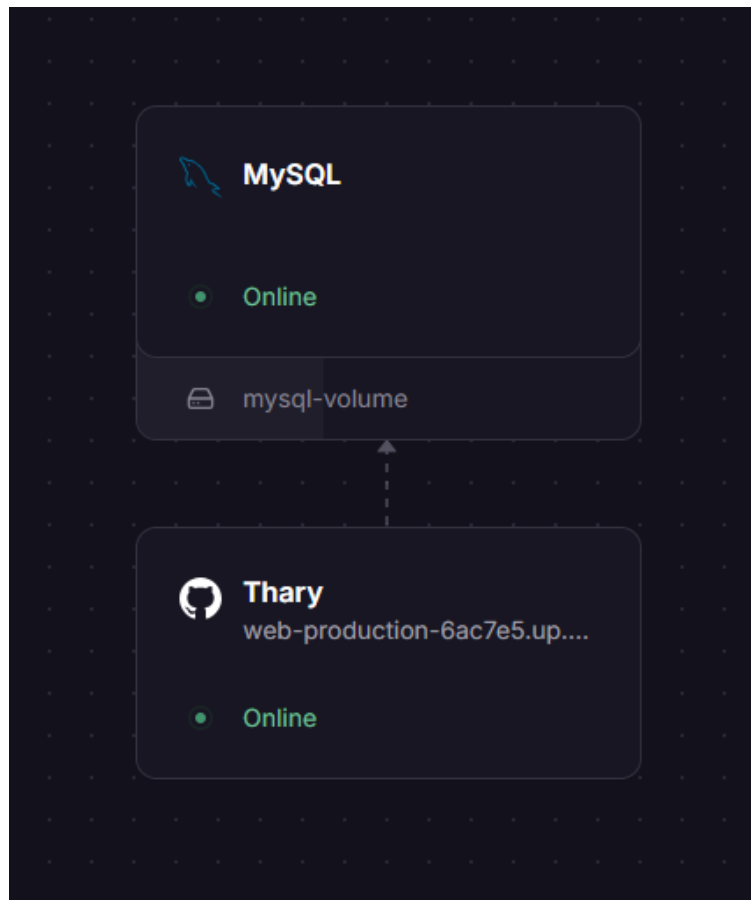


Fig. 27. Servicios desplegados en Railway: aplicación web (Thary) y base de datos MySQL.

Como se ve en la figura, Thary y la base de datos, se encuentran corriendo en Railway como servicios independientes, los cuales se comunican entre si para compartir información y funcionalidades. La aplicación se ejecuta mediante un servidor WSGI de producción (Gunicorn), configurado a través del archivo `Procfile`:

Código 42. Contenido del archivo `Procfile` para el servidor Gunicorn.

```
web: gunicorn run:app --bind 0.0.0.0:$PORT
```

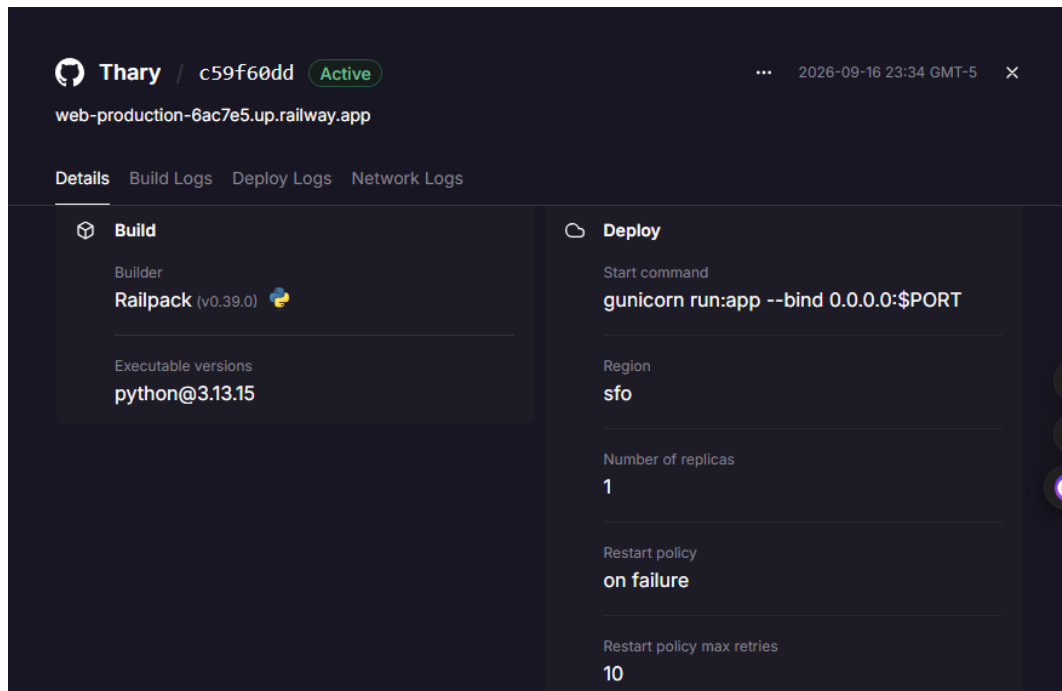



Fig. 28. Detalles de construcción y despliegue del servicio Thary en Railway.

Railway detecta y se encarga de ejecutar este comando *start command* cuando se realiza un despliegue. Automáticamente identifica la versión de python con la que se está trabajando, en este caso, `.python-version (3.13)`. El proyecto fija todas sus dependencias en `requirements.txt`, incluyendo las versiones exactas de las librerías de predicción (`pandas`, `scikit-learn`, `XGBoost`, `statsmodels`) y las necesarias para la conexión a la base de datos (`SQLAlchemy`, `PyMySQL`) y el servidor de producción (`gunicorn`).

En cuanto a la persistencia de los datos, esta se manejó mediante MySQL. La configuración de conexión a la base de datos solo se obtiene de las variables de entorno, tal como se detalló en la Sección 3.2.16 (función `_config()`).

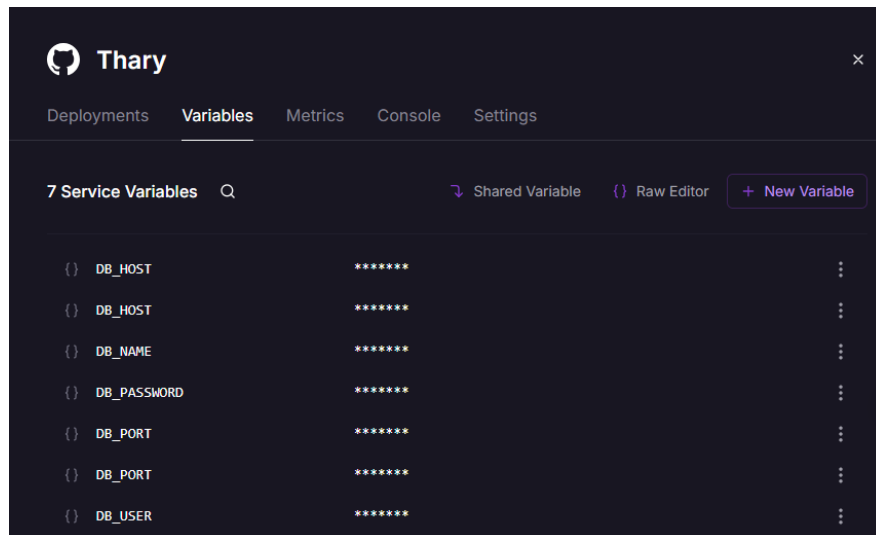


Fig. 29. Variables de entorno del servicio Thary en Railway, con los valores ocultos.

Las credenciales se configuran en el panel de Railway como variables de entorno del servicio, nunca visibles en el repositorio de código.

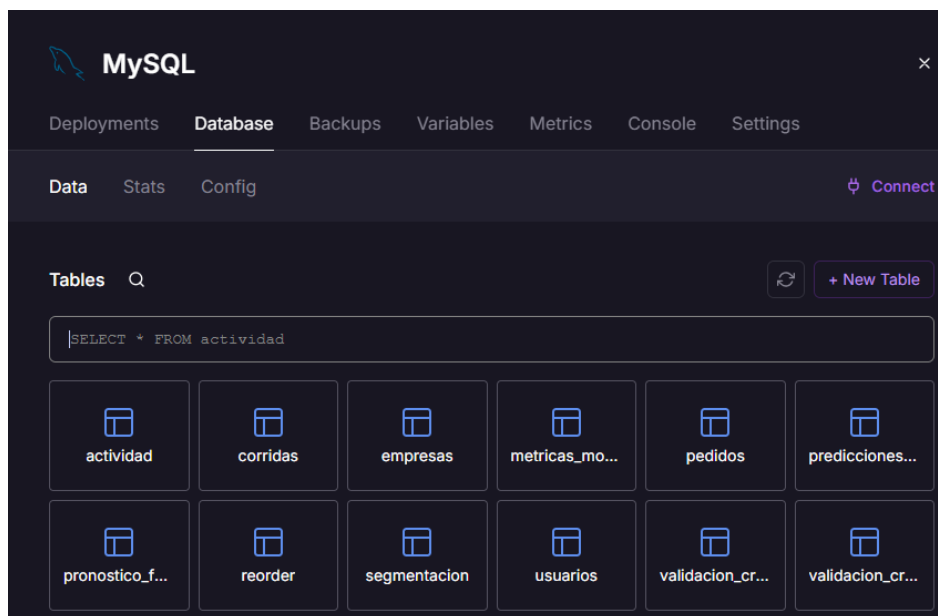


Fig. 30. Tablas del esquema relacional de Thary en el entorno de producción, vistas desde el explorador de datos de Railway.

La Fig. 30 muestra que el mismo esquema de 12 tablas que se tiene localmente, también existe en el entorno desplegado de Railway.

El servicio de Thary no cuenta con un volumen configurado, esto puede verse en el panel de

administración en el que no se ve la pestaña de "volumenes". Esto quiere decir que los archivos que están en el servicio son efímeros, es decir, cada vez que se redesplice el servicio, como por ejemplo, por una actualización del código o un reinicio del servicio, se crea un nuevo contenedor, por lo que se pierden por completo los archivos generados por el sistema en las corridas anteriores del pipeline. Esto se reconoce como una limitación del entorno de producción.

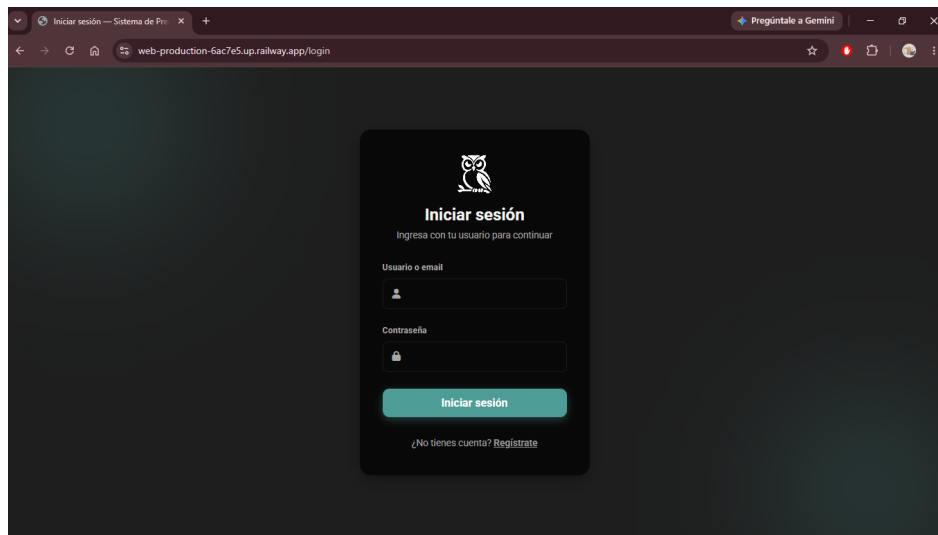


Fig. 31. Pantalla de inicio de sesión de Thary, accedida desde su URL pública en Railway.

Finalmente, la figura muestra la aplicación funcionando correctamente en el entorno de producción a través de la URL pública asignada por Railway.

3.4 Plan de Verificación y Validación

3.4.1 Estrategia de verificación

3.4.2 Estrategia de validación del modelo de predicción

3.4.3 Criterios de aceptación

3.4.4 Plan de validación con usuarios

4 Capítulo 4: Análisis de Resultados y Juicio de Ingeniería

4.1 Resultados de Verificación (Pruebas de Software)

La verificación del sistema de Thary, se llevó a cabo haciendo pruebas automatizadas escritas con *pytest*. Las pruebas se enfocaron en evaluar tres componentes principales

4.2 Resultados de Validación (Evaluación del Usuario)

4.2.1 Validación del modelo de predicción

A continuación, se presentan los resultados obtenidos en la validación del modelo de predicción. Estos resultados se obtuvieron luego de aplicar la estrategia de validación ya definida anteriormente, sobre el historial real de consumo de 856 productos (medicamentos) del año 2021 - 2022 en la Clínica Nuestra Señora de los remedios.

Las métricas obtenidas al evaluar los tres modelos candidatos, junto con el método de selección de modelo por producto. Todo esto sobre el último periodo de prueba disponible, que corresponde a junio 2022, con seis meses de entrenamiento previo.

TABLA VI
Métricas del holdout final por modelo.

Modelo	MAE	RMSE	MAPE (%)	WAPE (%)
Media Móvil (3 meses)	48.79	243.65	80.32	23.86
Regresión Lineal	56.43	276.41	95.30	27.60
XGBoost (log1p)	50.59	254.82	65.12	24.74
Sistema final (selección por producto)	39.60	207.14	57.63	19.37

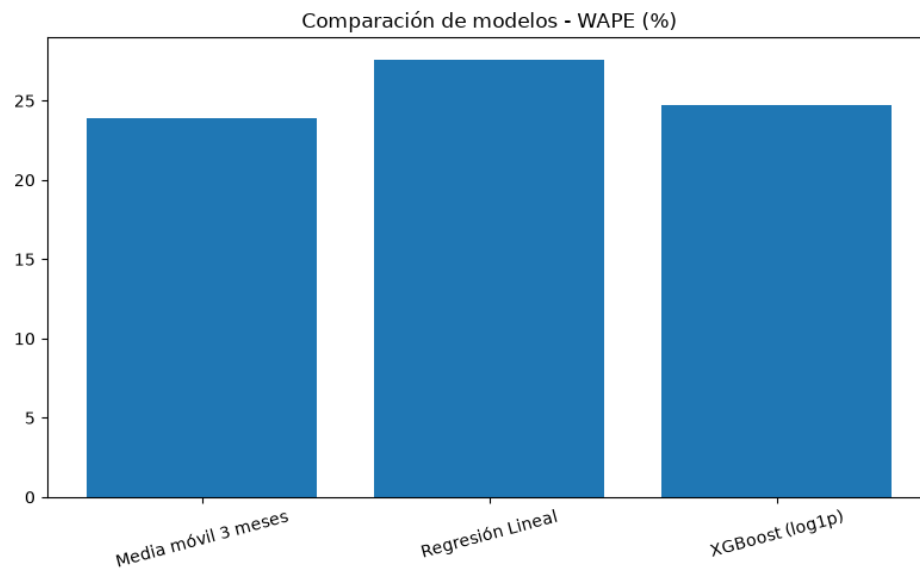


Fig. 32. Comparación de WAPE entre los tres modelos candidatos (holdout final).

4.2.2 Consistencia mediante validación cruzada temporal

Después de obtener los resultados anteriores, se propuso el uso de una validación cruzada para descartar que el resultado obtenido dependiera de las condiciones y particularidades de un único periodo de prueba.

La idea principal, es probar el modelo varias veces simulando estar en un mes distinto al pasado, para poder evaluar si acierta de forma consistente y no solo una vez por casualidad. Para poder predecir un mes, solo se puede usar los meses que vinieron antes, no se pueden usar los datos que serían "Futuros", porque queremos simular un escenario real, y en ese caso, no se tendría esa información.

1. Prueba 1 (Fold 1): Es el más antiguo, se le enseña al modelo con los primeros 4 meses del año (Enero, febrero, Marzo, Abril), y se hace que el sistema adivine Mayo, Junio y Julio. Se compara lo que adivino contra los resultados reales de esos meses y se registra que tan grande fue el error.
2. Prueba 2 (Fold 2): En esta segunda prueba o Fold, se le enseña al modelo con 5 meses en vez de 4, es decir, de enero a Mayo, y se le pide que prediga Junio, Julio y agosto, una vez más, se compara y se registra el error.
3. Prueba 3 (Fold 3): Por último, se le enseña al modelo con 6 meses de Enero a junio y se le solicita al sistema que prediga julio, agosto y septiembre, se compara y, una vez más, se registra el error.

Dicha validacion cruzada temporal se implemento en el sistema en el archivo `app/services/prediccion/validacion.py`.

Código 43. Constantes y función `_armar_folds` de la validación cruzada temporal.

```
TEST_SIZE_MESES = 3
MIN_TRAIN_MESES = 3
MAX_FOLDS = 3

def _armar_folds(meses_ordenados):
    n = len(meses_ordenados)
    folds = []
    fin_test = n
    while len(folds) < MAX_FOLDS:
        inicio_test = fin_test - TEST_SIZE_MESES
        if inicio_test < MIN_TRAIN_MESES:
            break
        folds.append((meses_ordenados[:inicio_test], meses_ordenados[inicio_test:fin_test]))
        fin_test -= 1
    folds.reverse()
    return folds
```

Se definen tres constantes, `TEST_SIZE_MESES`, `MIN_TRAIN_MESES` y `MAX_FOLDS`, que definen el tamaño de la ventana de prueba, el tamaño minimo de la ventana de entrenamiento y el numero maximo de folds a generar, respectivamente.

Se arma la funcion que genera las particiones `_armar_folds`, esta recibe una lista de todos los meses disponibles organizados cronologicamente. Se define el fold mas reciente, es decir, los ultimos meses disponibles `TEST_SIZE_MESES`. En cada iteracion se retrocede el punto de corte de un mes hacia atras en el tiempo, de esta manera, se crea un nuevo Fold cuyo entrenamiento crece en un mes respecto al anterior. Este proceso se repite hasta completar `MAX_FOLDS`, o hasta que ya no quede suficiente historial y el tamaño de la ventana de entrenamiento sea menor a `MIN_TRAIN_MESES` meses de entrenamiento.

Al finalizar, los folds se ordenan en orden cronologico del mas antiguo al mas nuevo antes de ser devueltos.

Al ya tener los folds organizados, el sistema entrena nuevamente los tres modelos candidatos, y selecciona el mejor modelo por producto, evaluando el resultado bajo la etiqueta de "Sistema final". Es decir, es la estrategia que usa Thary para predecir el consumo, en ves de escoger uno solo, para

cada producto, el sistema entrena los 3 modelos y selecciona el que se equivoque menos con ese producto en particular. El proceso se repite producto por producto.

Un aspecto importante es que, la elección de que modelo usar para cada producto se hace solo con los datos de entrenamiento, antes de ver los datos de prueba. Esto demuestra que no se está "haciendo trampa", ya que el sistema no tiene acceso a los datos de prueba, y por lo tanto, no puede "aprender" de ellos.

Aun así, solo esto no basta para confiar en el resultado, ya que aun si el sistema está eligiendo el modelo de forma honesta, un único experimento podría haber salido bien por casualidad y no porque el sistema de verdad sea bueno.

Es por eso que la validación cruzada aporta valor al sistema, ya que al repetir este mismo proceso (entrenamiento, elección del modelo y evaluación) en diferentes periodos de prueba, se confirma que el resultado obtenido en el holdout final no fue un resultado aislado, sino que se repite de forma consistente en distintos periodos de prueba.

En la siguiente tabla se describen el promedio y la desviación estándar de las métricas obtenidas en cada fold, para cada modelo candidato y el sistema final:

TABLA VII
Resumen de la validación cruzada temporal (promedio $\pm$ desviación, 3 folds).

Modelo	MAE	RMSE	MAPE (%)	WAPE (%)
Media Móvil	44.27 $\pm$ 3.70	200.82 $\pm$ 41.31	82.38 $\pm$ 1.85	22.31 $\pm$ 1.29
Regresión Lineal	58.13 $\pm$ 2.21	270.35 $\pm$ 6.32	106.24 $\pm$ 10.64	29.38 $\pm$ 1.84
XGBoost	46.58 $\pm$ 3.46	223.64 $\pm$ 31.40	64.93 $\pm$ 2.25	23.48 $\pm$ 1.15
Sistema final	36.25 $\pm$ 2.54	174.24 $\pm$ 27.89	59.13 $\pm$ 1.84	18.27 $\pm$ 0.81

De la tabla se puede observar que:

De la tabla se puede observar que:

- El sistema final es el que obtiene el mejor promedio en las cuatro métricas evaluadas, y también la menor desviación estándar en todas ellas. Esto es significativo, porque demuestra que vale la pena la complejidad de elegir un modelo distinto para cada producto: el sistema final no solo es más preciso en promedio, sino también más consistente y estable entre los distintos periodos evaluados.
- La media móvil es el segundo mejor modelo, seguido por XGBoost y finalmente la regresión lineal. Cuando se comparan los tres modelos candidatos de forma individual, la media móvil es la que obtiene un mejor MAE, RMSE y WAPE, mientras que solo XGBoost la supera en MAPE. El hecho de que ninguno de los dos modelos más sofisticados domine de forma

general sobre la línea base avala la decisión de seleccionar un modelo distinto por producto, ya que no existe un único modelo que sea el mejor en todos los casos.

- La Regresión Lineal es la que tiene los peores resultados en las cuatro métricas, además de que su MAPE es el que más varía entre los 3 folds. Esto es coherente con la tendencia de este modelo a sobreestimar los productos de alta demanda, observada previamente en el análisis de dispersión entre modelos, lo que ayuda a explicar por qué su error resulta más alto que el de los otros dos modelos.
- Finalmente, se observa que la efectividad del sistema final no se debe solamente a "elegir el mejor modelo en general", que en este caso sería la media móvil, sino a elegir un modelo distinto para cada producto. En caso de haber elegido media móvil para todos los productos, el error hubiera sido de 22.31% de WAPE, mientras que al elegir el mejor modelo por producto, el error se reduce a 18.27%.

4.3 Evaluación de la Seguridad y Estándares

4.4 Análisis de Impactos de la Solución

4.5 Conclusiones

4.6 Recomendaciones de Operación y Mantenibilidad

4.7 Trabajos Futuros

4.8 Declaración de Uso Responsable de IA

Referencias Bibliográficas

- [1] P. MEANA COALLA, *Gestión de inventarios* (Comercio y Marketing). Ediciones Paraninfo, S.A, 2024, ISBN: 9788428363570. [Internet]. available from <https://books.google.com.co/books?id=RfgyEQAAQBAJ>.
- [2] IBM, (n.d.). “¿Qué es la gestión de inventario?”, IBM Think. [Accessed: Aug. 5, 2026]. [Internet]. available from <https://www.ibm.com/mx-es/think/topics/inventory-management>.
- [3] Teaganne Finn , Ian Smalley, (n.d.). “¿Qué es la previsión de demanda?”, IBM Think. [Accessed: Aug. 5, 2026]. [Internet]. available from <https://www.ibm.com/es-es/think/topics/demand-forecasting>.
- [4] M. Valencia Cárdenas, F. J. Díaz Serna, and J. C. Correa Morales, “Inventory planning with dynamic demand. A state of art review”, DYNA, vol. 82, no. 190, pp. 182–191, Mar. 2015. doi: 10.15446/dyna.v82n190.42828 [Internet]. available from <https://revistas.unal.edu.co/index.php/dyna/article/view/42828>.
- [5] S. C. S. Fernando and O. B. C. Camilo, *Análisis de series temporales para predicción de demanda de productos*, 2024-11-05. [Internet]. available from <https://repository.unad.edu.co/handle/10596/69801>.
- [6] X. Wen and W. Li, “Time series prediction based on lstm-attention-lstm model”, IEEE Access, vol. 11, pp. 48 322–48 331, 2023. doi: 10.1109/ACCESS.2023.3276628
- [7] F. M. Salem, “Gated rnn: The gated recurrent unit (gru) rnn”, pp. 85–100, 2022. doi: 10.1007/978-3-030-89929-5_5 [Internet]. available from https://doi.org/10.1007/978-3-030-89929-5_5.
- [8] I. M. Peláez, “Modelos de regresión: Lineal simple y regresión logística”, Revista Seden, vol. 14, pp. 195–214, 2016.
- [9] X. Ren, H. Guo, S. Li, S. Wang, and J. Li, “A novel image classification method with cnn-xgboost model”, in *International workshop on digital watermarking*, Springer, 2017, pp. 378–390.
- [10] D. Shah, *Demand Forecasting & Inventory Optimization for Consumer Products*, GitHub repository, Disponible en: <https://github.com/dhruvi002/demand-forecast-inventory-optimizer>, 2026.
- [11] S. Saci, (Aug. 2020). “Machine Learning for Retail Demand Forecasting: Comparative Study of Demand Forecasting Methods for a Retail Store (XGBoost Model vs. Rolling Mean)”. Disponible en: https://www.samirsaci.com/inventory-management-for-retail-stochastic-demand-2/?source=post_page-----743b2f4e4057.

-
- [12] ksmooi, *Forecasting inventory demand using xgboost*, <https://www.kaggle.com/code/ksmooi/inventory-demand-forecast-xgboost-reg>, Kaggle Notebook, n.d.
- [13] Tushar Verma, *Linear regression: Demand forecasting*, Disponible en: https://github.com/21-f3000932/LinearRegression_Demand-forecasting/tree/main, n.d.
- [14] F. Diebold and R. Mariano, “Comparing predictive accuracy”, *Journal of Business & Economic Statistics*, vol. 20, pp. 134–44, Feb. 2002, Disponible en version working paper: https://www.researchgate.net/publication/4722069_Comparing_Predictive_Accuracy/citation/download. doi: 10.1080/07350015.1995.10524599
- [15] F. X. Diebold, “Comparing predictive accuracy, twenty years later: A personal perspective on the use and abuse of diebold-mariano tests”, *Journal of Business & Economic Statistics*, vol. 33, no. 1, pp. 1–9, 2015, Disponible en version working paper: <http://www.nber.org/papers/w18391.pdf>. doi: 10.1080/07350015.2014.983236
- [16] S. Ben Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, “A review and comparison of strategies for multi-step ahead time series forecasting based on the nn5 forecasting competition”, *Expert Systems with Applications*, vol. 39, no. 8, pp. 7067–7083, 2012, issn: 0957-4174. doi: <https://doi.org/10.1016/j.eswa.2012.01.039> [Internet]. available from <https://www.sciencedirect.com/science/article/pii/S0957417412000528>.

ANEXOS

Anexo A Documento completo SRS (historias de usuario / backlog)

TABLA VIII

Especificación del caso de uso CU-01: Iniciar sesión

Elemento	Especificación
ID	CU-01
Nombre	Iniciar sesión
Actor principal	Usuario registrado
Requisitos relacionados	RF-10
Objetivo	El objetivo es permitir que los usuarios registrados accedan al sistema utilizando sus credenciales.
Precondiciones	Para poder iniciar sesión, el usuario debe encontrarse previamente registrado en el sistema.
Postcondiciones	El usuario accede exitosamente al sistema y puede acceder a las funcionalidades a las que tenga acceso dependiendo de su rol.
Flujo principal	1. El usuario ingresa su usuario y contraseña. 2. El sistema valida que la información sea correcta. 3. El sistema identifica el perfil del usuario. 4. El sistema permite el acceso a Thary.
Flujos alternativos	1. Si el usuario o la contraseña introducidos son incorrectas, el sistema le informa al usuario que los datos ingresados no son válidos. 2. Si el usuario no está registrado, el sistema rechaza el acceso y el usuario tiene la oportunidad de registrarse.

TABLA IX
Especificación del caso de uso CU-02: Cargar historial de consumo

Elemento	Especificación
ID	CU-02
Nombre	Cargar historial de consumo
Actor principal	Administrador
Requisitos relacionados	RF-01, RF-07
Objetivo	El objetivo del caso de uso es permitirle al Administrador poder cargar un archivo CSV que contenga el historial de consumo de los productos del inventario.
Precondiciones	• Para acceder a esta funcionalidad, el usuario debe haber iniciado sesión con perfil de Administrador. • El archivo debe subirse en formato de CSV.
Postcondiciones	El archivo es recibido por el sistema para su posterior procesamiento
Flujo principal	1. El Administrador inicia sesión y es recibido por la pantalla inicial que le da la opción de subir un archivo csv. 2. El Administrador selecciona un archivo CSV. 3. El sistema recibe el archivo. 4. El sistema se encarga de validar el formato y que contenga los campos mínimos requeridos para el funcionamiento del sistema como: fecha, ID del producto y cantidad demandada de cada uno. 5. El sistema informa que el archivo fue cargado correctamente.
Flujos alternativos	1. Si el archivo no tiene formato CSV, el sistema informa el error con el mensaje "El archivo debe tener extensión .csv" 2. Si faltan campos obligatorios, el sistema rechaza el archivo e informa cuáles son los datos requeridos, ejemplo: Falta la columna obligatoria: fecha" 3. Si el usuario no cuenta con los permisos de Administrador, Thary simplemente no le muestra la opción de carga archivo, limitándose con solo mostrarle el dashboard con los resultados y demás pantallas con información.

TABLA X
Especificación del caso de uso CU-03: Procesar archivo

Elemento	Especificación
ID	CU-03
Nombre	Procesar archivo
Actor principal	Administrador
Requisitos relacionados	RF-02, RF-07
Objetivo	Iniciar el procesamiento del historial de consumo previamente cargado para generar las estimaciones de demanda.
Precondiciones	• El Administrador debe haber iniciado sesión. • Debe existir un archivo de historial cargado y válido.
Postcondiciones	El sistema procesa los datos, ejecuta el proceso de predicción y genera los resultados de demanda para los productos.
Flujo principal	1. El Administrador accede al panel de procesamiento. 2. El sistema verifica que exista un archivo válido. 3. El Administrador selecciona la opción “Procesar archivo”. 4. El sistema procesa y prepara los datos. 5. El sistema ejecuta el proceso de predicción. 6. El sistema genera las estimaciones de demanda mensual para los productos. 7. El sistema genera los resultados necesarios para las etapas posteriores.
Flujos alternativos	1. Si no existe un archivo válido, el sistema informa al Administrador que debe cargar un historial. 2. Si ocurre un error durante el procesamiento, el sistema informa que no fue posible completar la operación. 3. Si el usuario no es Administrador, el sistema impide la ejecución.

TABLA XI
Especificación del caso de uso CU-04: Consultar demanda pronosticada

Elemento	Especificación
ID	CU-04
Nombre	Consultar demanda pronosticada
Actor principal	Personal de Inventario
Requisitos relacionados	RF-02, RF-05
Objetivo	Permitirle al los empleados del inventario consultar cual es la demanda mensual estimada para cada uno de los producto.
Precondiciones	Como precondicion, ya debe de existir un archivo procesado anteriormente por el administrador, tambien ya deben haber resultados de predicción disponibles.
Postcondiciones	El usuario visualiza las estimaciones de demanda correspondientes a los productos.
Flujo principal	1. El usuario accede al dashboard con las predicciones. 2. El sistema consulta los resultados generados. 3. El sistema le muestra al usuario, sin importar si es administrador o usuario, la demanda pronosticada de los productos. 4. El usuario consulta la información presentada.
Flujos alternativos	Si no existen resultados de predicción, el sistema informa que primero debe realizarse el procesamiento del archivo mediante el siguiente mensaje "Todavía no hay datos cargados. En cuanto tu administrador suba un archivo, vas a poder ver el pronóstico, el inventario y confirmar pedidos."

TABLA XII
Especificación del caso de uso CU-05: Consultar estado del inventario

Elemento	Especificación
ID	CU-05
Nombre	Consultar estado del inventario
Actor principal	Personal de Inventario
Requisitos relacionados	RF-04, RF-05
Objetivo	El objetivo es que se pueda consultar el inventario actual de los productos e identificar fácilmente a aquellos que requieran atención o esten en mayor riesgo de desabatecimiento.
Precondiciones	Como precondicion para este caso de uso, es importante que ya exista información del inventario disponible y resultados del procesamiento.
Postcondiciones	Las postcondiciones son que el usuario puede visualizar sin problemas el estado actual del inventario y las alertas correspondientes para cada producto.
Flujo principal	1. El usuario accede al módulo de "Estado de inventario". 2. El sistema consulta la informacion actual del inventario. 3. El sistema se encarga de comparar el inventario disponible con el punto de reposición de cada producto. 4. El sistema identifica los productos que se encuentran en o por debajo de su punto de reposición. 5. El sistema muestra las alertas visuales correspondientes dependiendo de cada caso, se marcaran de color rojo en caso de estar por debajo del punto de reposicion.
Flujos alternativos	Si no existe información de inventario para un producto, el sistema informa que no se dispone de los datos necesarios para determinar su estado.

TABLA XIII

Especificación del caso de uso CU-06: Consultar recomendaciones de reposición y presupuesto

Elemento	Especificación
ID	CU-06
Nombre	Consultar recomendaciones de reposición y presupuesto
Actor principal	Personal de Inventario
Requisitos relacionados	RF-03, RF-05
Objetivo	Permitir consultar las recomendaciones de reposición generadas para cada producto, teniendo en cuenta distintos datos como la demanda estimada, el inventario que ya se tiene disponible, el tiempo de entrega, que varía de producto a producto, y el presupuesto establecido.
Precondiciones	La principal precondición es que debe existir un archivo csv ya procesado, y el sistema ya debe contar con toda la información necesaria para poder calcular las recomendaciones.
Postcondiciones	El usuario visualiza la cantidad recomendada de compra de una manera clara mediante el uso de gráficas y tablas, también puede ver otros datos relevantes como el punto de reposición y información acerca del presupuesto.
Flujo principal	1. El usuario accede al módulo de recomendaciones, es decir, el dashboard principal. 2. El sistema consulta las estimaciones de demanda. 3. El sistema obtiene el inventario actual y los parámetros de reposición. 4. El sistema considera el tiempo de entrega del proveedor y las restricciones presupuestales. 5. El sistema calcula la recomendación de reposición. 6. El sistema le muestra los resultados al usuario.
Flujos alternativos	Si no existen datos suficientes para realizar el cálculo, el sistema informa qué información se encuentra pendiente.

TABLA XIV
Especificación del caso de uso CU-07: Registrar pedido

Elemento	Especificación
ID	CU-07
Nombre	Registrar pedido
Actor principal	Personal de Inventario
Requisitos relacionados	RF-06
Objetivo	El objetivo es permitirle al Personal de Inventario registrar un pedido de reposición a partir de las recomendaciones generadas.
Precondiciones	El usuario debe haber iniciado sesión y debe existir información del producto y de la cantidad a solicitar.
Postcondiciones	El pedido queda registrado con la información correspondiente y asociado con el nombre del usuario responsable.
Flujo principal	1. El usuario consulta la pagina de "Pedidos" del sistema. 2. Selecciona uno de los pedidos para registrarlos. 3. Selecciona el proveedor. 4. A partir de esto, el sistema registra la fecha del pedido. 5. El sistema identifica al usuario responsable. 6. El sistema guarda la información del pedido. 7. El sistema confirma el registro.
Flujos alternativos	Si no se selcciona ningun producto, pero de todas maneras se hace click en el boton para registrar, saldra el siguiente mensaje: No seleccionaste ningún producto para confirmar"

TABLA XV
Especificación del caso de uso CU-08: Gestionar usuarios

Elemento	Especificación
ID	CU-08
Nombre	Gestionar usuarios
Actor principal	Administrador
Requisitos relacionados	RF-08
Objetivo	Permitirle al Administrador poder crear cuentas para sus empleados con menos privilegios a los de un administrador.
Precondiciones	El usuario debe haber iniciado sesión con perfil de Administrador.
Postcondiciones	El nuevo empleado queda registrado y puede utilizar el sistema con permisos restringidos.
Flujo principal	1. El Administrador accede a la gestión de usuarios. 2. Selecciona la opción para crear un usuario. 3. Ingresa la información requerida. 4. Define el perfil o nivel de privilegios. 5. El sistema valida la información. 6. El sistema registra al nuevo usuario.
Flujos alternativos	1. Si la información es inválida o incompleta, el sistema solicita corregirla mediante un mensaje de error. 2. Si el usuario ya existe, el sistema informa que no puede crearse una cuenta duplicada.

TABLA XVI
Especificación del caso de uso CU-09: Deshacer pedido

Elemento	Especificación
ID	CU-09
Nombre	Deshacer pedido
Actor principal	Personal de Inventario
Requisitos relacionados	RF-10
Objetivo	El objetivo es permitirle al Personal de Inventario revertir un pedido previamente confirmado, en caso de haberse equivocado o de que la confirmacion ya no sea necesaria.
Precondiciones	• El usuario debe haber iniciado sesion. • Debe existir al menos un pedido previamente confirmado por el mismo usuario.
Postcondiciones	El pedido queda marcado como no confirmado, y vuelve a aparecer entre las recomendaciones pendientes de confirmar.
Flujo principal	1. El usuario accede a la pagina de "Pedidos" del sistema. 2. Consulta el historial de pedidos confirmados. 3. Selecciona el pedido que desea deshacer. 4. El sistema solicita confirmacion de la accion. 5. El usuario confirma que desea deshacer el pedido. 6. El sistema revierte el estado del pedido y lo registra en la bitacora de actividad.
Flujos alternativos	1. Si el usuario cancela la confirmacion, el sistema no realiza ningun cambio y el pedido permanece confirmado. 2. Si el pedido ya fue deshecho previamente por otro usuario, el sistema informa que la accion ya no es posible.

TABLA XVII
Especificación del caso de uso CU-10: Consultar actividad de la empresa

Elemento	Especificación
ID	CU-10
Nombre	Consultar actividad de la empresa
Actor principal	Administrador
Requisitos relacionados	RF-11
Objetivo	Permitirle al Administrador consultar la bitacora completa de acciones realizadas por todos los usuarios de su empresa, con fines de auditoria.
Precondiciones	• El usuario debe haber iniciado sesion con perfil de Administrador. • Debe existir al menos un registro de actividad generado previamente en la empresa.
Postcondiciones	El administrador visualiza el listado de acciones realizadas por los usuarios de su empresa, ordenadas cronologicamente.
Flujo principal	1. El Administrador accede al modulo de "Actividad". 2. El sistema consulta el registro de actividad de la empresa del administrador. 3. El sistema muestra el listado de acciones, indicando el usuario responsable, el tipo de accion y la fecha y hora en que ocurrio.
Flujos alternativos	Si aun no se ha registrado ninguna actividad en la empresa, el sistema informa que no hay eventos para mostrar.

TABLA XVIII
Especificación del caso de uso CU-11: Consultar calendario de pedidos

Elemento	Especificación
ID	CU-11
Nombre	Consultar calendario de pedidos
Actor principal	Personal de Inventario
Requisitos relacionados	RF-12
Objetivo	Permitirle al usuario visualizar en un calendario las fechas estimadas en las que debe realizarse el pedido de cada producto, según las recomendaciones generadas por el sistema.
Precondiciones	Debe existir un archivo previamente procesado, con resultados de reposición disponibles.
Postcondiciones	El usuario visualiza, organizadas por fecha, las recomendaciones de pedido próximas a vencer.
Flujo principal	1. El usuario accede al módulo de "Calendario de pedidos". 2. El sistema consulta las recomendaciones de reposición vigentes. 3. El sistema organiza dichas recomendaciones según la fecha estimada en la que deberían realizarse. 4. El sistema muestra el calendario con los pedidos próximos, por producto.
Flujos alternativos	Si no existen recomendaciones de reposición disponibles, el sistema informa que primero debe procesarse un archivo.

TABLA XIX
Especificación del caso de uso CU-12: Consultar perfil

Elemento	Especificación
ID	CU-12
Nombre	Consultar perfil
Actor principal	Usuario registrado
Requisitos relacionados	RF-13
Objetivo	Permitirle a cualquier usuario, sin importar su rol, consultar los datos de su propia cuenta y el historial de las acciones que el mismo ha realizado en el sistema.
Precondiciones	El usuario debe haber iniciado sesion.
Postcondiciones	El usuario visualiza la informacion de su cuenta y sus propios movimientos dentro del sistema.
Flujo principal	1. El usuario accede al modulo de "Perfil". 2. El sistema consulta los datos de la cuenta del usuario. 3. El sistema consulta los movimientos propios del usuario (pedidos confirmados o revertidos, y si es administrador, tambien sus archivos cargados y los usuarios que ha creado). 4. El sistema muestra la informacion recopilada.
Flujos alternativos	Si el usuario aun no ha realizado ninguna accion registrada en el sistema, el sistema muestra los datos de la cuenta sin movimientos asociados.

Anexo B Modelos de datos extensos, diccionarios de datos, mockups

Anexo C Manual de usuario y/o guía de instalación y despliegue

Anexo D Enlaces a repositorios de código fuente y tableros de gestión (Azure DevOps, Jira, GitHub)

Anexo E Protocolo y seguimiento de la revisión sistemática de literatura (si aplica)